

Graph Algorithms

Community Analysis

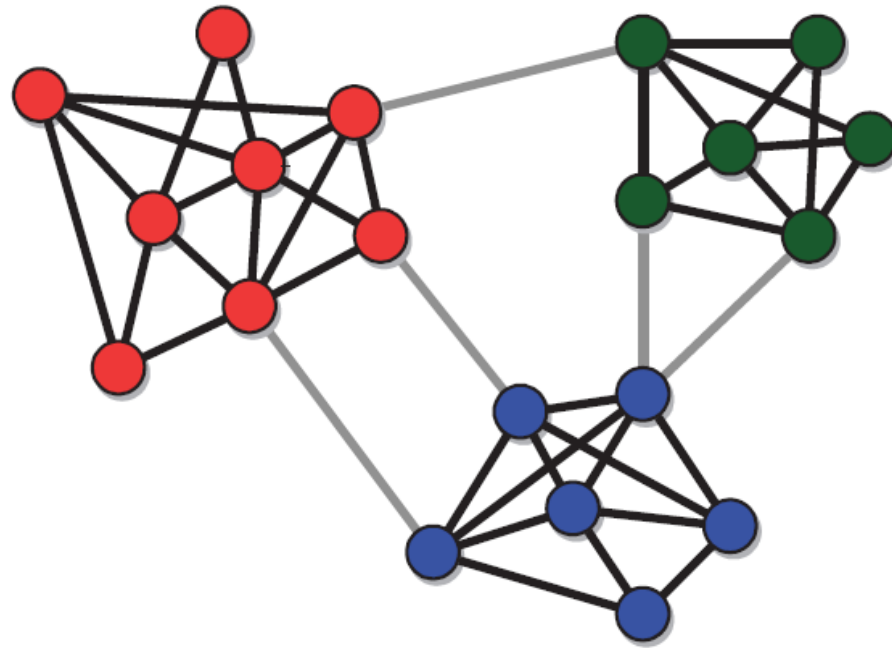
Spring 2026

Dr. Avner Priel

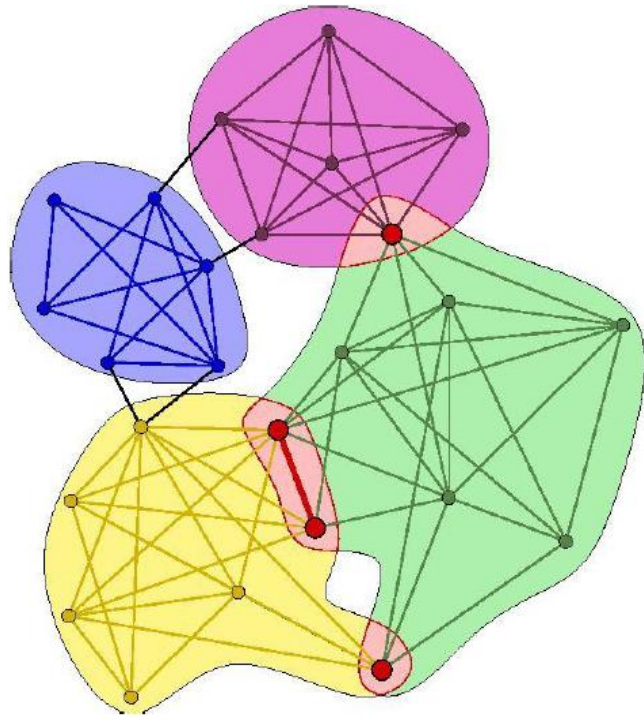
avnerp@ruppin.ac.il

Finding communities – an example

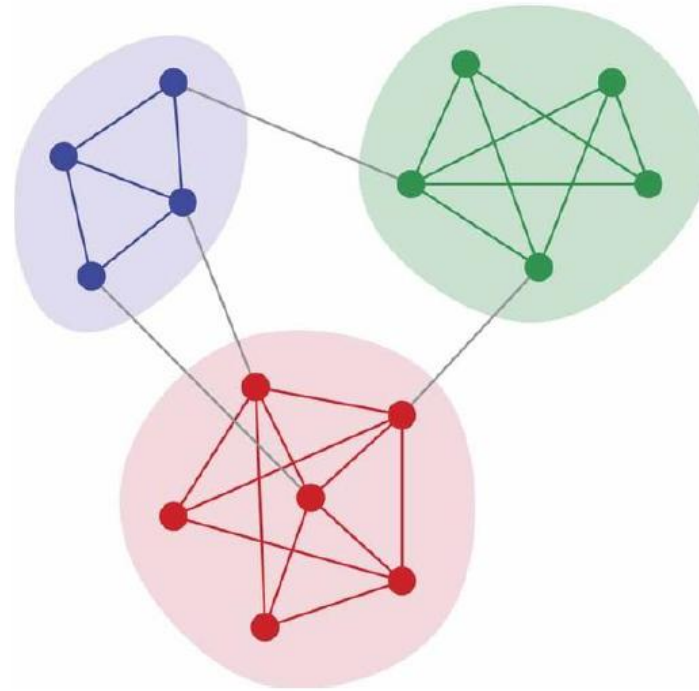
- A simple graph in which three implicit communities are found, indicated by node colors



Overlapping vs. Disjoint Communities



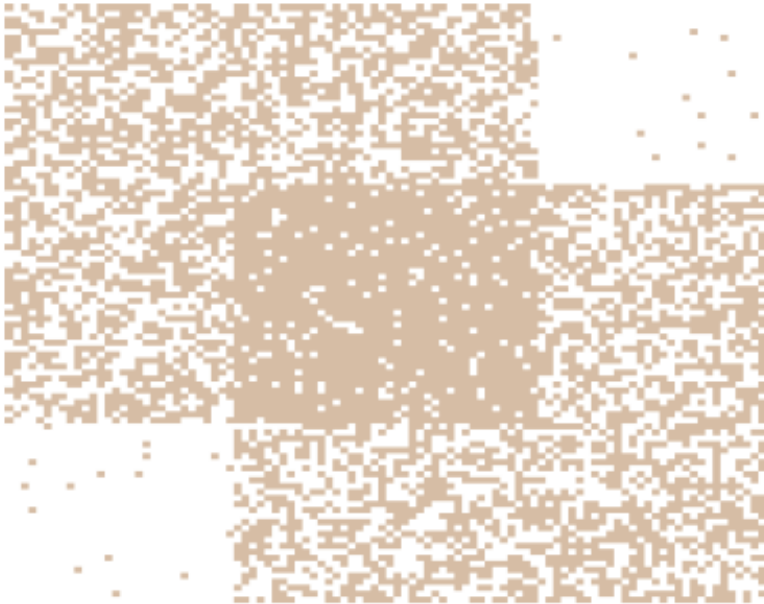
Overlapping Communities



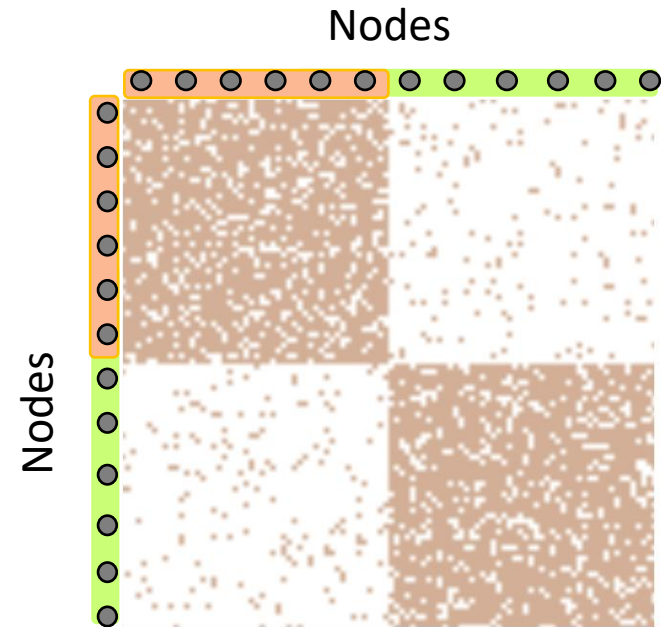
Disjoint Communities

Overlapping vs. Disjoint Communities

- Adjacency matrix



Overlapping



Disjoint (non-overlapping)

Communities are found in many domains

Protein-protein interaction networks

- Communities are likely to group proteins having the same specific function within the cell

World Wide Web

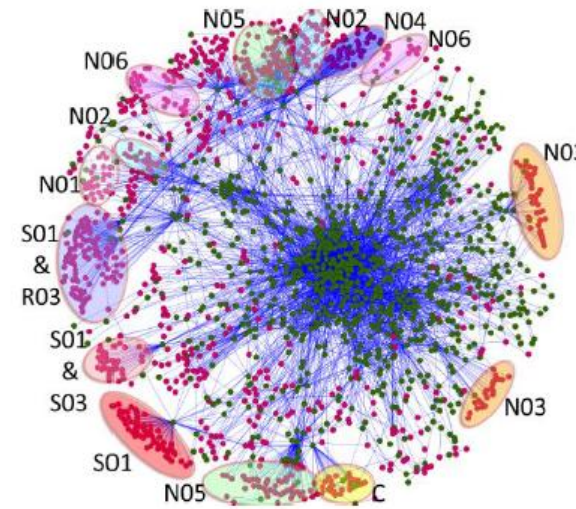
- Communities may correspond to groups of pages dealing with the same or related topics

Metabolic networks

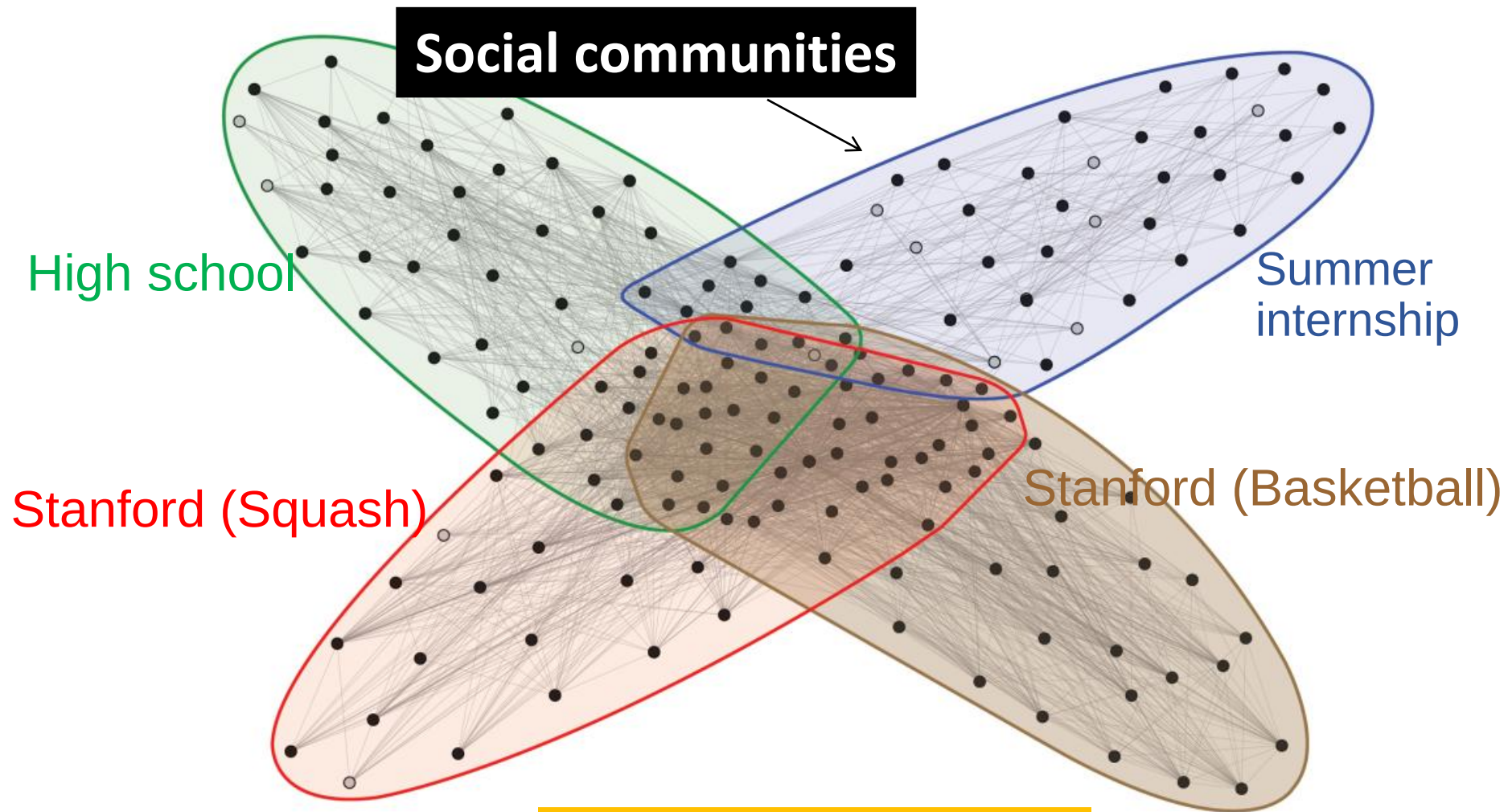
- Communities may be related to functional modules such as cycles and pathways

Food webs

- Communities may identify compartments

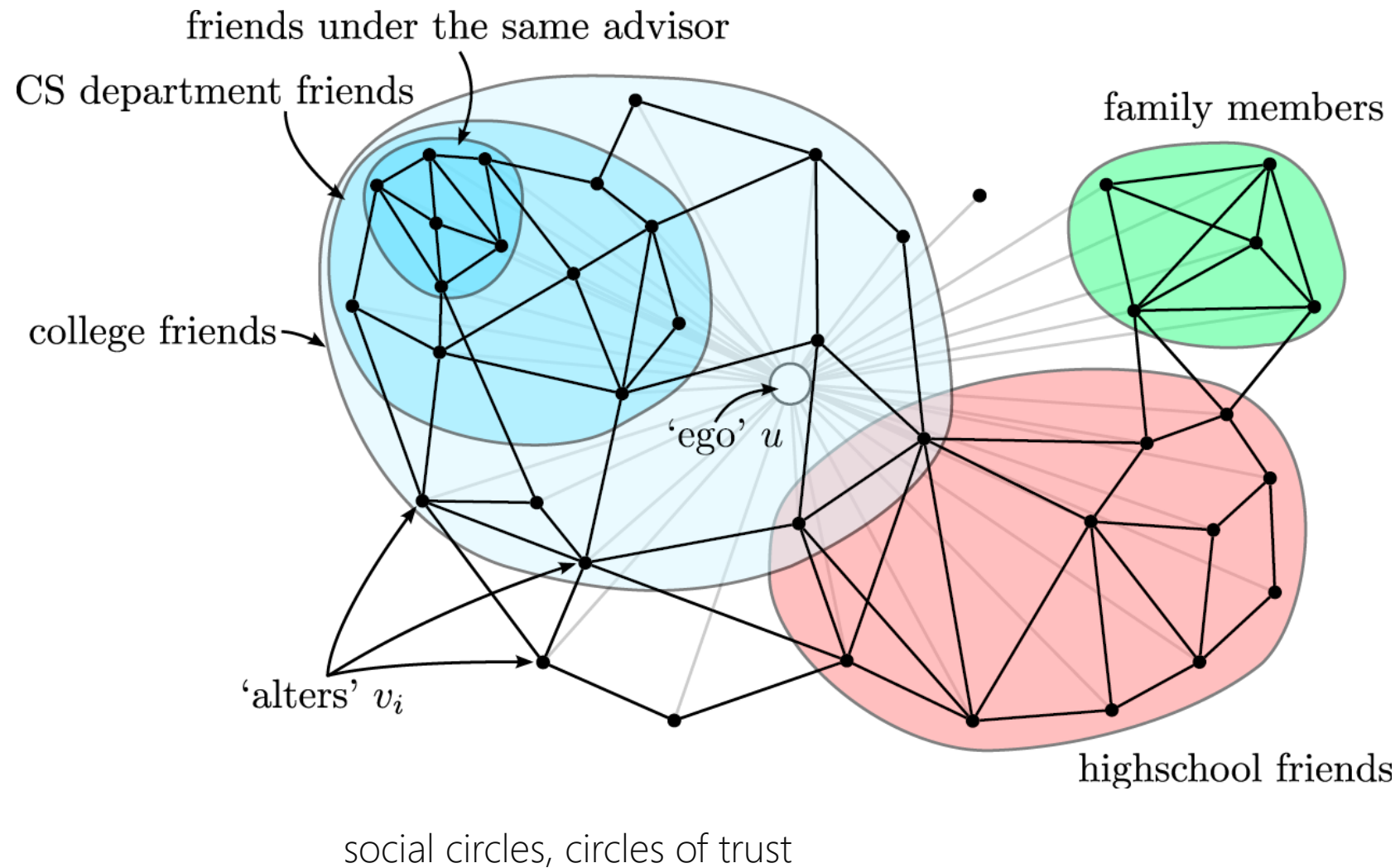


Facebook Network



Twitter & Facebook

הכלה



Why (analyzing communities) ?

- **Knowledge discovery**
- **Groups based on common interests, behavior, etc.**
 - **Recommendations, marketing**
- **Collective behavior** (observable at the group, not the individual level, local view is noisy and ad hoc)
- **Performance-wise** (partition a large graph into many machines → parallelization)
- **Classification of the nodes** (by identifying modules)
- **Summary, visual representation** of the graph

What is community analysis?

- **Community detection**
 - Discovering implicit communities
- **Community evolution**
 - Studying temporal evolution of communities
- **Community evaluation**
 - Evaluating the detected communities (how good are they?)

Community Detection

Given a graph $G(V, E)$, find subsets C_i of V , such that $\cup_i C_i \subseteq V$

- Edges can also represent content or attributes shared by individuals (in the same location, of the same gender, etc.)
- Undirected graphs
- Unweighted (easily extended)
- Attributed, or labeled graphs

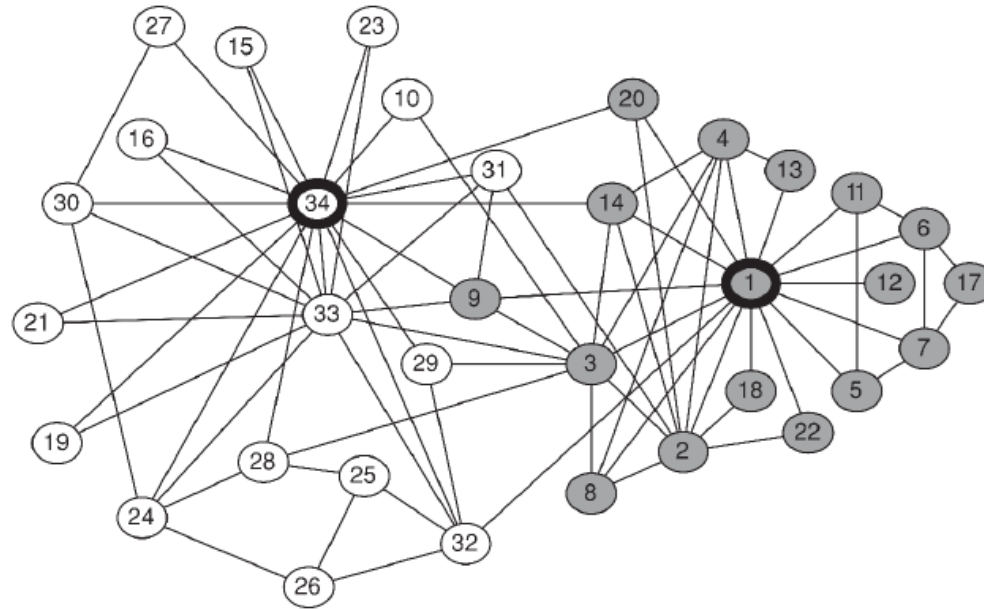
So what is Community Detection?

- The process of finding clusters of nodes (*"communities"*)
 - With **Strong** internal connections and
 - **Weak** connections between different communities
- Ideal decomposition of a large graph
 - Completely disjoint communities
 - There are no interactions between different communities.
- In practice,
 - find community partitions that are maximally decoupled.

Famous example

Zachary's karate club

Interactions between
34 members of a
karate club
for over two years



- The club members split into two groups (**gray** and **white**)
- Disagreement between the administrator of the club (node **34**) and the club's instructor (node **1**),
- The members of one group left to start their own club

**The same communities can be
found using community detection**

Taxonomy of Community Criteria

- Criteria vary depending on the tasks
- Roughly, the methods can be divided into 4 (not exclusive) categories:
 - **Node**-Centric Community
 - **Each node** in a group satisfies certain properties
 - **Group**-Centric Community
 - Consider the connections **within a group** as a whole. The group has to satisfy certain properties without zooming into node-level
 - **Network**-Centric Community
 - Partition **the whole network** into several disjoint sets
 - **Hierarchy**-Centric Community
 - Construct a **hierarchical structure** of communities

Node-based Community Detection Methods

- Node degree (familiarity)
 - cliques
- Node similarity
 - Cluster(ing)
- Node reachability
 - Betweenness
-

Community detection vs. Clustering (ML)

Clustering

- Data is often non-linked (matrix rows)
- Clustering works on the distance or similarity matrix, e.g., k -means.
- If you use k -means with adjacency matrix rows, you are only considering the ego-centric network

Nearest
Neighbors

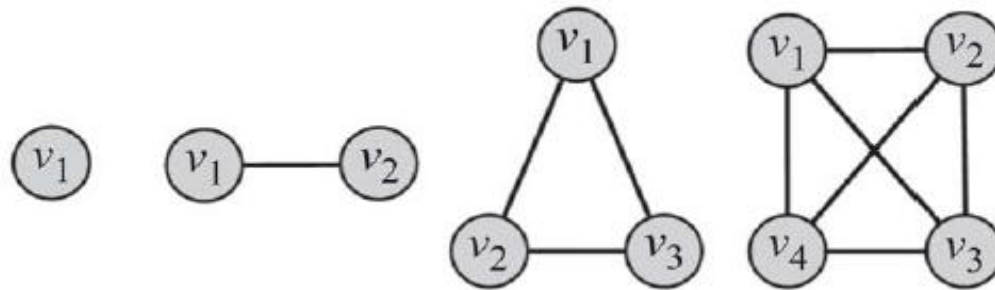
Community detection

- Data is linked (a graph)
- Network data tends to be “discrete”, leading to algorithms using the graph property directly
 - k -clique, quasi-clique, or edge-betweenness

Cliques (degree similarity)

Clique: a maximum *complete subgraph* in which all pairs of vertices are connected by an edge.

A *clique of size k* is a subgraph of k vertices where the degree of **all vertices** in the induced subgraph is $k - 1$.

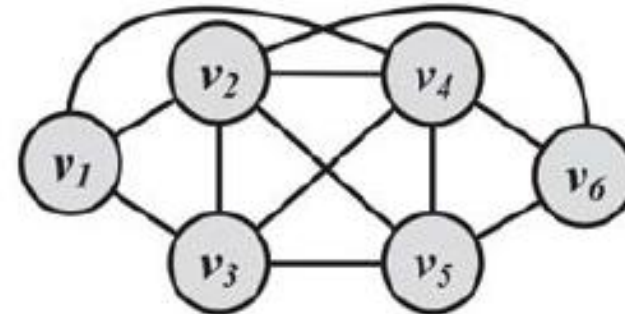
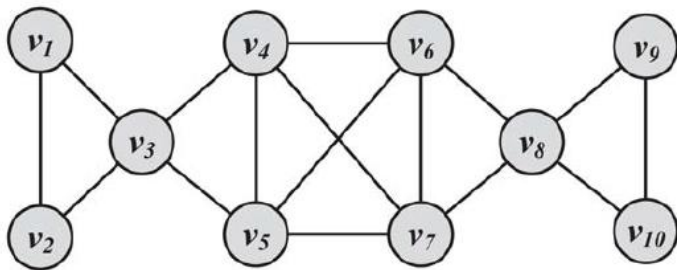


Cliques (degree similarity)

Search for:

- the *maximum clique* (the one with the largest number of vertices) or
- all *maximal cliques* (cliques that are not subgraphs of a larger clique; i.e., cannot be expanded further).
- A maximum clique is always maximal, but the converse does not hold

Both problems are **NP-hard**, as is verifying whether a graph contains a clique larger than size k .



Cliques algorithm

Algorithm Brute-Force Clique Identification

Require: Adjacency Matrix A , Vertex v_x

```
1: return Maximal Clique  $C$  containing  $v_x$ 
2: CliqueStack =  $\{\{v_x\}\}$ , Processed =  $\{\}$ ;
3: while CliqueStack not empty do
4:    $C = \text{pop}(\text{CliqueStack})$ ; push(Processed,  $C$ );
5:    $v_{last} = \text{Last node added to } C$ ;
6:    $N(v_{last}) = \{v_i | A_{v_{last}, v_i} = 1\}$ .
7:   for all  $v_{temp} \in N(v_{last})$  do
8:     if  $C \cup \{v_{temp}\}$  is a clique then
9:       push(CliqueStack,  $C \cup \{v_{temp}\}$ );
10:    end if
11:  end for
12: end while
13: Return the largest clique from Processed
```

Enumerate all cliques.

Checks all permutations!

For 100 vertices, $2^{99} - 1$ different cliques

Cliques

Pruning

- Prune all vertices (and incident edges) with degrees less than $k - 1$.
- Effective due to the power-law distribution of vertex degrees

“Exact cliques” are *rarely observed* in real networks.

E.g., a clique of 1,000 vertices has $(999 \times 1000) / 2 = 499,500$ edges.

- A single edge removal results in a subgraph that is no longer a clique.
- That represents less than 0.0002% of the edges (not practical)

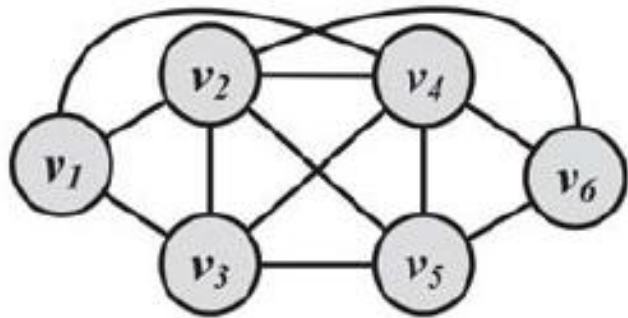
Relaxing Cliques (several methods)

Definition: All vertices have *a minimum degree* but not necessarily $k-1$

k-plex

For a set of vertices V_0 , for all u , $d_u \geq |V_0| - k$
where d_u is the degree of v in the induced subgraph

What is k for a clique?



Maximal

1-plex : $\{v_2, v_3, v_4, v_5\}$ a clique

2-plex : $\{v_1, v_2, v_3, v_4, v_5\}, \{v_2, v_3, v_4, v_5, v_6\}$

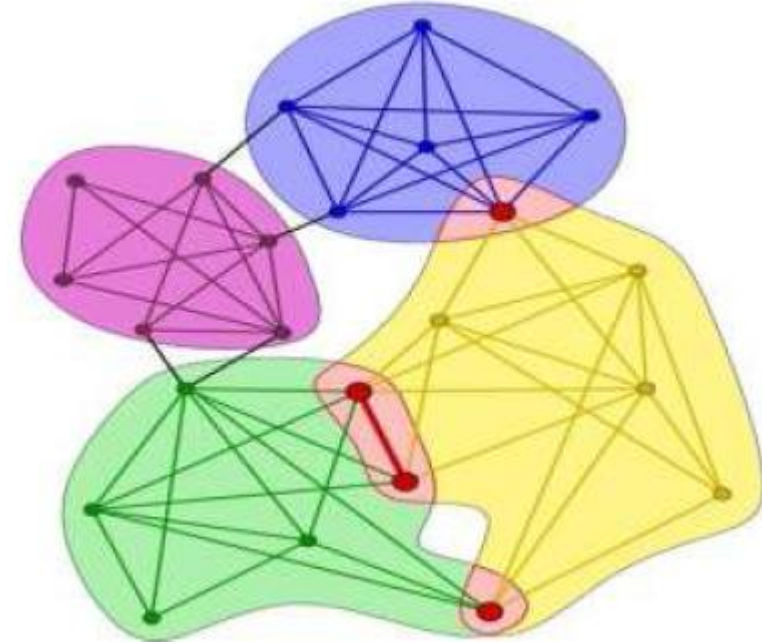
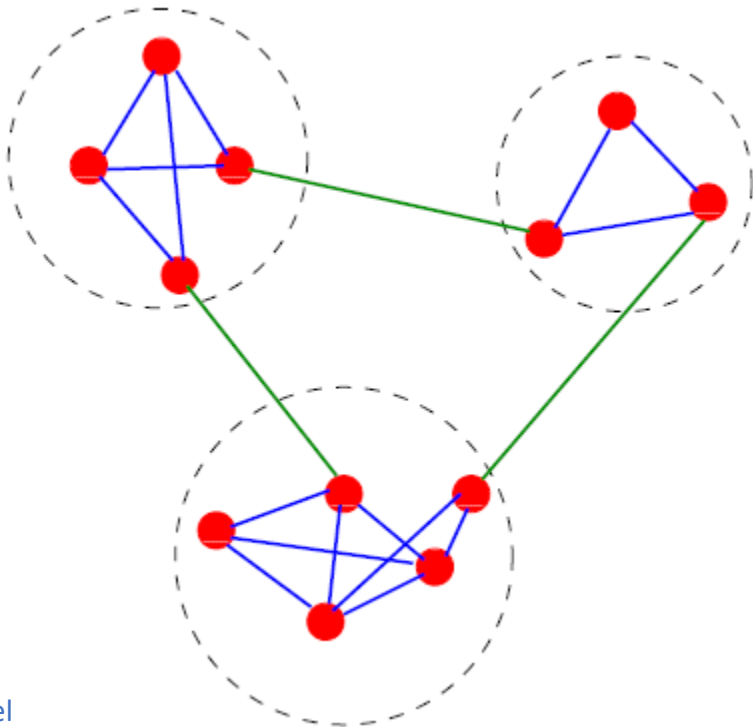
3-plex : $\{v_1, v_2, v_3, v_4, v_5, v_6\}$

Clique Percolation Method (CPM): Using cliques as seeds

CPM for community detection

Assumption:

communities are formed from cliques and edges that connect them



Clique Percolation Method (CPM)

1. Given k , find all cliques of size k .
2. Create graph (clique graph) where all cliques are vertices, and two cliques that **share $k - 1$ vertices** **are connected via an edge**.
3. Communities are the connected components of this graph.

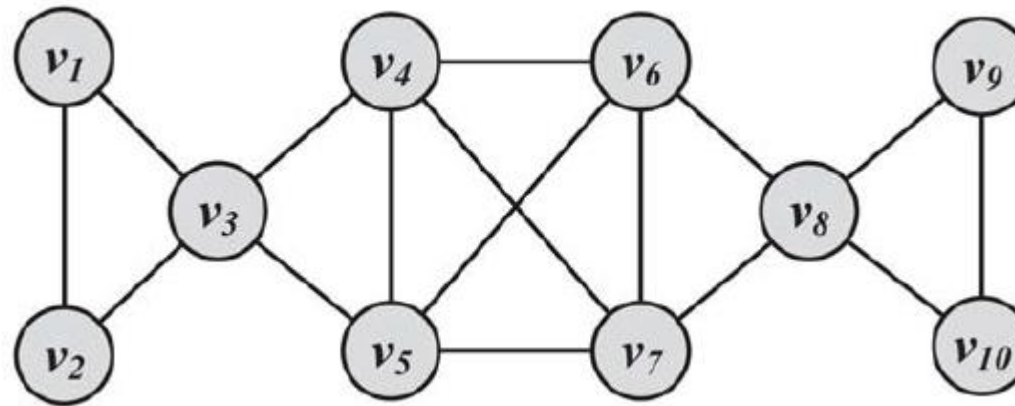
Algorithm Clique Percolation Method (CPM)

Require: parameter k

- 1: **return** Overlapping Communities
 - 2: $Cliques_k =$ find all cliques of size k
 - 3: Construct clique graph $G(V, E)$, where $|V| = |Cliques_k|$
 - 4: $E = \{e_{ij} \mid \text{clique } i \text{ and clique } j \text{ share } k - 1 \text{ nodes}\}$
 - 5: Return all connected components of G
-

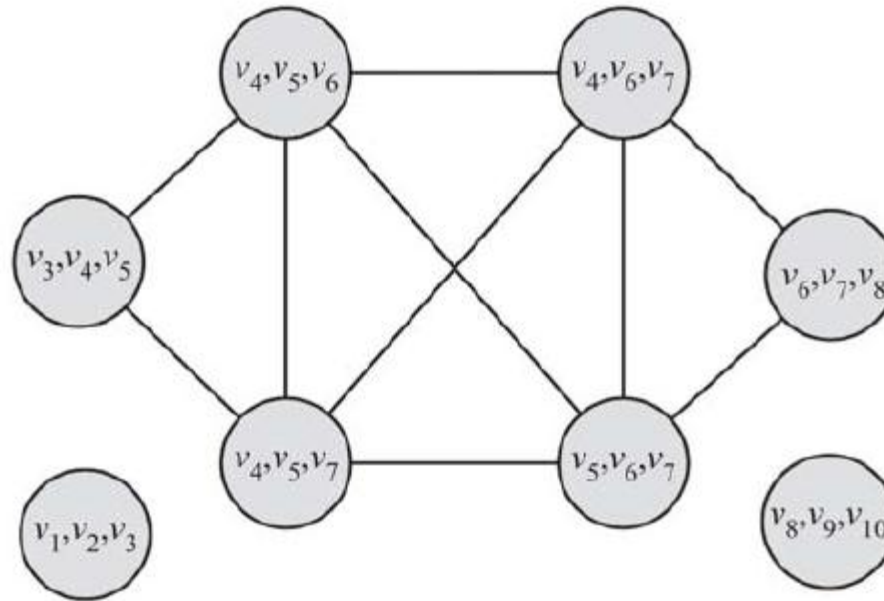
Clique Percolation Method (CPM): example

Input graph, let $k = 3$



CPM example cont.

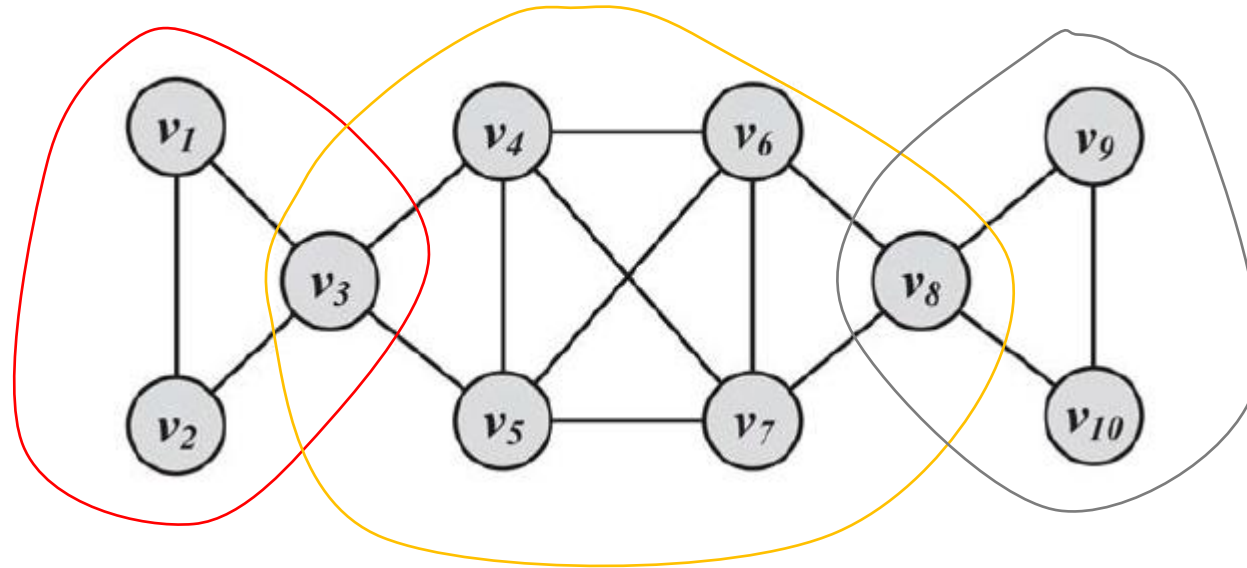
Clique graph for $k = 3$



(v_1, v_2, v_3) , (v_8, v_9, v_{10}) , and $(v_3, v_4, v_5, v_6, v_7, v_8)$

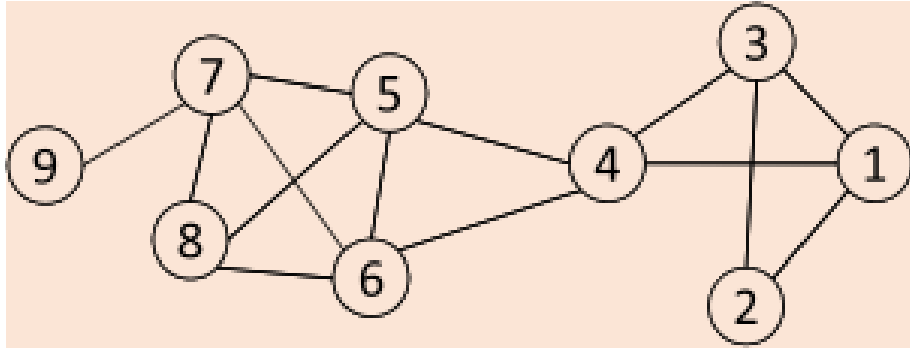
CPM example cont.

Result



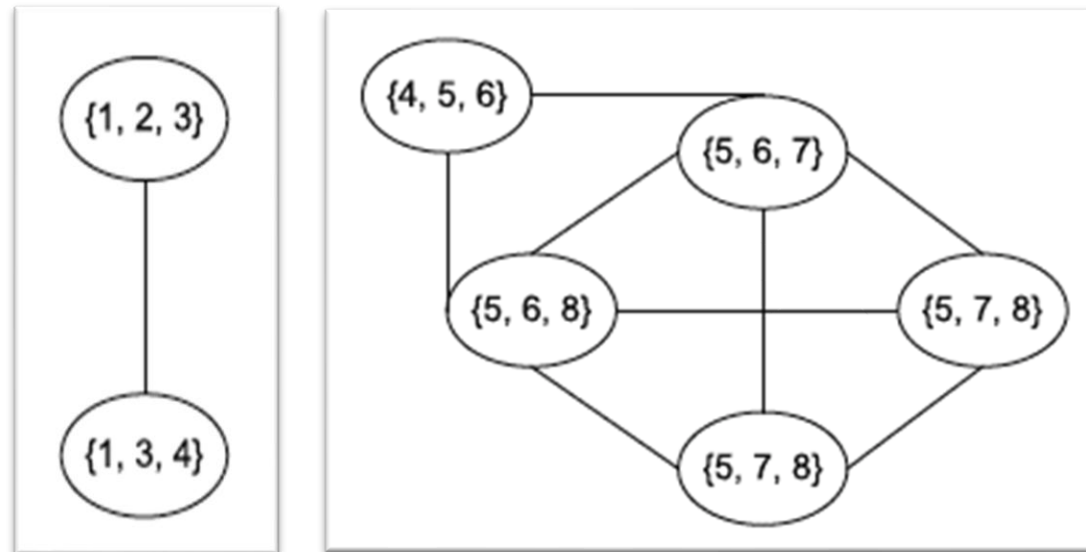
(v_1, v_2, v_3) , (v_8, v_9, v_{10}) , and $(v_3, v_4, v_5, v_6, v_7, v_8)$

Another CPM example



Cliques of size 3:

$\{1, 2, 3\}$, $\{1, 3, 4\}$, $\{4, 5, 6\}$, $\{5, 6, 7\}$,
 $\{5, 6, 8\}$, $\{5, 7, 8\}$, $\{6, 7, 8\}$



Communities:

$\{1, 2, 3, \underline{4}\}$
 $\{\underline{4}, 5, 6, 7, 8\}$



CPM characteristics

- Two k -cliques are **adjacent**, if they share $k - 1$ vertices.
- The union of adjacent k -cliques is called **k -clique chain**.
- Two k -cliques are **connected** if they are part of a k -clique chain.
- A **k -clique community** is the largest connected subgraph obtained by the union of a k -clique and of all k -cliques which are connected to it.

CPM characteristics

- A k -clique community is identified by making a k -clique “roll” over adjacent k -cliques, where rolling means rotating a k -clique about the $k-1$ vertices it shares with any adjacent k -clique.
- By construction, overlapping communities
- There may be vertices belonging to nonadjacent k -cliques, which could be reached by different paths and end up in different clusters. There are also vertices that cannot be reached by any k -clique
- Instead of $k = 3$, maximal cliques?
- Theoretical complexity grows exponentially with size, but *efficient on sparse graphs*

Vertex/Node Similarity

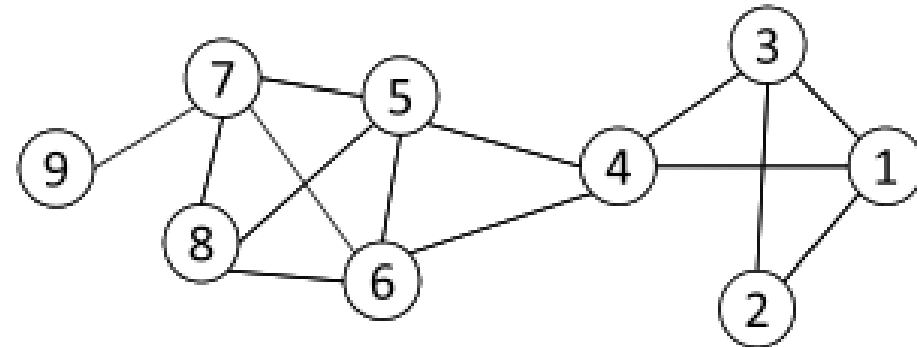
Another “Node-based Community Detection Method”

- Assumption:
Similar (or most similar) nodes belong to the same community
- Define similarity between two vertices (how?)
- Place similar vertices in the same cluster
- Use traditional *cluster analysis* (e.g. k-means)

Vertex/Node Similarity

- Structural equivalence: two nodes are **structurally equivalent** iff. they are connected to the same set of nodes ('actors')

Nodes 1 and 3 are
structurally equivalent;
So are nodes 5 and 6.



- Structural equivalence is too strict for practical use.

Node Similarity (can be used as node-feature)

- Structural equivalence: (or, what is the similarity between v_i, v_j) based on the overlap between their neighborhoods

$$\sigma(v_i, v_j) = |N(v_i) \cap N(v_j)| \quad N() = \text{set of neighbors}$$

- Jaccard and Cosine similarity are **normalized** to $[0, 1]$

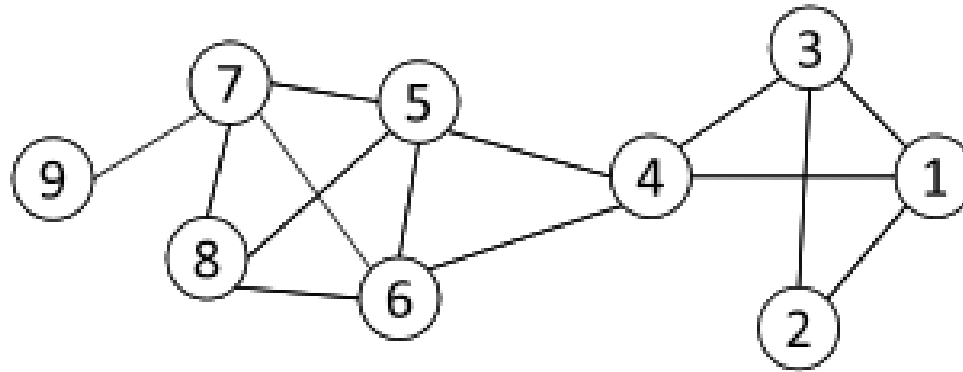
- Jaccard similarity

$$Jaccard(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}$$

- Cosine similarity

$$Cosine(v_i, v_j) = \frac{|N_i \cap N_j|}{\sqrt{|N_i| \cdot |N_j|}}$$

Node Similarity



$$Jaccard(v_i, v_j) = \frac{|N_i \cap N_j|}{|N_i \cup N_j|}$$

$$Cosine(v_i, v_j) = \frac{|N_i \cap N_j|}{\sqrt{|N_i| \cdot |N_j|}}$$

$$Jaccard(4, 6) = \frac{|\{5\}|}{|\{1, 3, 4, 5, 6, 7, 8\}|} = \frac{1}{7}$$

$$cosine(4, 6) = \frac{1}{\sqrt{4 \cdot 4}} = \frac{1}{4}$$

Cut-based methods

- Min cut
- Ratio cut
- Normalized cut

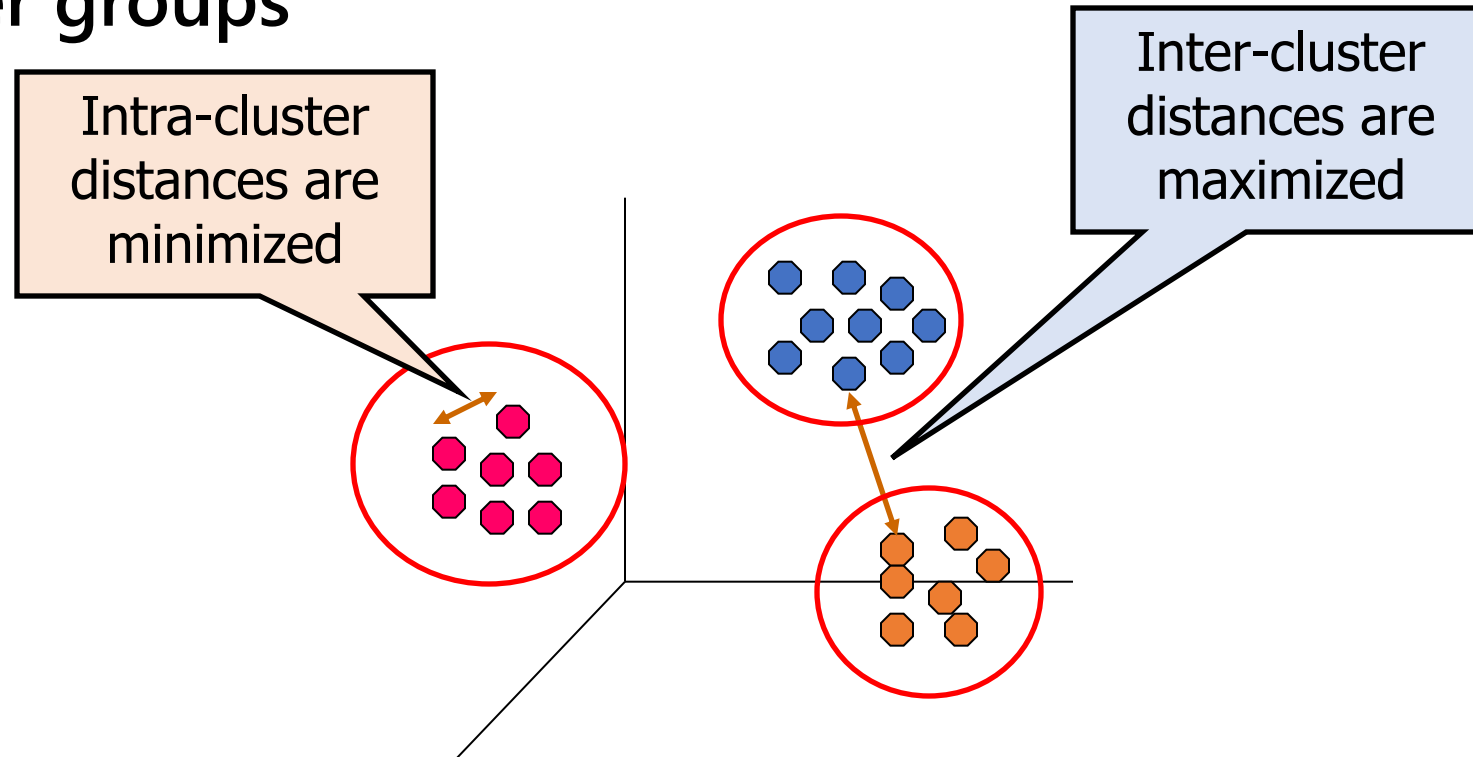
- Later

Cluster Analysis

- **Mainly a review**

What is Cluster Analysis?

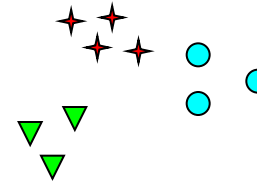
Finding groups of objects such that the objects in a group are similar (or related) to one another and different from (or unrelated to) the objects in other groups



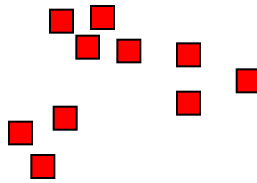
Notion of a cluster can be ambiguous



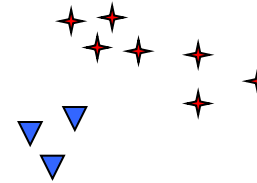
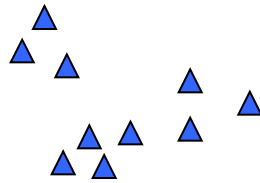
How many clusters?



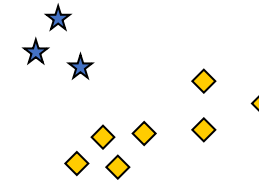
Six Clusters



Two Clusters



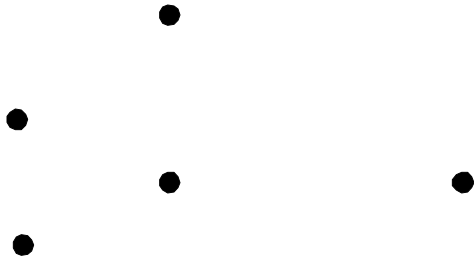
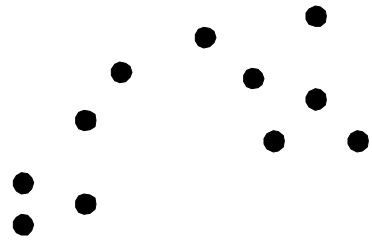
Four Clusters



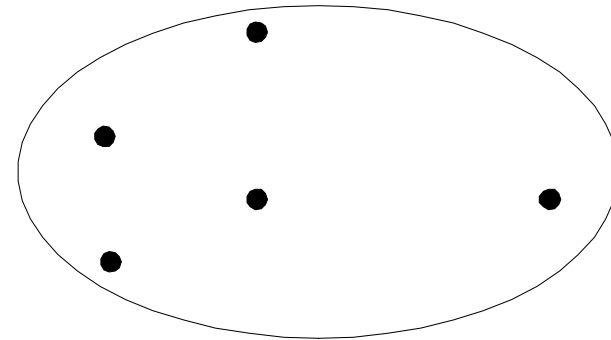
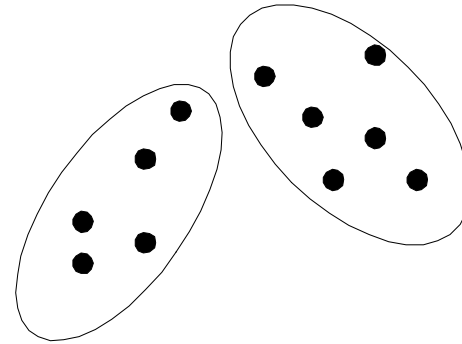
Types of Clustering

- A **clustering** is a set of clusters
- Important distinction between **hierarchical** and **partitional** sets of clusters
- **Partitional Clustering**
 - Division of data objects into *non-overlapping* subsets (clusters) such that each data object is in exactly one subset
 - Assumes that the number of clusters is given
- **Hierarchical clustering**
 - A set of nested clusters organized as a hierarchical tree

Partitional Clustering



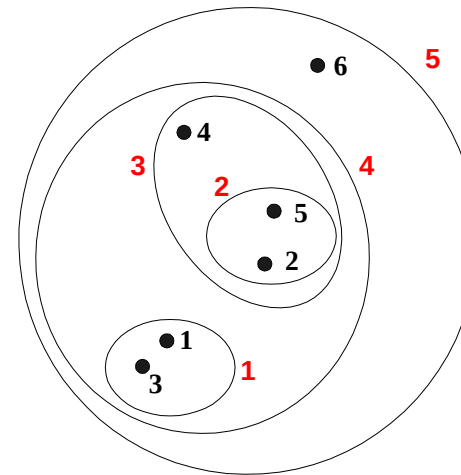
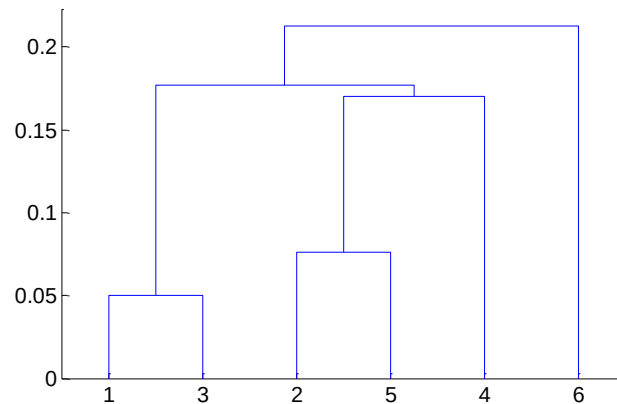
Original Points



A Partitional Clustering

Hierarchical Clustering

- Produces a set of nested clusters organized as a hierarchical tree
- Can be visualized as a **dendrogram**
 - A tree like diagram that records the sequences of merges or splits



Clusters defined by an objective function

Finds clusters that minimize or maximize an **objective function**.

- *Enumerate all possible ways of dividing the points* into clusters and evaluate the 'goodness' of each potential set of clusters by using the given objective function. (NP Hard)
- Can have *global* or *local* objectives.
 - Hierarchical clustering algorithms typically have local objectives
 - Partitional algorithms typically have global objectives
- A variation of the global objective function approach is to *fit the data to a parameterized model*.
 - Parameters for the model are determined from the data (e.g. GMM)

Clustering Algorithms

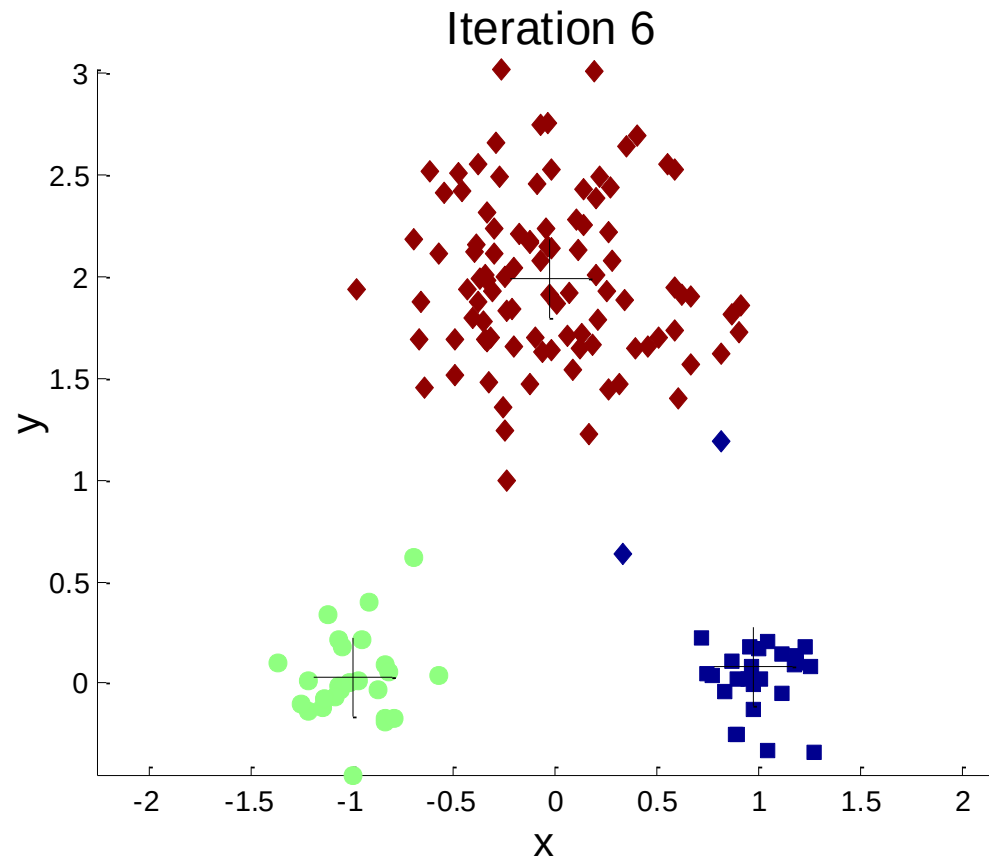
- K-means
- Hierarchical clustering
- *Density clustering*

K-means Clustering

- Partitional clustering approach
- Each cluster is associated with a *centroid* (center point)
- Each point is assigned to the cluster with the closest centroid
- Number of clusters, K , must be specified
- The basic algorithm is very simple

- 1: Select K points as the initial centroids.
- 2: **repeat**
- 3: Form K clusters by assigning all points to the closest centroid.
- 4: Recompute the centroid of each cluster.
- 5: **until** The centroids don't change

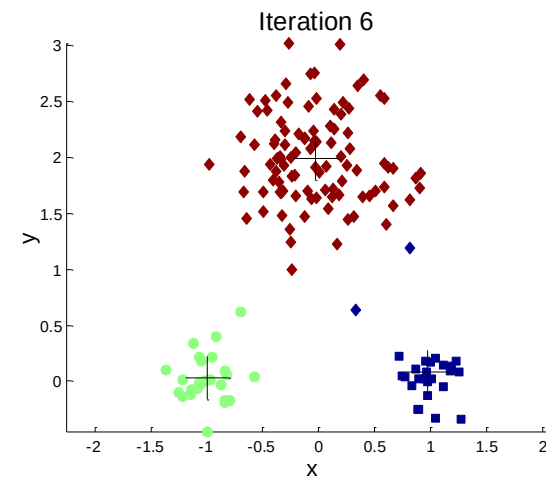
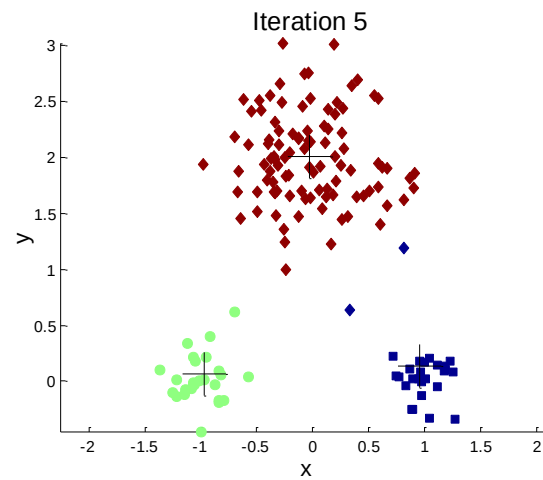
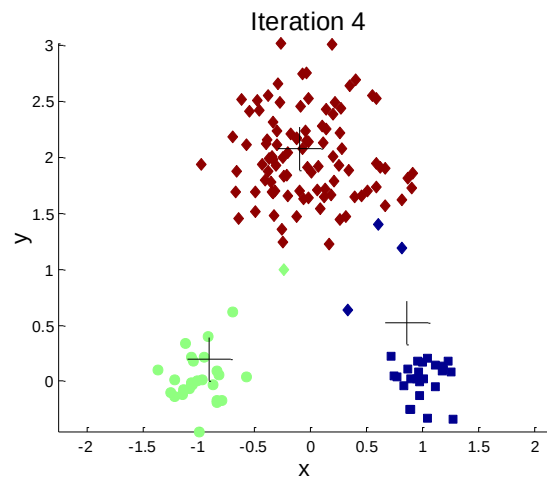
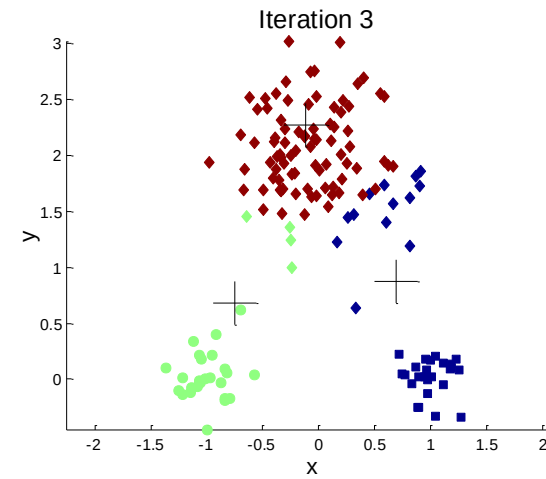
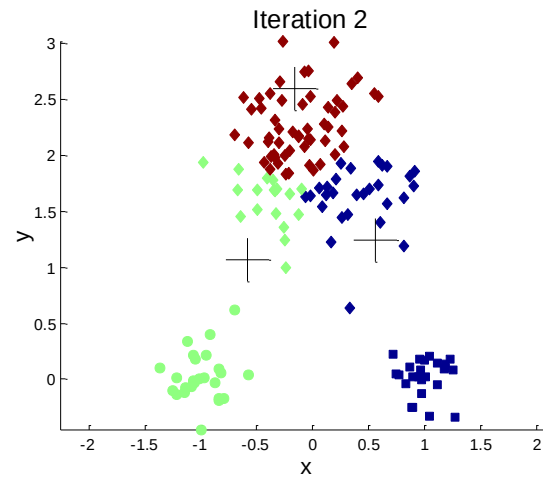
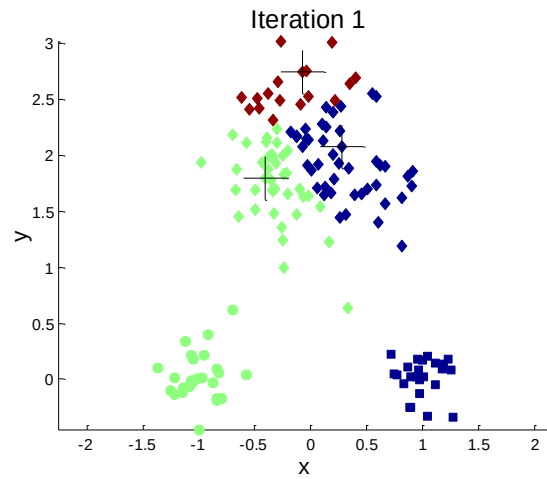
Example



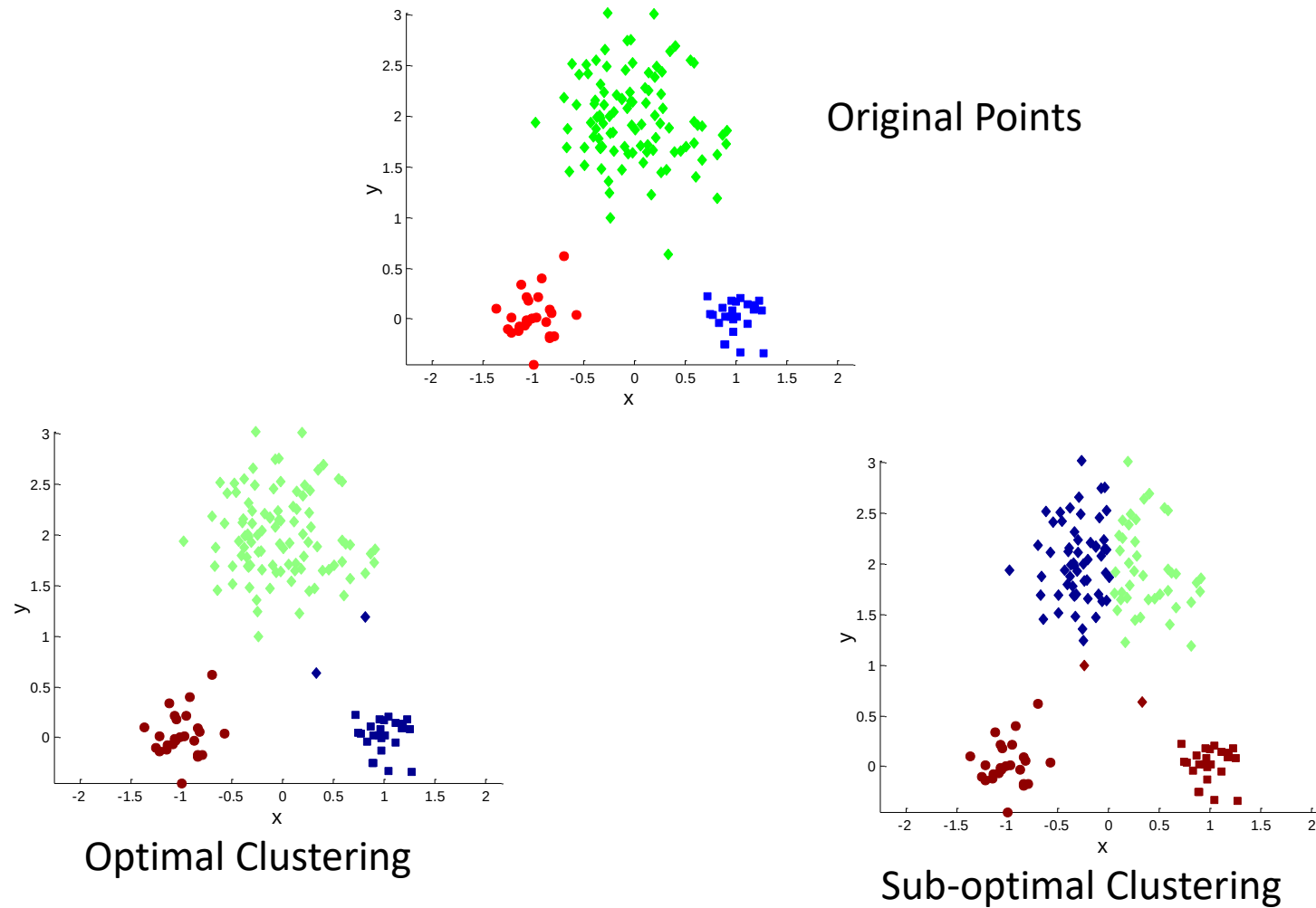
K-means Clustering

- Initial centroids are often chosen randomly.
 - Clusters produced vary from one run to another.
- The centroid is (typically) the mean of the points in the cluster.
- 'Closeness' is measured by Euclidean distance, cosine similarity, correlation, etc.
- K-means *will converge* for common similarity measures mentioned above.
- Most of the convergence happens in the first few iterations.
 - Often the stopping condition is changed to 'Until relatively few points change clusters'
- Complexity is $O(n * K * I * d)$
 - n = number of points, K = number of clusters,
 I = number of iterations, d = number of attributes

Example



Two different K-means clustering trials (I.C.)



Importance of choosing initial points

K-means Clusters

- Most common measure is **Sum of Squared Error (SSE)**
 - For each point, the **error** is the distance to the nearest cluster
 - To get SSE, we square these errors and sum them.

$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

- x is a data point in cluster C_i and m_i is the representative point for cluster C_i
 - can show that m_i corresponds to the center (mean) of the cluster
- Given two clusters, we can choose the one with the smallest error
- One easy way to reduce SSE is to increase K , the number of clusters
 - A good clustering with smaller K can have a lower SSE than a poor clustering with higher K

Limitations of K-means

- K-means has problems when clusters are of differing
 - Sizes
 - Densities
 - Non-globular shapes
- K-means has problems when the data contains outliers.

Pre-processing and Post-processing

- **Pre-processing**
 - Normalize the data
 - Eliminate outliers
- **Post-processing**
 - Eliminate small clusters that may represent outliers
 - Split 'loose' clusters, i.e., clusters with relatively high SSE
 - Merge clusters that are 'close' and that have relatively low SSE
 - Can use these steps during the clustering process

Hierarchical Clustering

- Two main types of hierarchical clustering
 - Agglomerative: bottom-up
 - Start with the points (vertices) as individual clusters
 - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left
 - Divisive: top-down
 - Start with one, all-inclusive cluster (the whole graph)
 - At each step, split a cluster until each cluster contains a point (vertex) (or there are k clusters)
- Traditional hierarchical algorithms use a similarity or distance matrix
 - Merge or split one cluster at a time

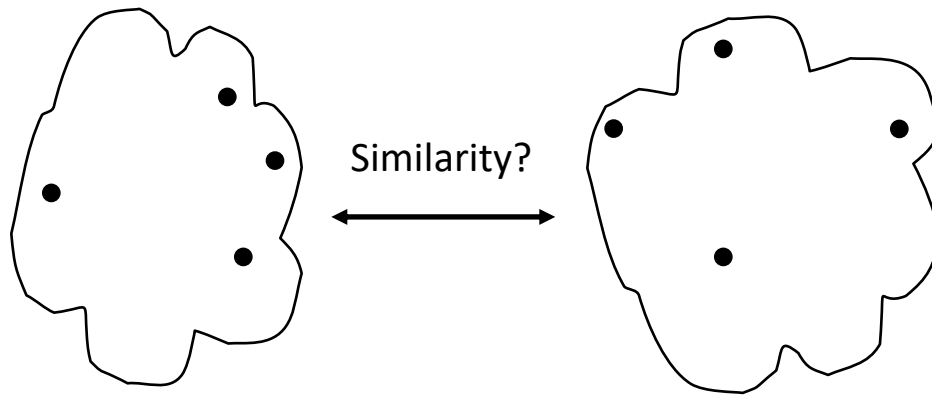
Strengths of Hierarchical Clustering

- Do not have to assume any particular number of clusters
 - Any desired number of clusters can be obtained by 'cutting' the **dendrogram** at the proper level
- They may correspond to meaningful taxonomies
 - Example in biological sciences (e.g., animal kingdom, phylogeny reconstruction, ...)

Agglomerative Clustering Algorithm

- Popular hierarchical clustering technique
- Basic algorithm is straightforward
 1. [Compute the proximity matrix]
 2. Let each data point be a cluster
 3. Repeat
 4. Merge the *two closest clusters*
 5. [Update the proximity matrix]
 6. Until only a single cluster remains
- Key operation is the *computation of the proximity of two clusters*
 - Different approaches to defining the distance between clusters distinguish the different algorithms

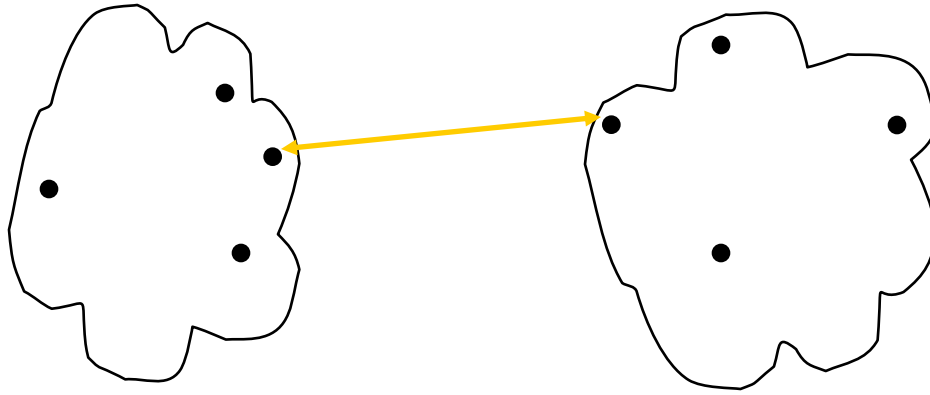
How to Define Inter-Cluster Similarity



	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity



MIN or single -link

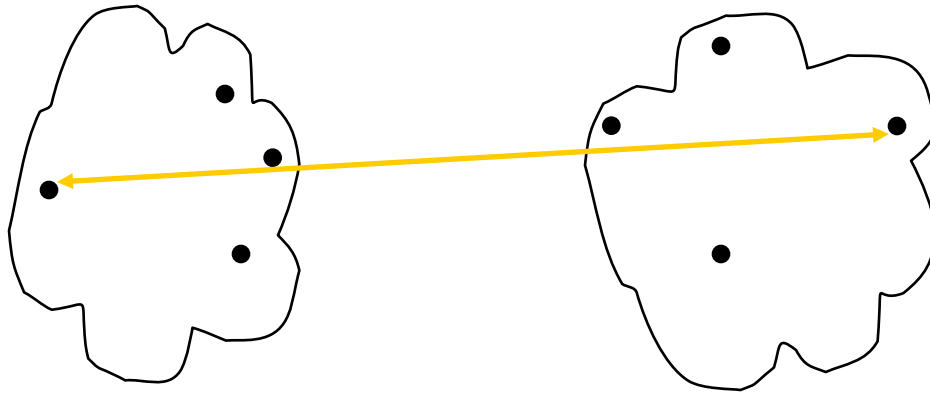
based on the two most similar (closest)
points in the different clusters

(sensitive to outliers)

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

How to Define Inter-Cluster Similarity



MAX or complete linkage

Similarity of two clusters is based on the two least similar (most distant) points in the different clusters

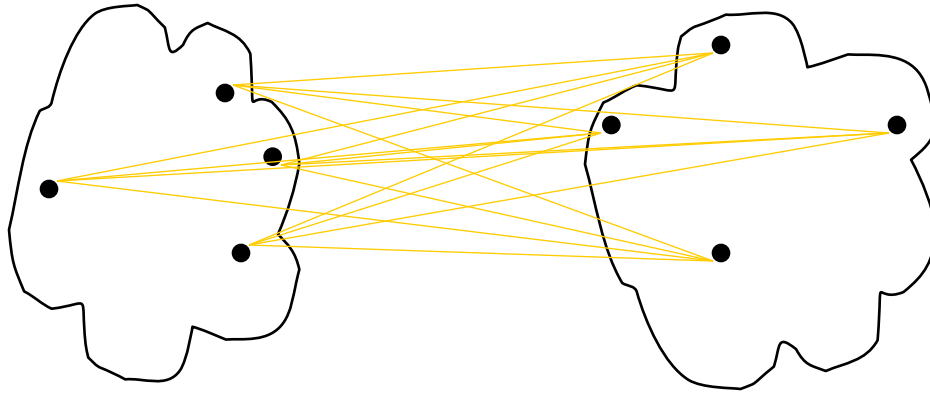
(Tends to break large clusters
Biased towards globular clusters)

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

·

· Proximity Matrix

How to Define Inter-Cluster Similarity



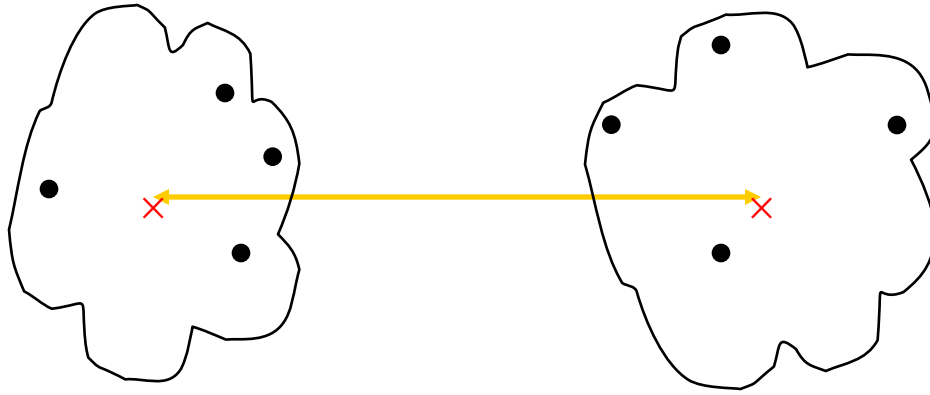
	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						

· Proximity Matrix

Group Average

Proximity of two clusters is the average of pairwise proximity between points in the two clusters.

How to Define Inter-Cluster Similarity



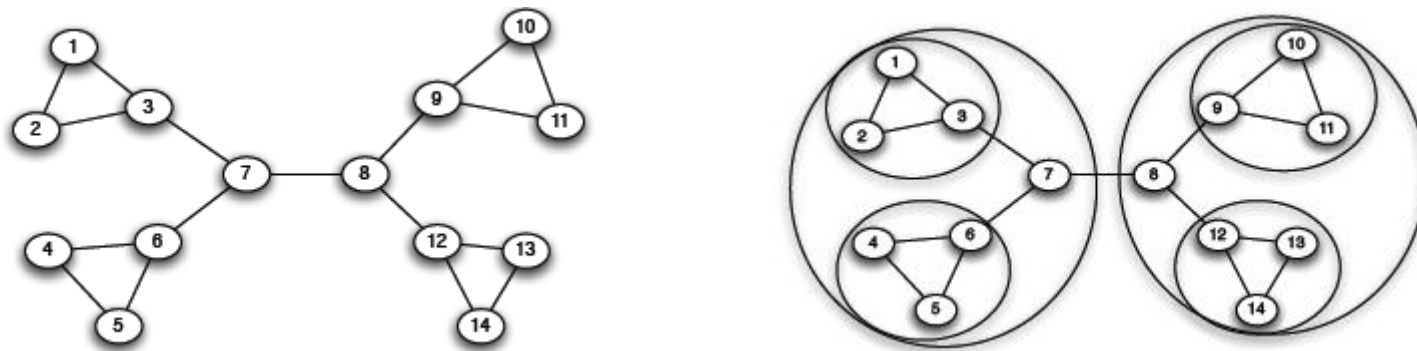
Distance Between Centroids

	p1	p2	p3	p4	p5	...
p1						
p2						
p3						
p4						
p5						
.						
.						
.						

Proximity Matrix

Graph Partitioning

- **Divisive methods:** try to identify and remove the “spanning links” between densely-connected regions
- **Agglomerative methods:** Find nodes that are likely to belong to the same region and merge them together (bottom-up)

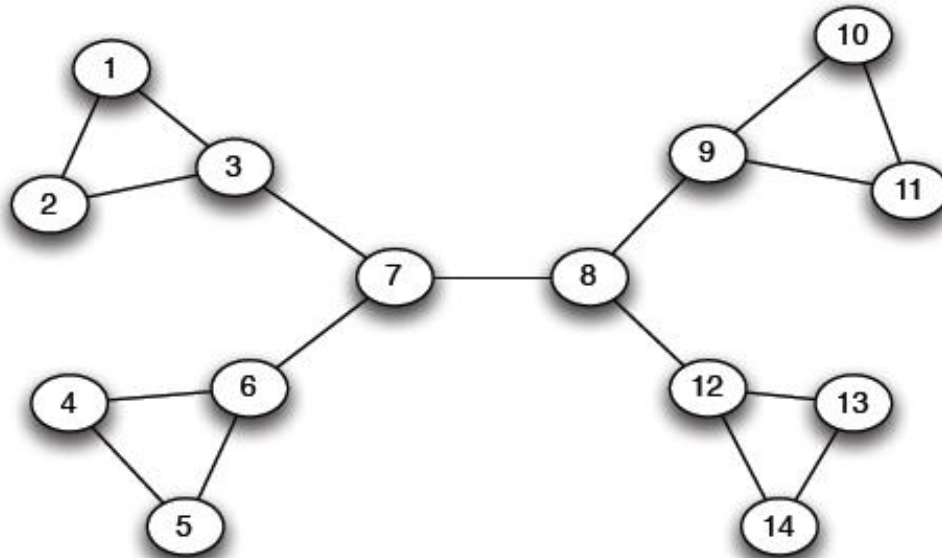


The Girvan Newman method

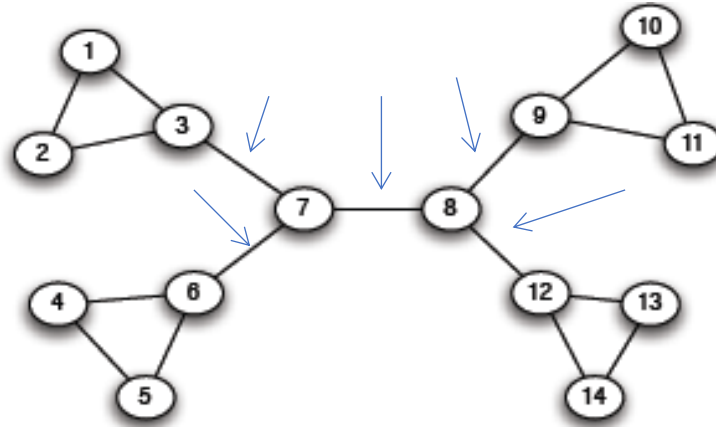
Hierarchical divisive method

- Start with the whole graph
- Find edges whose removal “partitions” the graph
- Repeat with each subgraph until single vertices

Which edge?



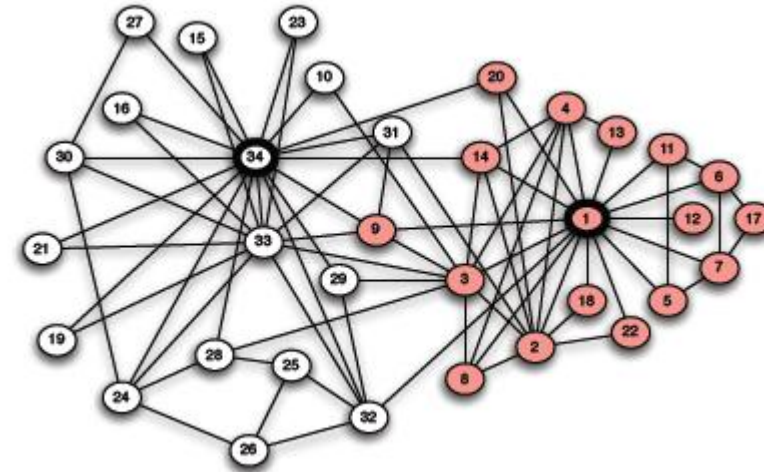
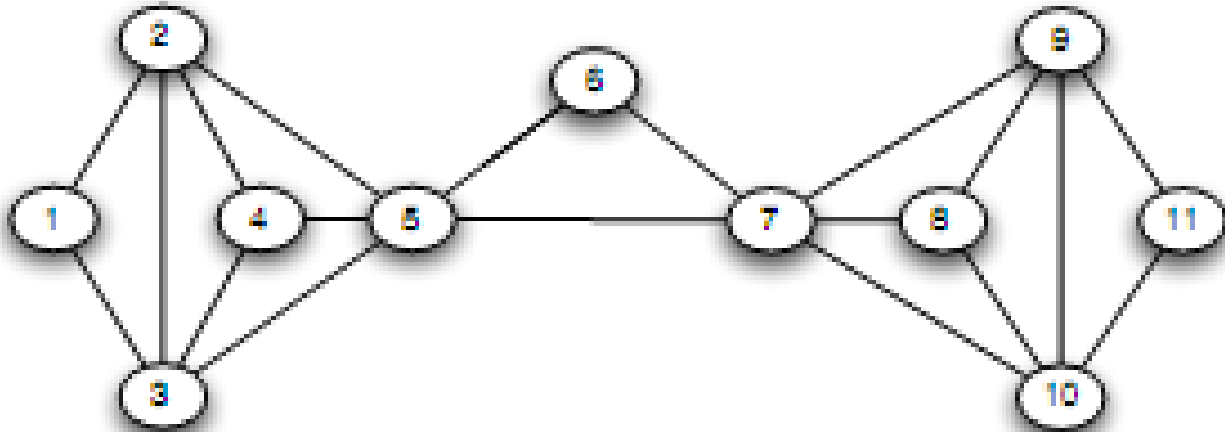
The Girvan Newman method



Use bridges or cut-edge (if removed, the nodes become disconnected)

but Which one to choose?

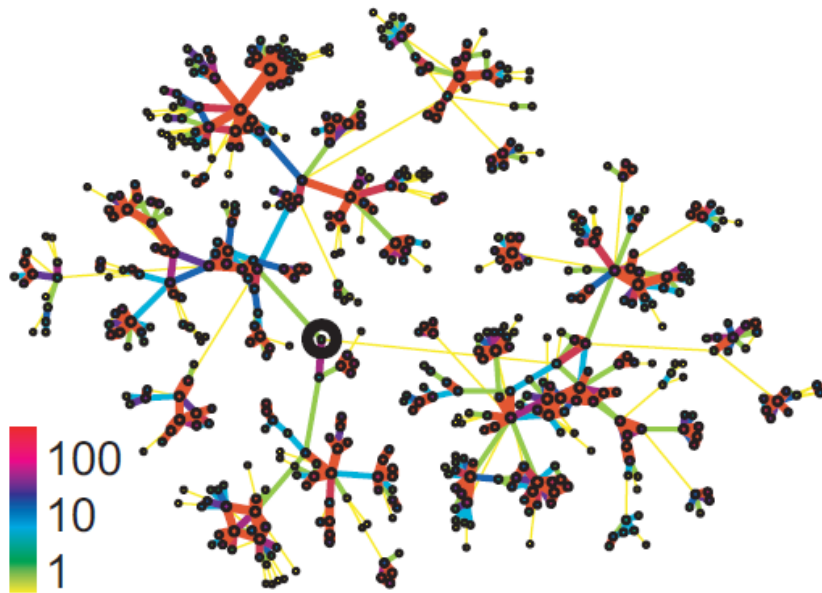
The Girvan Newman method



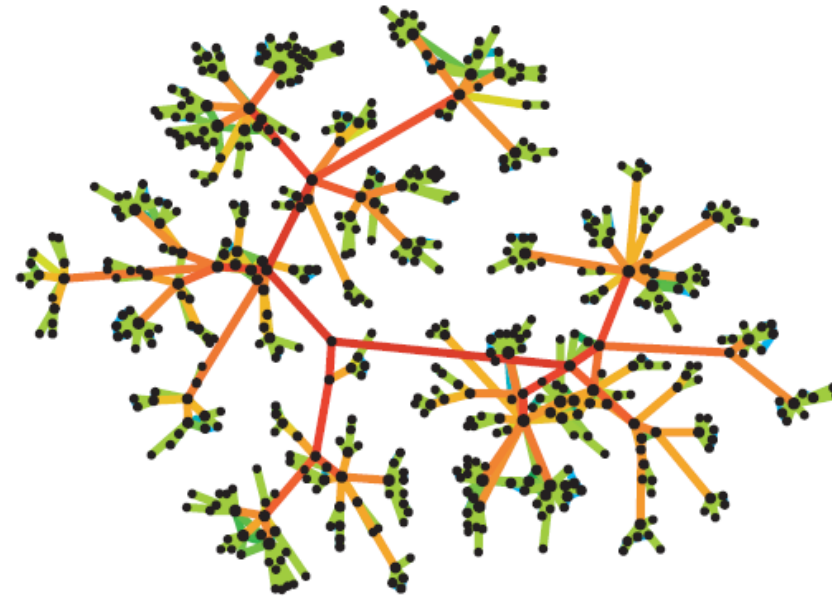
There may be none! Single cut solutions

Strength of Weak Ties

- **Edge betweenness**: Number of shortest paths passing over the edge
- **Intuition**:



Edge strengths (e.g. call volume)
in a real network



Edge betweenness
in a real network

Edge Betweenness

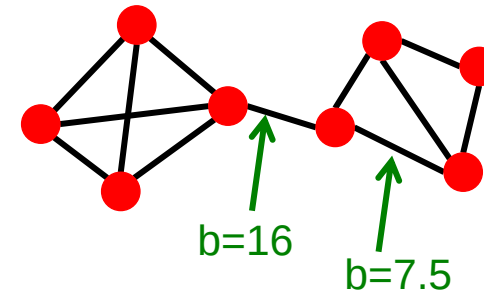
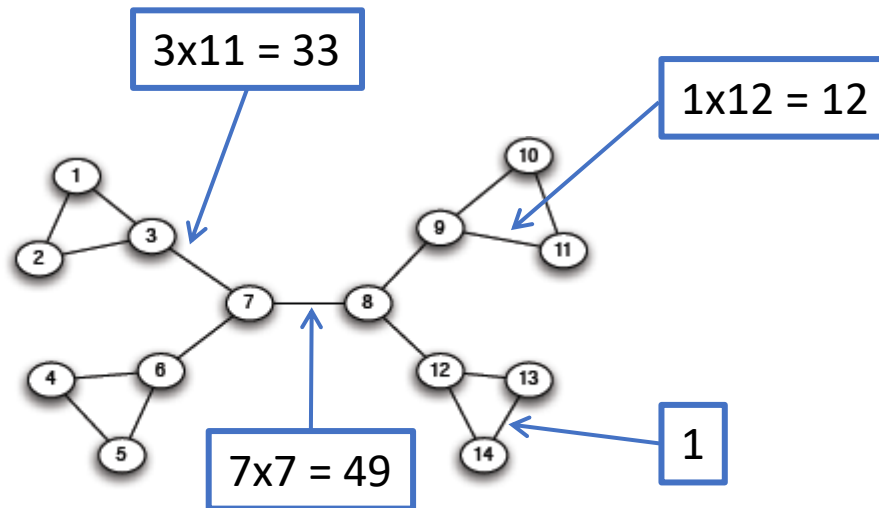
Betweenness of an edge (a, b):

- Number of pairs of nodes x and y such that the edge (a, b) lies on the shortest path between x and y
- Since there can be several such shortest paths, edge (a, b) is credited with the fraction of those shortest paths that include (a, b) .

$$bt(a, b) = \sum_{x, y} \frac{\# \text{shortest_paths}(x, y) \text{ through } (a, b)}{\# \text{shortest_paths}(x, y)}$$

Edge Betweenness

$$bt(a,b) = \sum_{x,y} \frac{\# \text{shortest_paths}(x,y) \text{ through } (a,b)}{\# \text{shortest_paths}(x,y)}$$



Edges that have a high probability to occur on a randomly chosen shortest path between two randomly chosen nodes have a high betweenness.

Traffic (unit of flow)

Computing Betweenness

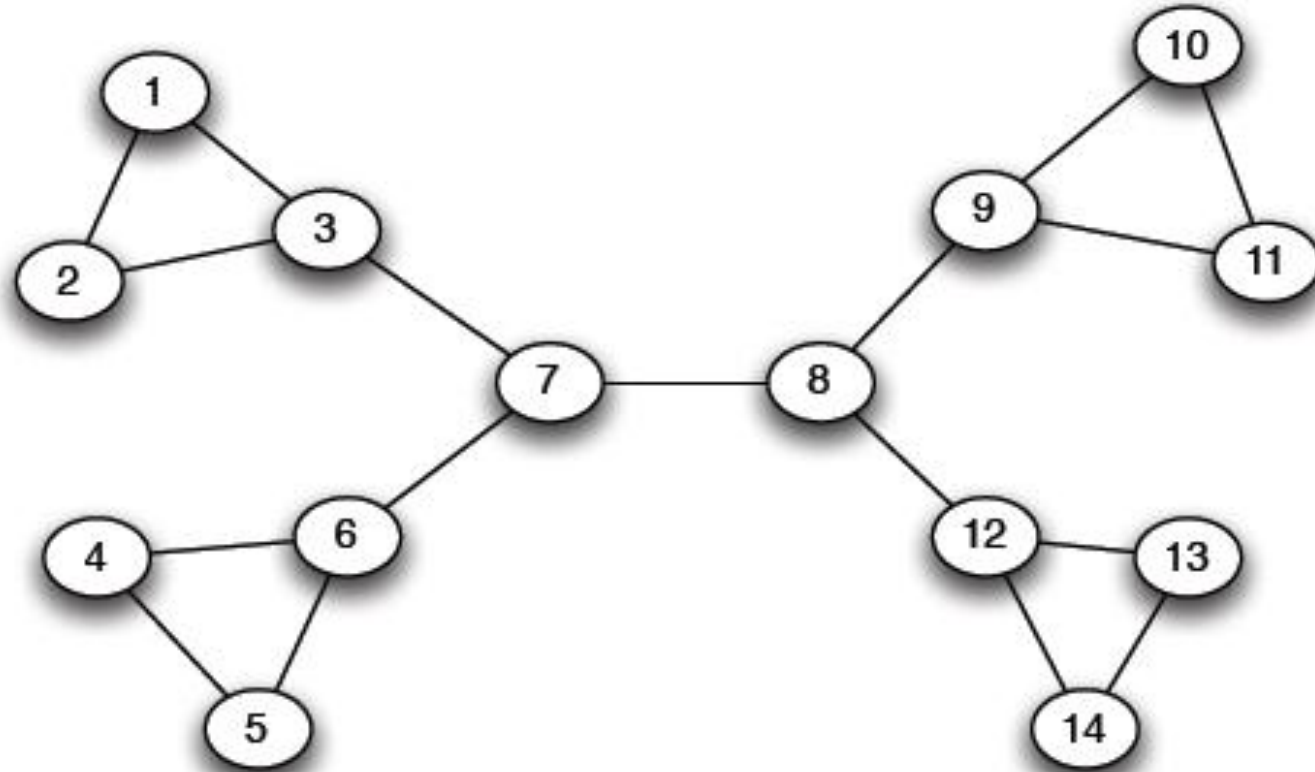
- For undirected graph, the complexity is governed by the **BFS** algo.
- For unweighted graphs the time complexity is $O(NE + N^2 \log N)$

The Girvan Newman method

Undirected unweighted networks

- Repeat until no edges are left:
 - Calculate betweenness of edges
 - Remove edges with highest betweenness
- Connected components are communities
- Gives a hierarchical decomposition of the network

Girvan Newman method: An example



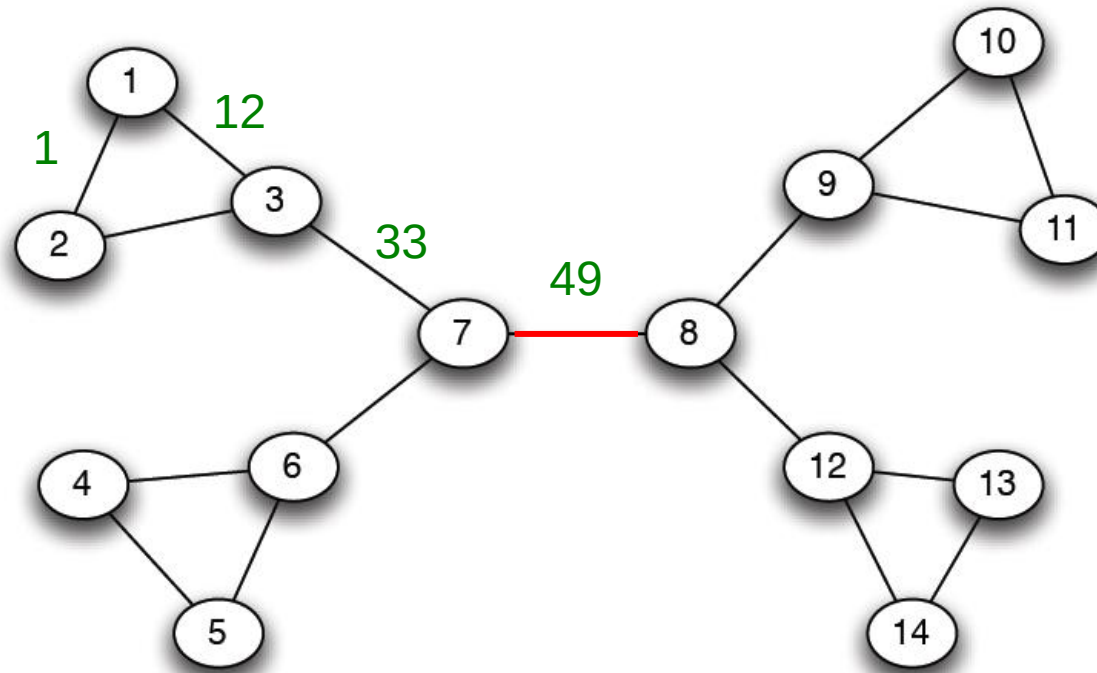
$$\text{Betweenness}(7, 8) = 7 \times 7 = 49$$

$$\text{Betweenness}(1, 3) = 1 \times 12 = 12$$

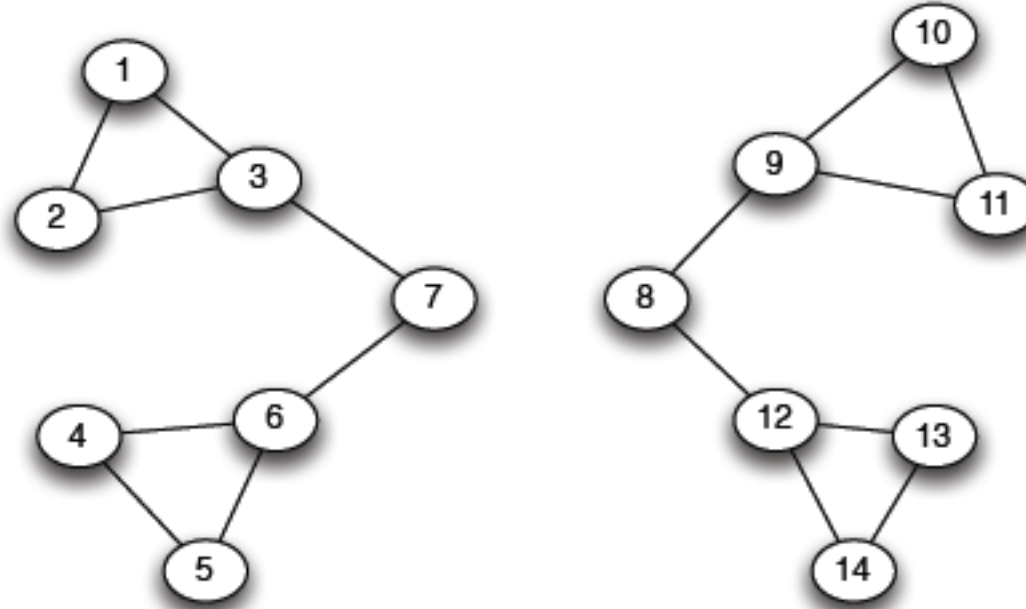
$$\text{Betweenness}(3, 7) = \text{Betweenness}(6, 7) = \text{Betweenness}(8, 9) = \text{Betweenness}(8, 12) = 3 \times 11 = 33$$

Girvan-Newman: Example

Need to re-compute betweenness at every step



Girvan Newman method: An example

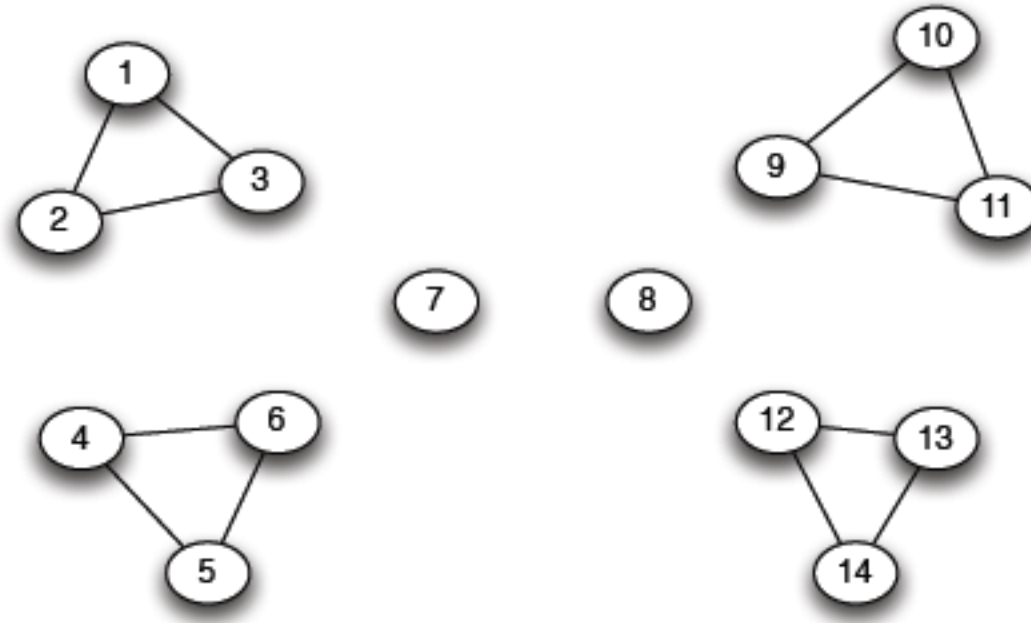


(a) *Step 1*

$\text{Betweenness}(1, 3) = 1 \times 5 = 5$

$\text{Betweenness}(3, 7) = \text{Betweenness}(6, 7) = \text{Betweenness}(8, 9) = \text{Betweenness}(8, 12) = 3 \times 4 = 12$

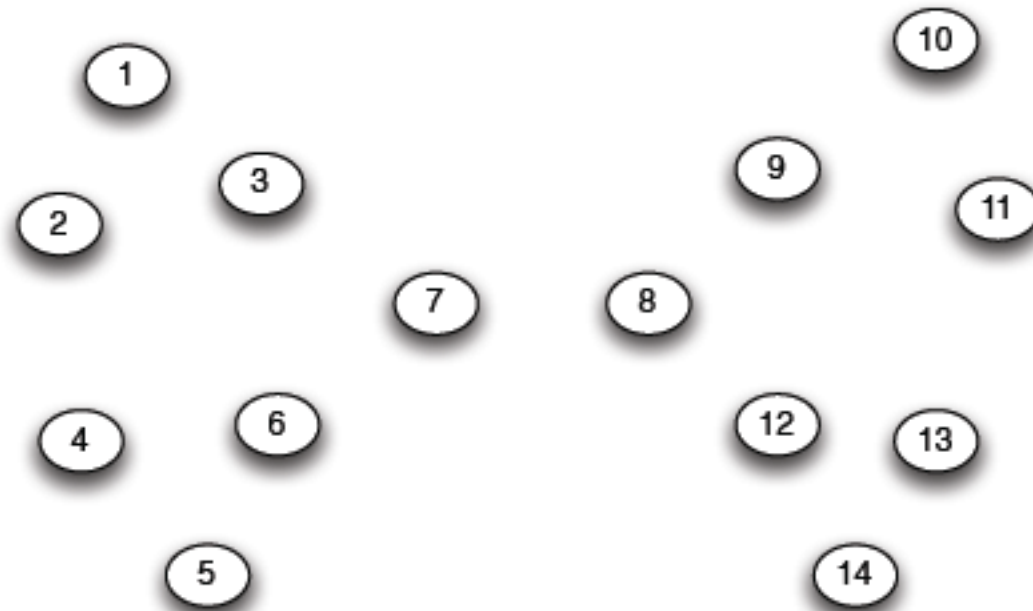
Girvan Newman method: An example



(b) *Step 2*

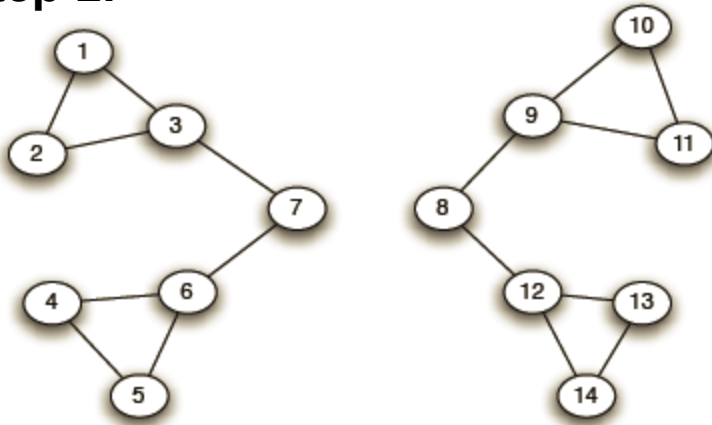
Betweenness of every edge = 1

Girvan Newman method: An example

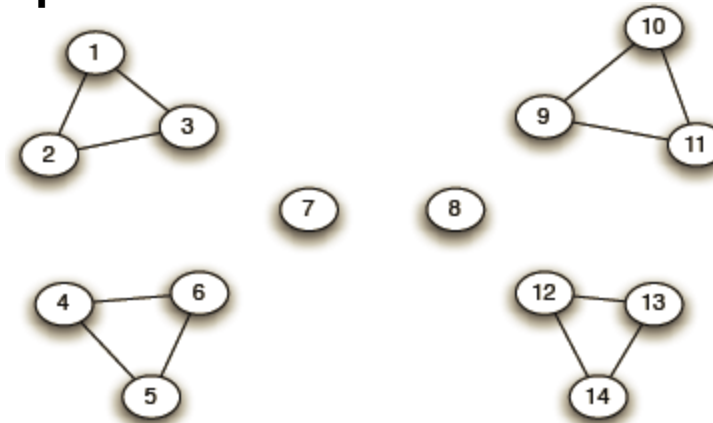


Girvan-Newman: Example

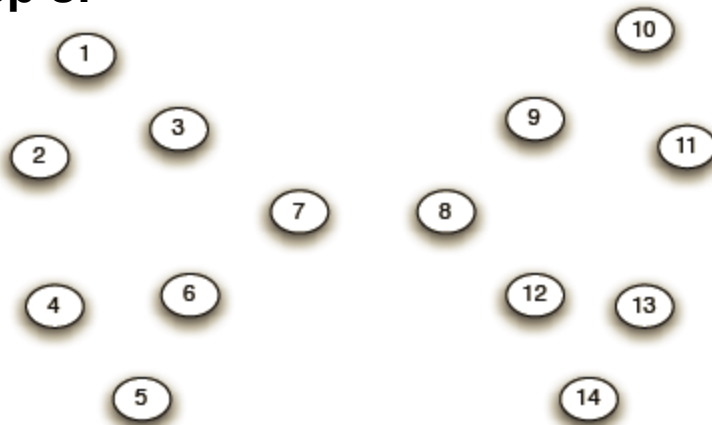
Step 1:



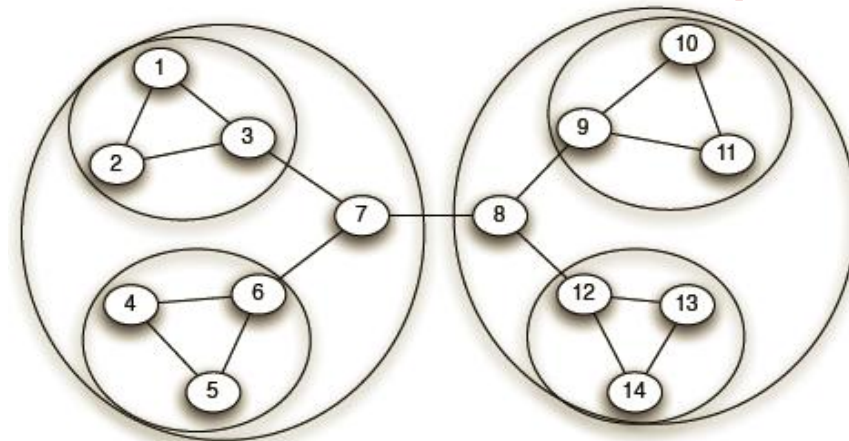
Step 2:



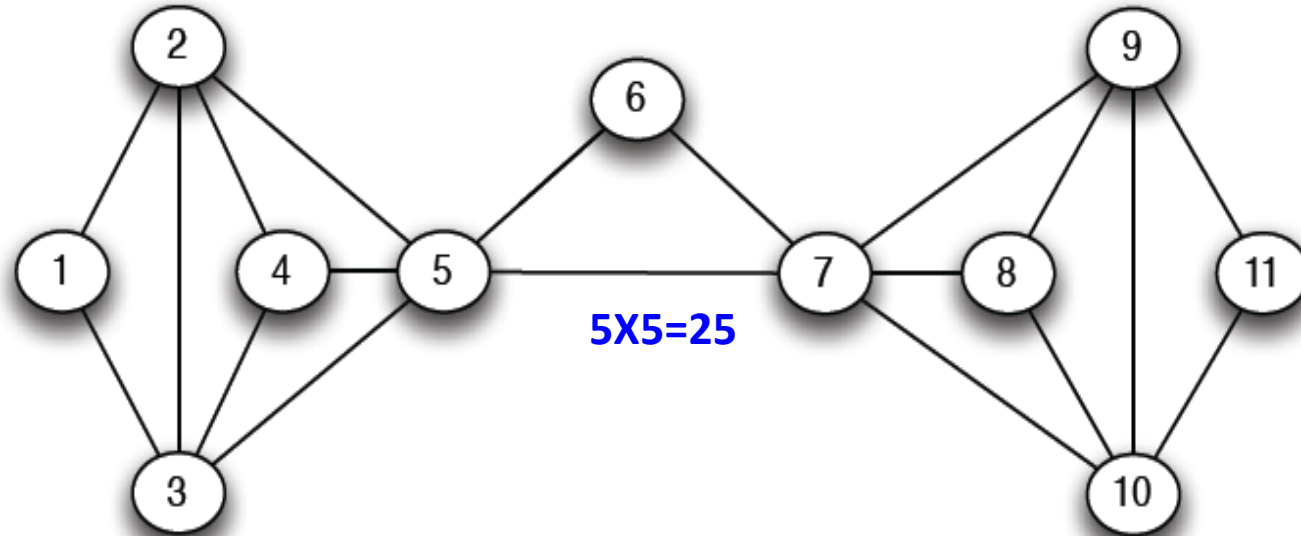
Step 3:



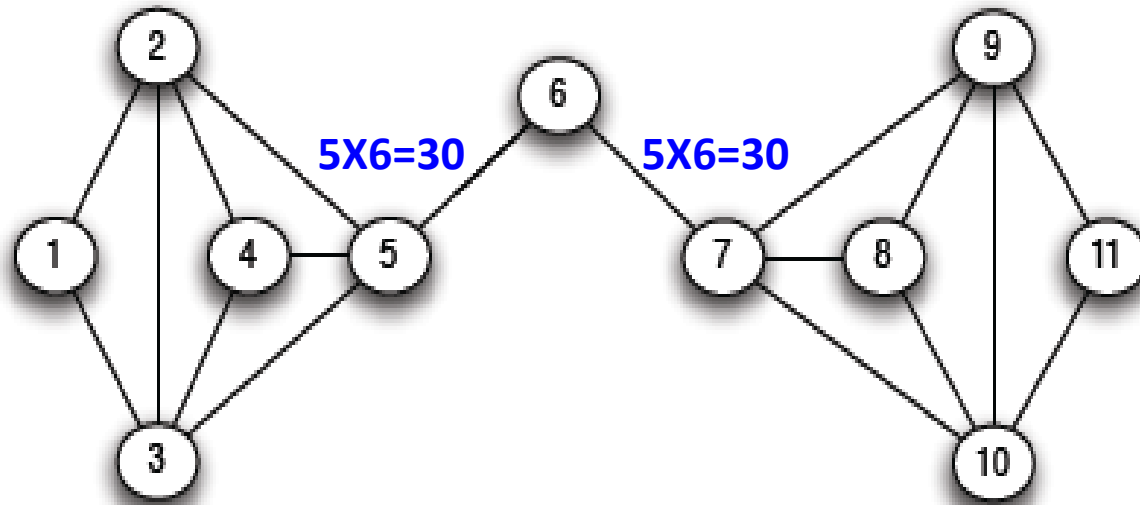
Hierarchical network decomposition:



Another example

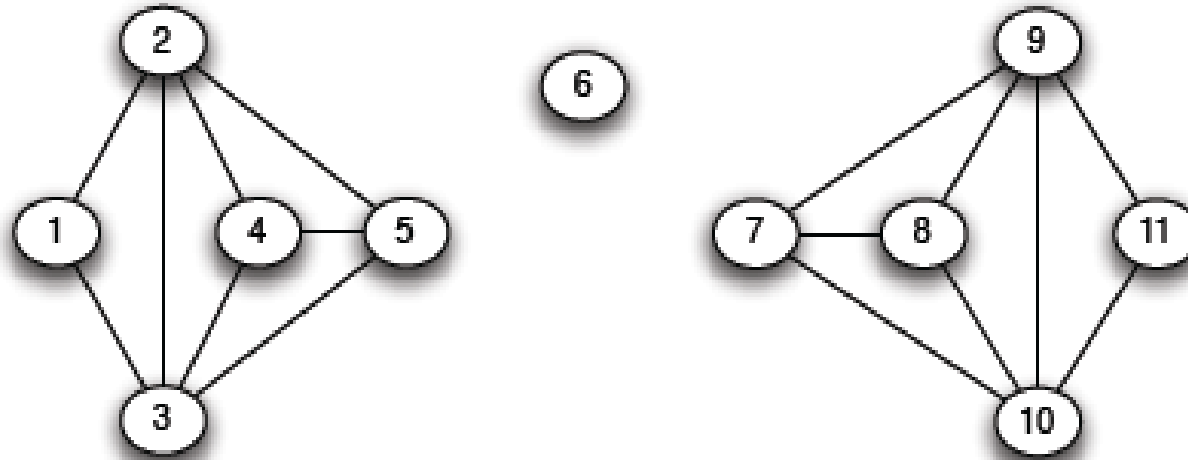


Another example



(a) *Step 1*

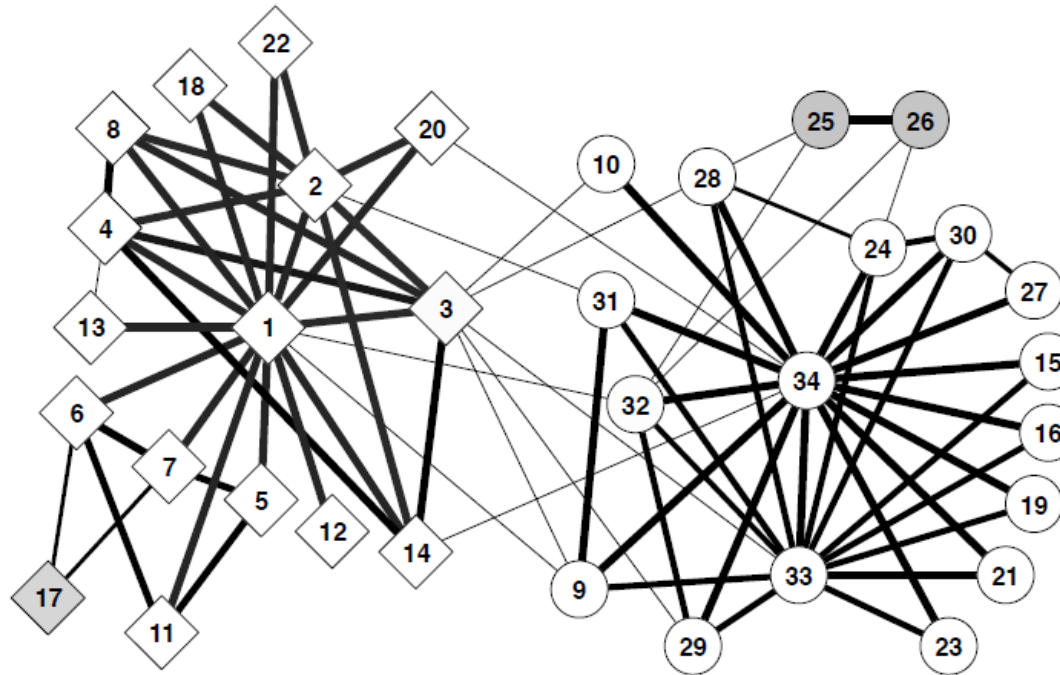
Another example



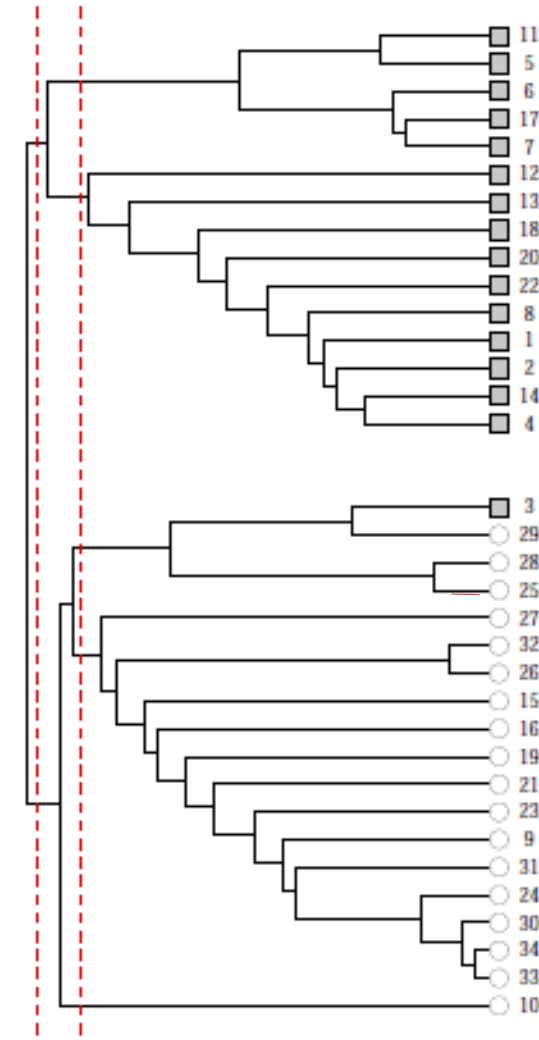
(b) *Step 2*

Girvan-Newman: Results

- **Zachary's Karate club:**
Hierarchical decomposition



Dendrogram



Spectral Clustering

Problems of K-means

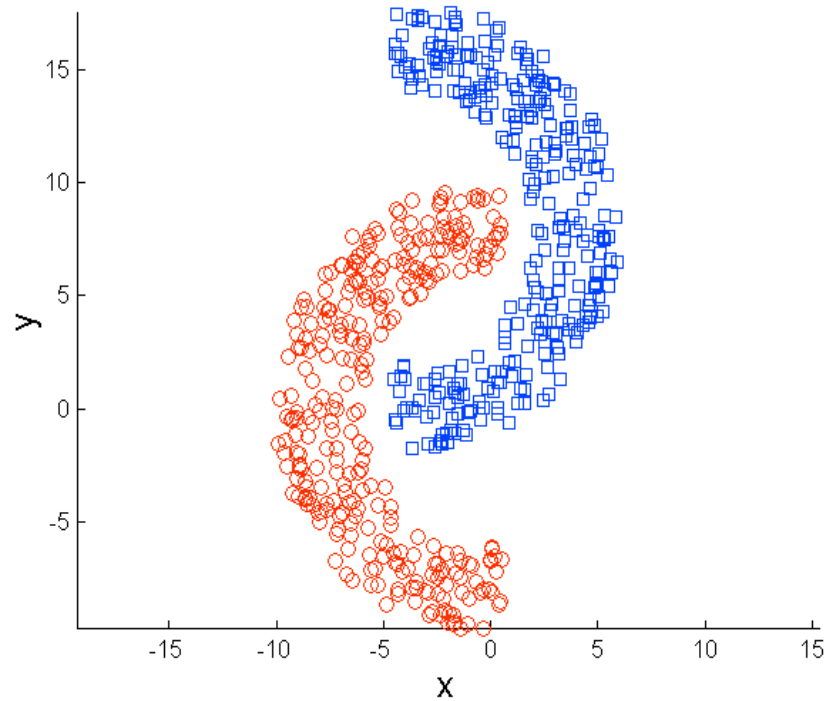
- **Centers:** random & density scan
- **K:** start from small K & separate iteratively;
start from large K and merge sequentially
- **Outliers:**
 - Locations of Centers
 - Number of Clusters K
 - Sensitive to Outliers
 - **Data Manifolds (Shapes of Data Distributions)**
 - Experiences

Problems of K-MEANS

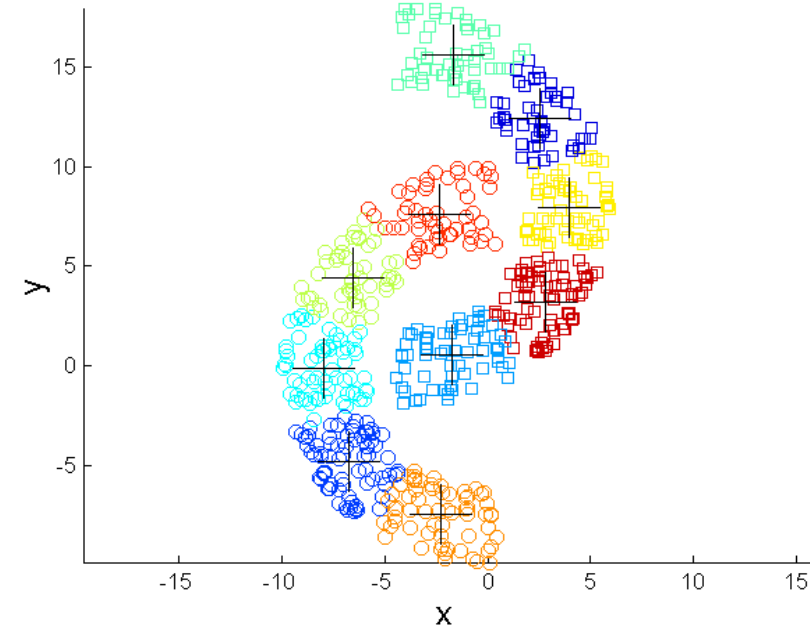
- Similarity function cannot handle special **data manifold** effectively!
- Intra-cluster similarity and inter-cluster similarity are not optimized jointly or simultaneously!
- Pre-selected locations of cluster centers may not be acceptable!

K-Means Clustering

Expected

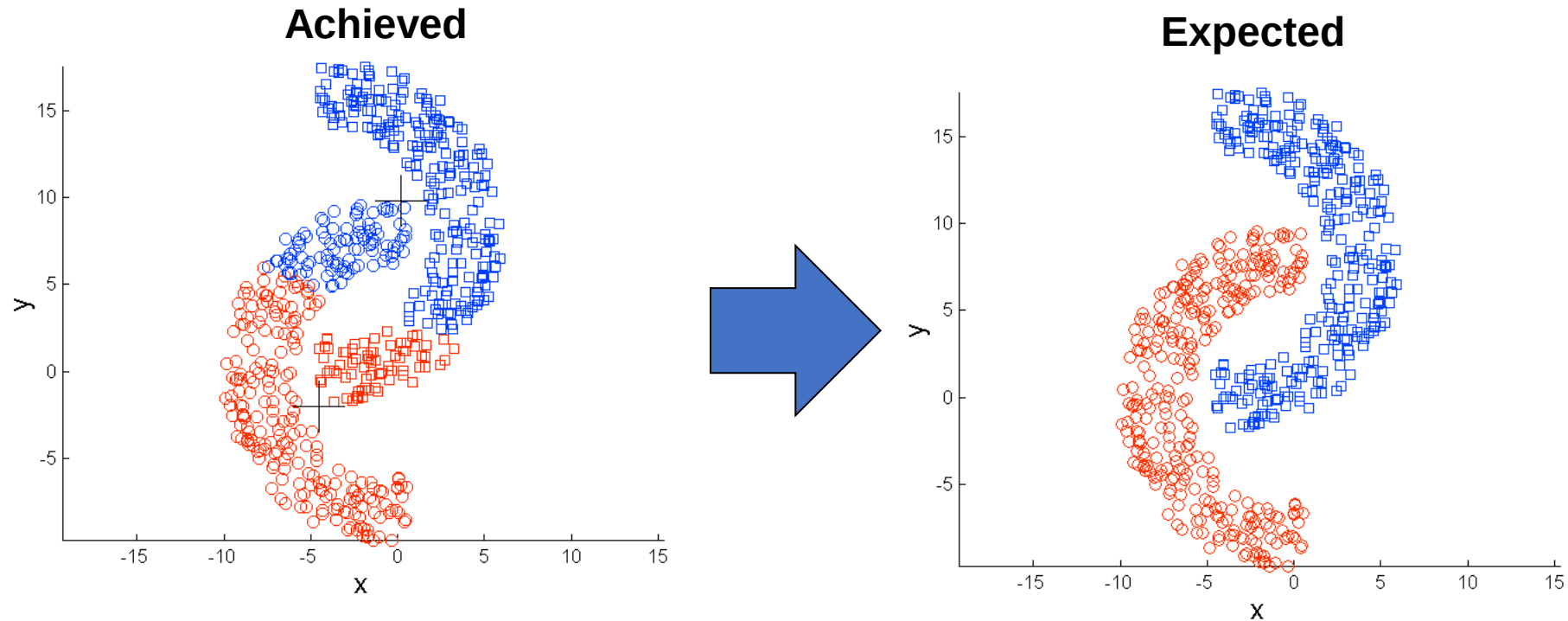


Achieved



Why K-Means fails?

Why K-Means Clustering Fails?

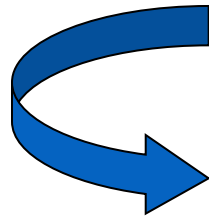
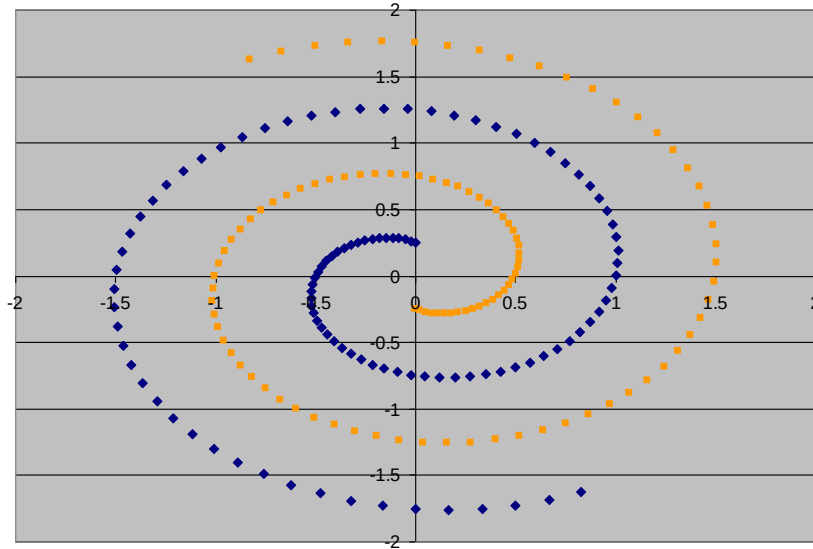


Number of clusters K may not be an issue here

Objective function?

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2$$

Spectral Clustering Example – 2 Spirals

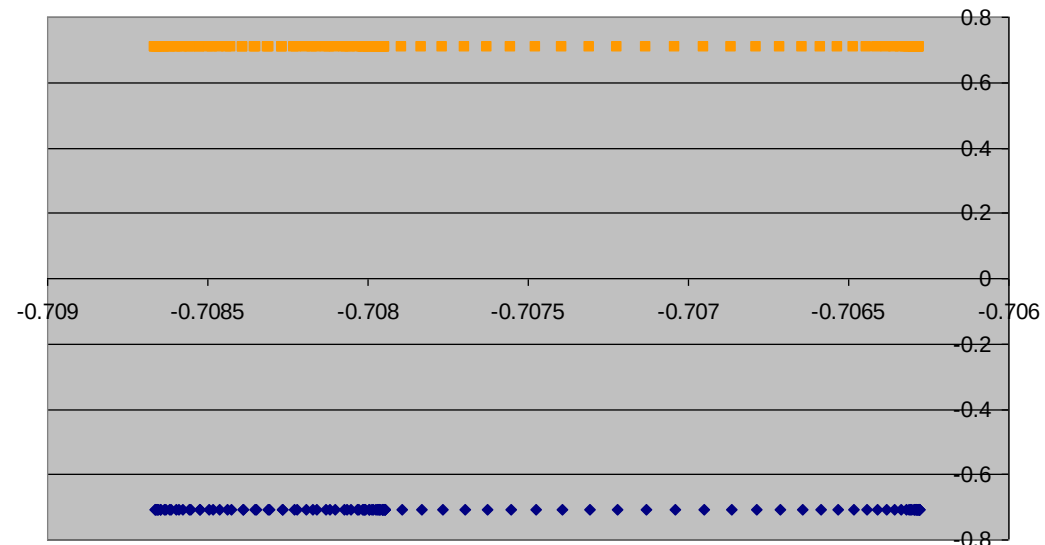


In the embedded space given by two leading eigenvectors, clusters are trivial to separate.

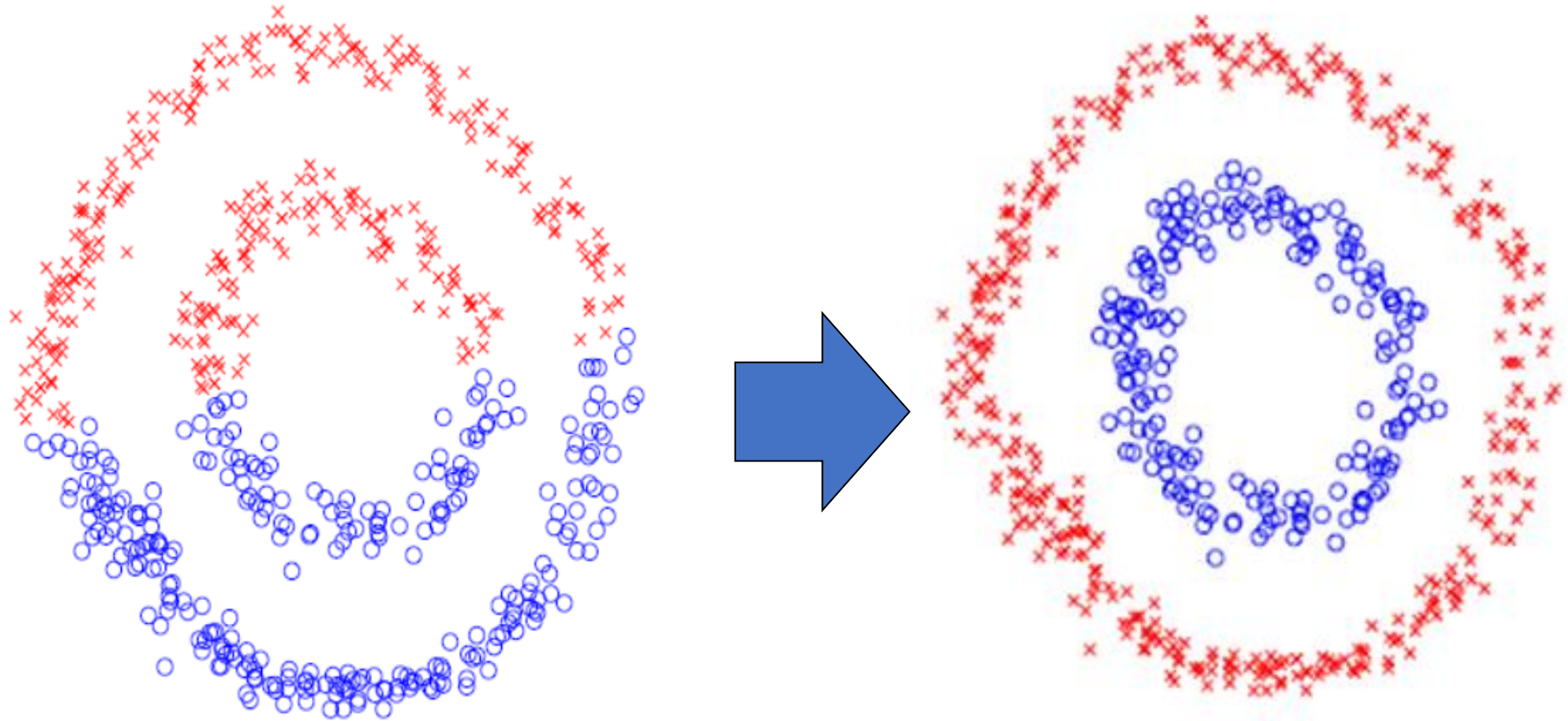
Dataset exhibits **complex cluster shapes**

⇒ **K-means** performs very poorly in this space due bias toward dense spherical clusters.

➡ **Relationship vs. Geometry Distance**

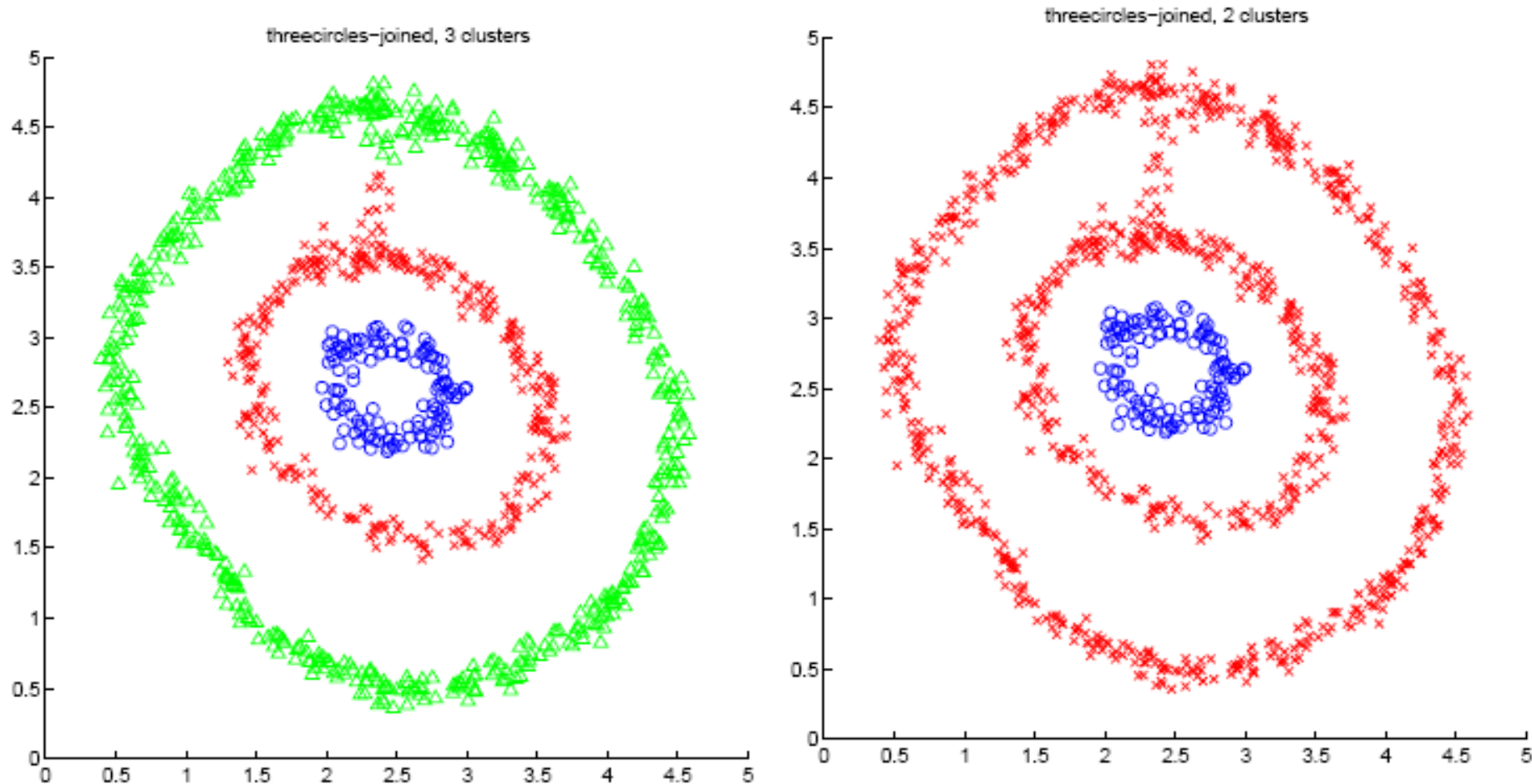


Graph-Based Similarity Representation



Distance vs. Relationship

Graph-Based Similarity Representation

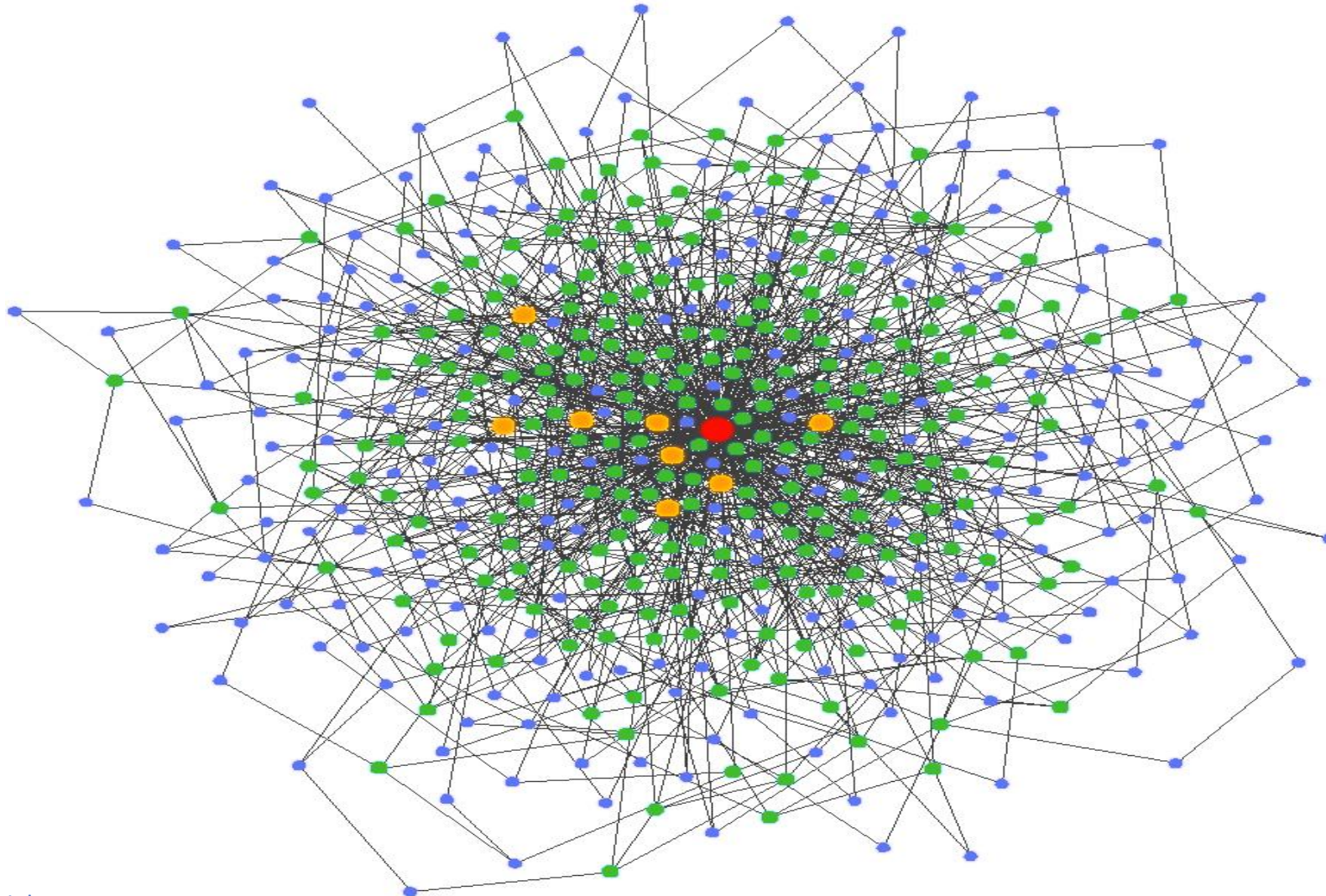


Number of clusters matters

Spectral Clustering

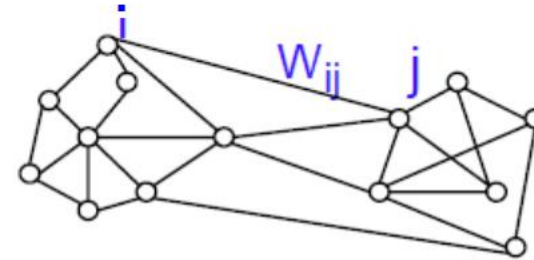
- **Spectral Clustering** --- considering intra-cluster similarity and inter-cluster similarity **jointly!**

How to Do Graph Partitioning?

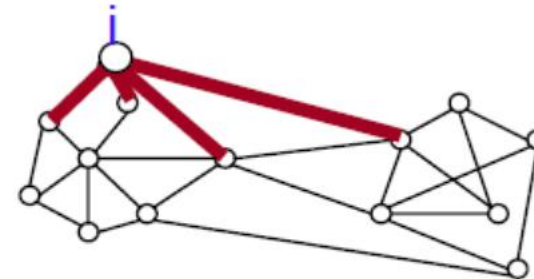


Some notations

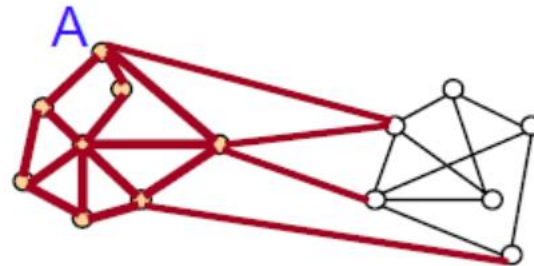
- $W = (w_{ij})$ adjacency matrix of the graph



- $d_i = \sum_j w_{ij}$ degree of a vertex
- $D = \text{diag}(d_1, \dots, d_n)$ degree matrix



- $|A|$ = number of vertices in A
- $\text{vol}(A) = \sum_{i \in A} d_i$



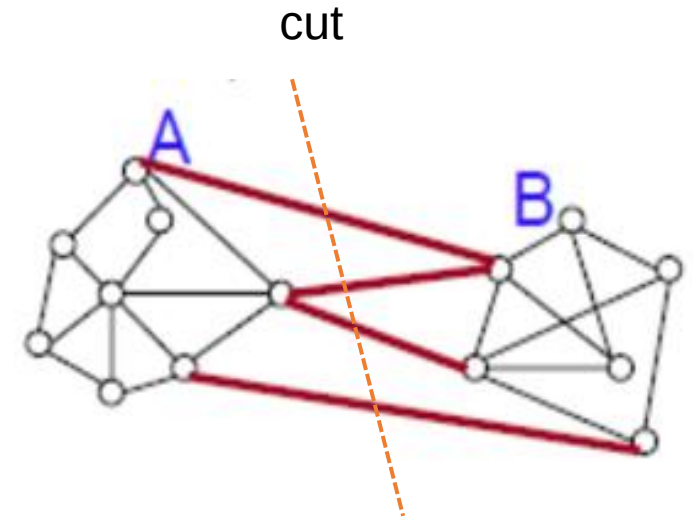
Intra-Cluster Similarity:

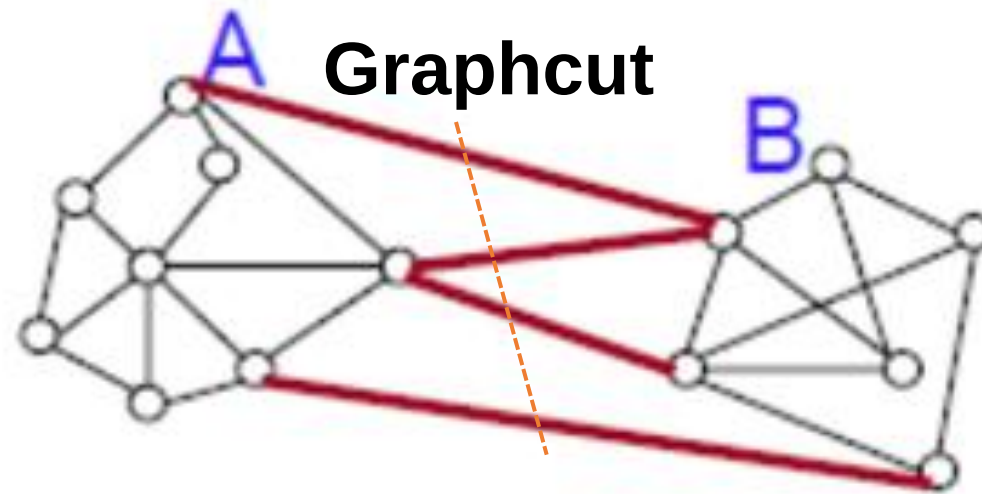
$$\text{vol}(A) = \sum_{i,j \in A} W_{ij}$$

$$\text{vol}(B) = \sum_{k,l \in B} W_{kl}$$

Inter-Cluster Similarity:

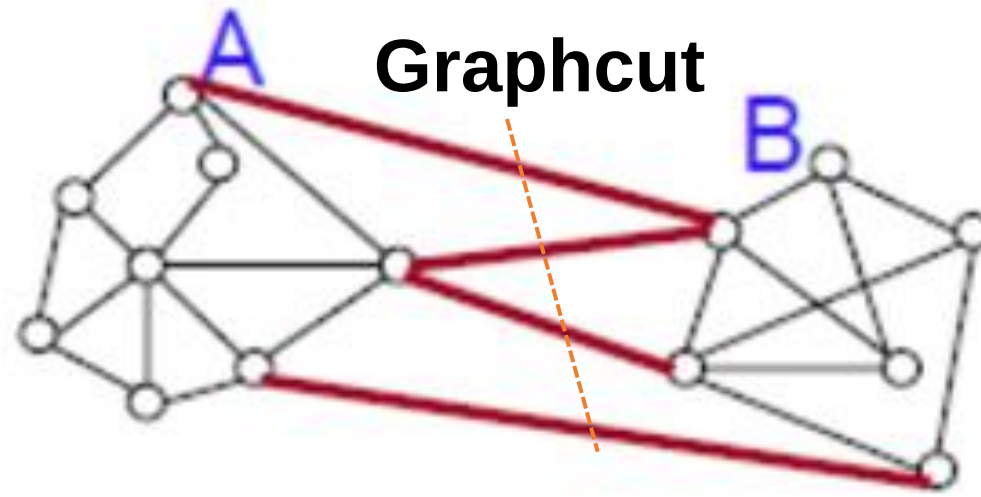
$$\text{vol}(A, B) = \sum_{k \in B} \sum_{i \in A} W_{ik}$$





Objective Function for Spectral Clustering

- **Maximize** Intra-Cluster Similarity
- **Minimize** Inter-Cluster Similarity



Objective Function for Spectral Clustering



$$\text{Max} \left\{ \frac{\text{intra-cluster similarity}}{\text{inter-cluster similarity}} \right\}$$

Clustering using graph cuts

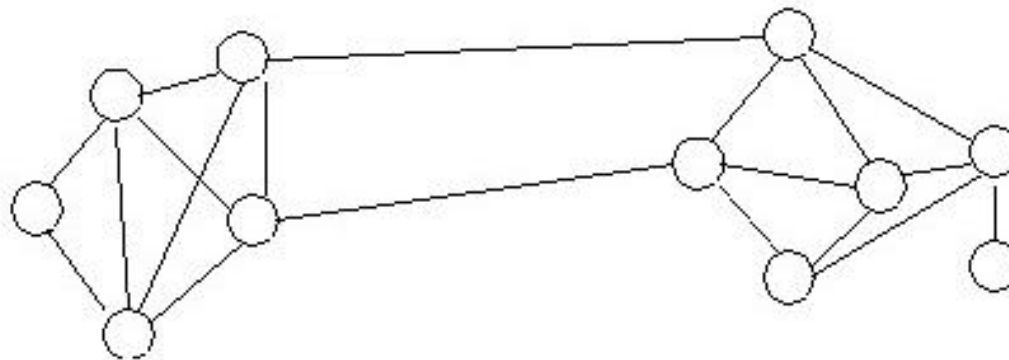
Clustering: within-similarity high, between similarity low

$$\text{minimize } \text{cut}(A, B) := \sum_{i \in A, j \in B} w_{ij}$$

Balanced cuts:

$$\text{RatioCut}(A, B) := \text{cut}(A, B) \left(\frac{1}{|A|} + \frac{1}{|B|} \right)$$

$$\text{Ncut}(A, B) := \text{cut}(A, B) \left(\frac{1}{\text{vol}(A)} + \frac{1}{\text{vol}(B)} \right) \quad \text{normalized}$$



$$\frac{1}{K} \text{cut}(A, B) \left(\frac{1}{|A|} + \frac{1}{|B|} \right)$$

$\text{cut} = \sum w_{ij}$ *sum weights of cut*

K – number of groups

Mincut can be solved efficiently, but RatioCut or Ncut is NP hard.

Spectral clustering: relaxation of RatioCut or Ncut, respectively.

Ratio Cut & Normalized Cut Example

For partition in red: π_1 Cut(Red)

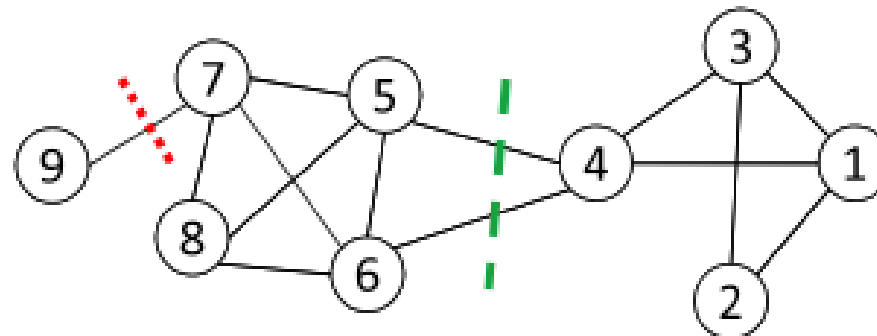
$$\text{Ratio Cut}(\pi_1) = \frac{1}{2} \left(\frac{1}{1} + \frac{1}{8} \right) = 9/16 = 0.56$$

$$\text{Normalized Cut}(\pi_1) = \frac{1}{2} \left(\frac{1}{1} + \frac{1}{27} \right) = 14/27 = 0.52$$

For partition in green: π_2 Cut(Green)

$$\text{Ratio Cut}(\pi_2) = \frac{1}{2} \left(\frac{2}{4} + \frac{2}{5} \right) = 9/20 = 0.45 < \text{Ratio Cut}(\pi_1)$$

$$\text{Normalized Cut}(\pi_2) = \frac{1}{2} \left(\frac{2}{12} + \frac{2}{16} \right) = 7/48 = 0.15 < \text{Normalized Cut}(\pi_1)$$



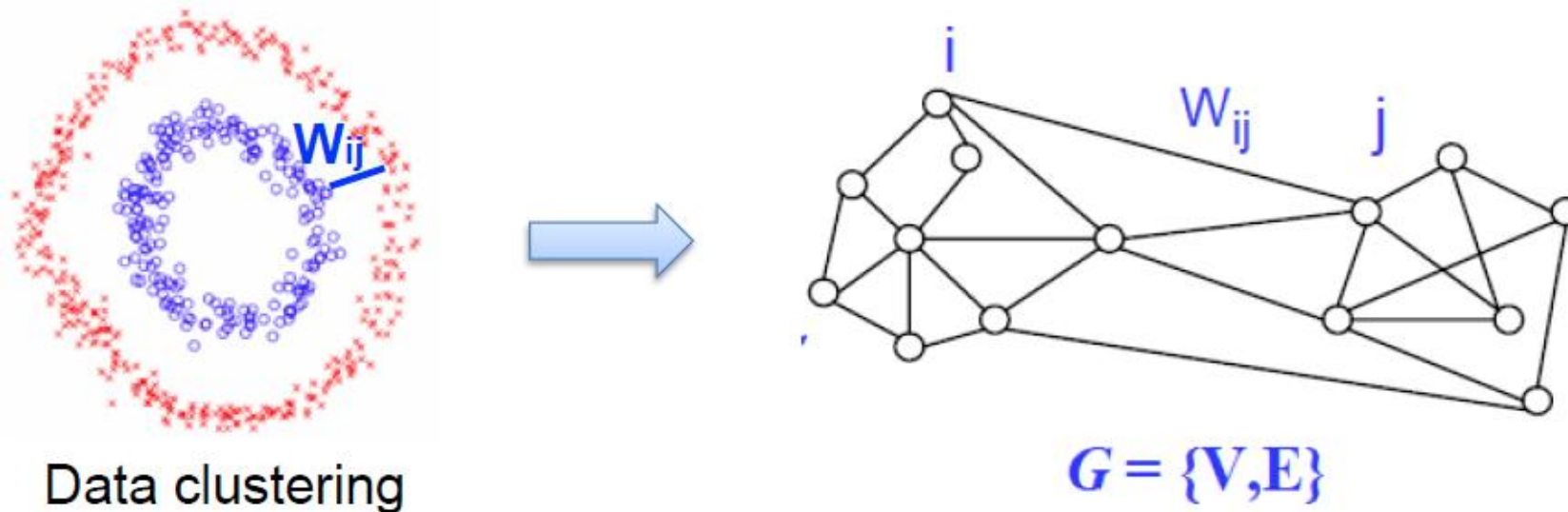
Cut(Red) = 1 Cut(Green) = 2

Num edges cut

Both ratio cut and normalized cut prefer a balanced partition

Similarity graph construction

- Given a set of *data points* and a *similarity metric*
- Encode the data points into a *graph*
- Each *node* is a data point
- Each *edge* is weighted by the similarity of the nodes



Matrix Representations of Graphs

- Adjacency matrix W

$$W = (w_{ij}) \quad i, j = 1, \dots, n \quad w_{ij} \geq 0$$

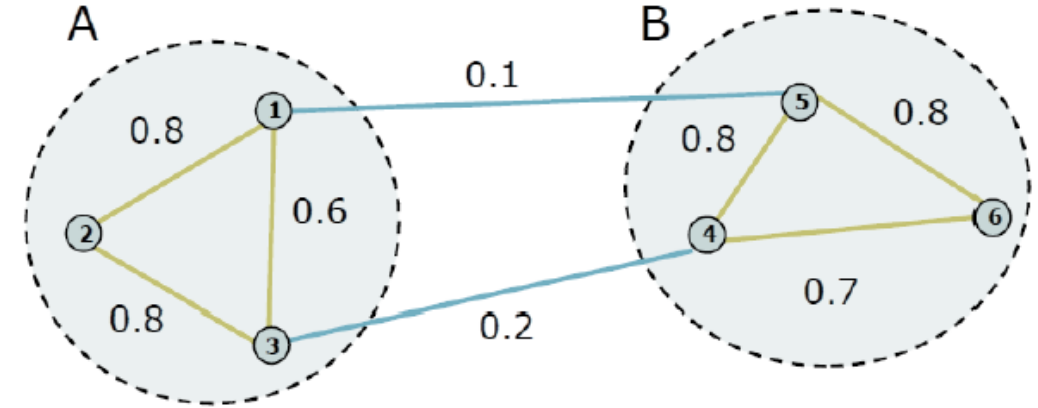
- Degree d_i of a node i

$$d_i = \sum_{j=1}^n w_{ij}$$

- Degree matrix D

Diagonal matrix with the degrees $d_1, \dots, d_n$ on the diagonal

$$W = \begin{pmatrix} 0 & 0.8 & 0.6 & 0 & 0.1 & 0 \\ 0.8 & 0 & 0.8 & 0 & 0 & 0 \\ 0.6 & 0.8 & 0 & 0.2 & 0 & 0 \\ 0 & 0 & 0.2 & 0 & 0.8 & 0.7 \\ 0.1 & 0 & 0 & 0.8 & 0 & 0.8 \\ 0 & 0 & 0 & 0.7 & 0.8 & 0 \end{pmatrix} \quad D = \begin{pmatrix} 1.5 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1.6 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1.6 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1.7 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1.7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1.5 \end{pmatrix}$$



Spectral Clustering Algorithm

Spectral Clustering Algorithm

Spectral Clustering Algorithm

- Input: Graph $S \in \mathbb{R}^{n \times n}$, , number k of clusters to form
 - Compute adjacency matrix W and degree matrix D
 - Laplacian $L = D - W$

Spectral Clustering Algorithm

- Input: Graph $S \in \mathbb{R}^{n \times n}$, number k of clusters to form
 - Compute adjacency matrix W and degree matrix D
 - Laplacian $L = D - W$
 - Compute the first k eigenvectors $u_1, \dots, u_k$ of L
 - Let $U \in \mathbb{R}^{N \times k}$ contain the vectors $u_1, \dots, u_k$ as columns

Spectral Clustering Algorithm

- Input: Graph $S \in \mathbb{R}^{n \times n}$, number k of clusters to form
 - Compute adjacency matrix W and degree matrix D
 - Laplacian $L = D - W$
 - Compute the first k eigenvectors $u_1, \dots, u_k$ of L
 - Let $U \in \mathbb{R}^{N \times k}$ contain the vectors $u_1, \dots, u_k$ as columns

New space found!

Spectral Clustering Algorithm

- Input: Graph $S \in \mathbb{R}^{n \times n}$, number k of clusters to form
 - Compute adjacency matrix W and degree matrix D
 - Laplacian $L = D - W$
 - Compute the first k eigenvectors $u_1, \dots, u_k$ of L
 - Let $U \in \mathbb{R}^{N \times k}$ contain the vectors $u_1, \dots, u_k$ as columns
- Let $y_i \in \mathbb{R}^k$ be the vector corresponding to the i -th row of U
- Cluster the points $(y_i)_{i=1, \dots, N}$ into k clusters using k-means

Spectral Clustering Algorithm

- Input: Graph $S \in \mathbb{R}^{n \times n}$, number k of clusters to form
 - Compute adjacency matrix W and degree matrix D
 - Laplacian $L = D - W$
 - Compute the first k eigenvectors $u_1, \dots, u_k$ of L
 - Let $U \in \mathbb{R}^{N \times k}$ contain the vectors $u_1, \dots, u_k$ as columns

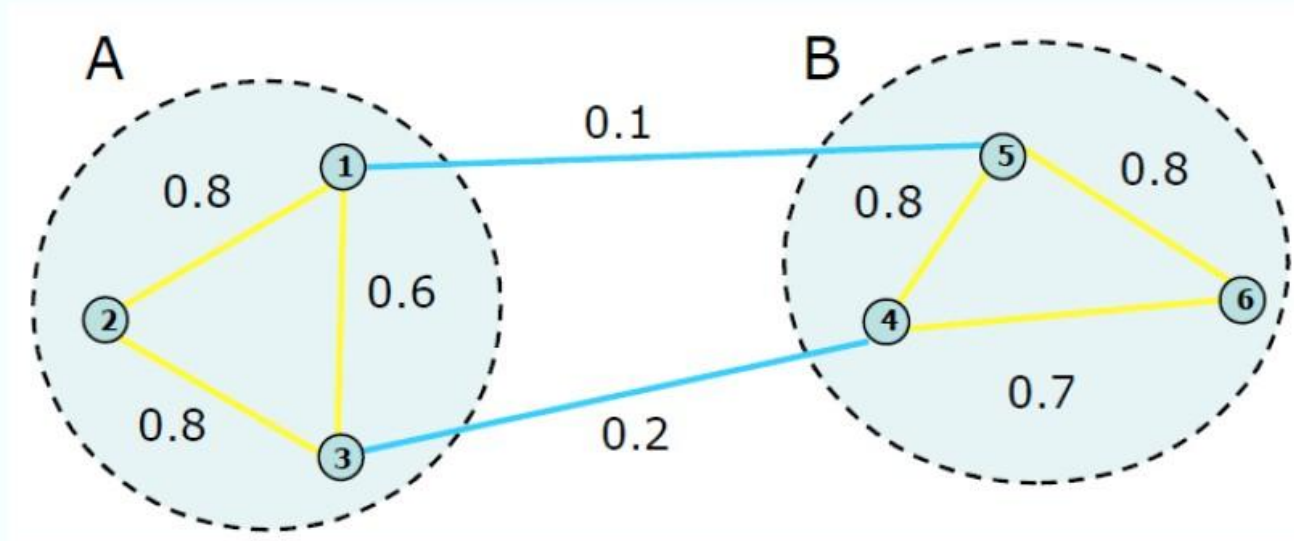
- Let $y_i \in \mathbb{R}^k$ be the vector corresponding to the i -th row of U
- Cluster the points $(y_i)_{i=1, \dots, N}$ into k clusters using k-means

Representing data in the new space!

Time Complexity: $O(n^3)$

Example

- Let's go through an example.
- $n = 6, k = 2$



Example

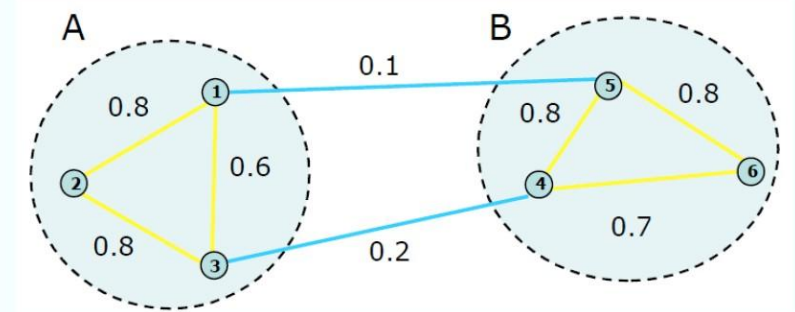
- Step 1: Weighted adjacency matrix W and degree matrix D

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	0	0.8	0.6	0	0.1	0
x_2	0.8	0	0.8	0	0	0
x_3	0.6	0.8	0	0.2	0	0
x_4	0	0	0.2	0	0.8	0.7
x_5	0.1	0	0	0.8	0	0.8
x_6	0	0	0	0.7	0.8	0

Adjacency Matrix W

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	0	0	0	0	0
x_2	0	1.6	0	0	0	0
x_3	0	0	1.6	0	0	0
x_4	0	0	0	1.7	0	0
x_5	0	0	0	0	1.7	0
x_6	0	0	0	0	0	1.5

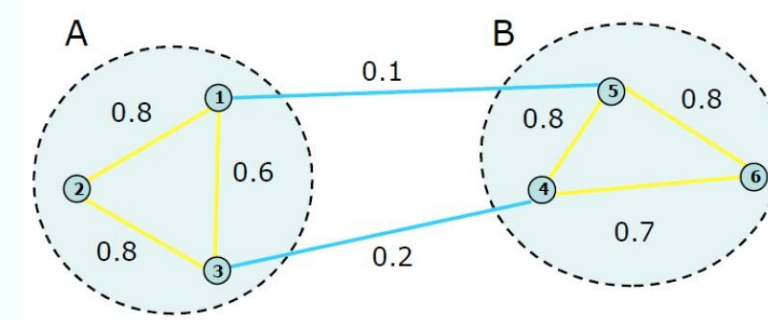
Degree Matrix D



Example

- Step 2: Laplacian matrix L
 - $L=D-W$

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5



Example

- Step 3: Eigen-decomposition

- Eigenvalues =

0
0.18
2.08
2.28
2.46
2.57

- Eigenvectors =

-0.4082	0.4084	...
-0.4082	0.4418	...
-0.4082	0.3713	...
-0.4082	-0.3713	...
-0.4082	-0.4050	...
-0.4082	-0.4452	...

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5

Laplacian Matrix L

Example

- Step 3: Eigen-decomposition

- Eigenvalues =

0
0.18
2.08
2.28
2.46
2.57

- Eigenvectors =

U

-0.4082	0.4084	...
-0.4082	0.4418	...
-0.4082	0.3713	...
-0.4082	-0.3713	...
-0.4082	-0.4050	...
-0.4082	-0.4452	...

	x_1	x_2	x_3	x_4	x_5	x_6
x_1	1.5	-0.8	-0.6	0	-0.1	0
x_2	-0.8	1.6	-0.8	0	0	0
x_3	-0.6	-0.8	1.6	-0.2	0	0
x_4	0	0	-0.2	1.7	-0.8	-0.7
x_5	-0.1	0	0	-0.8	1.7	-0.8
x_6	0	0	0	-0.7	-0.8	1.5

Example

- Step 4: Embedding

- $U =$

-0.4082	0.4084
-0.4082	0.4418
-0.4082	0.3713
-0.4082	-0.3713
-0.4082	-0.4050
-0.4082	-0.4452

Example

- Step 4: Embedding

- $U =$

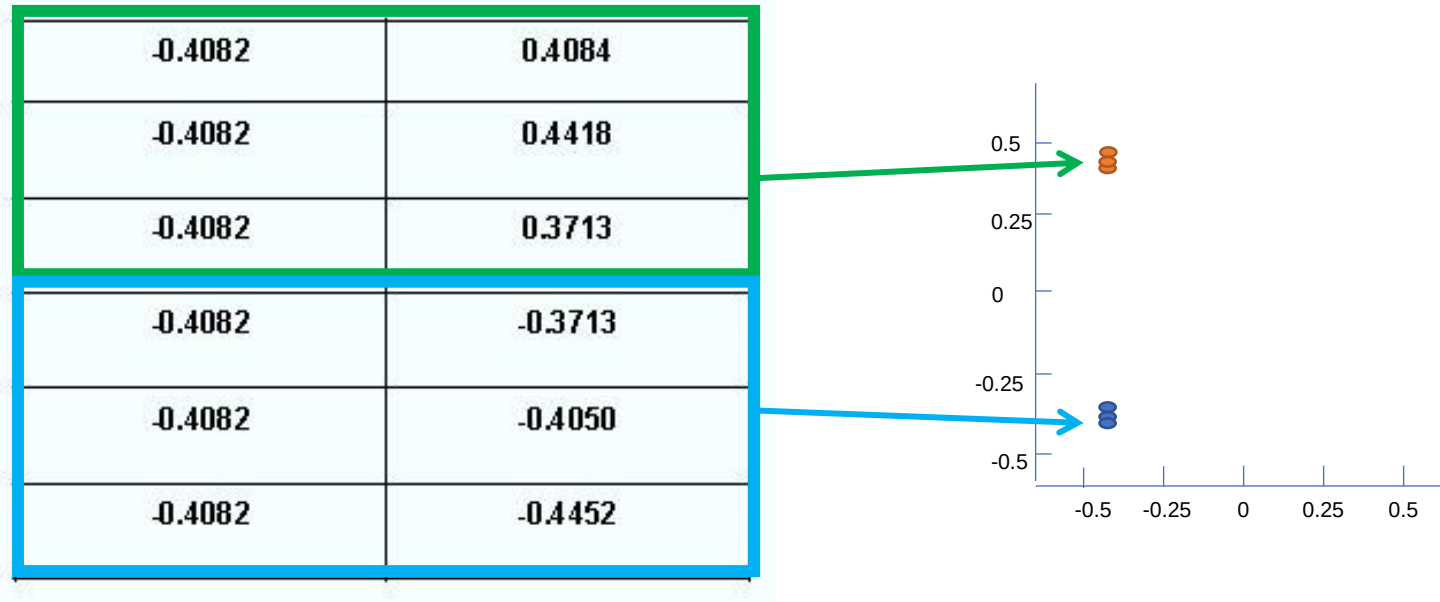
-0.4082	0.4084
-0.4082	0.4418
-0.4082	0.3713
-0.4082	-0.3713
-0.4082	-0.4050
-0.4082	-0.4452

Each row represents a data point for clustering

Example

- Step 4: Embedding

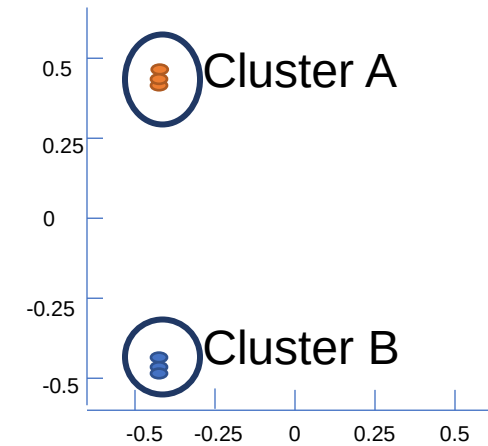
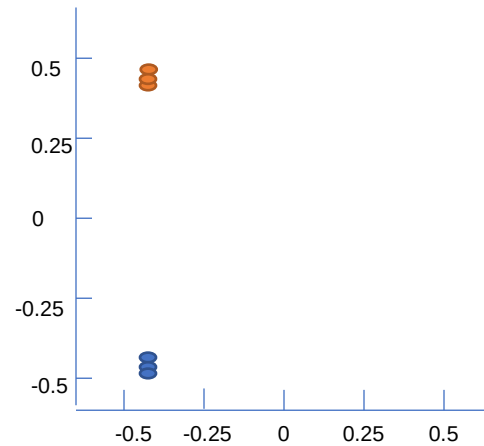
- $U =$



- Map it to a two-dimensional space

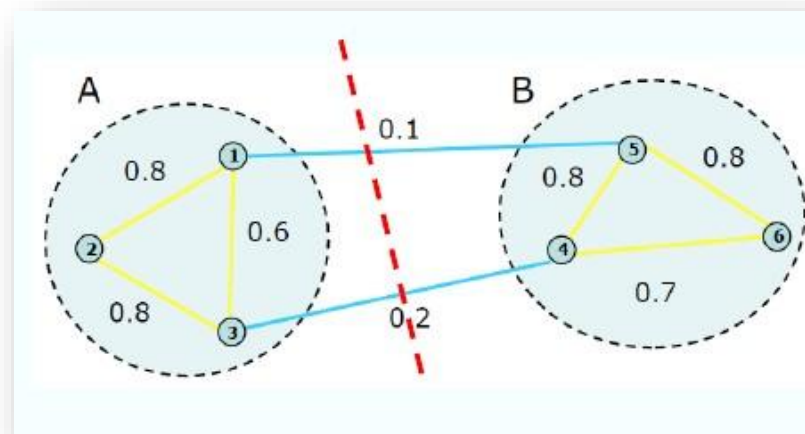
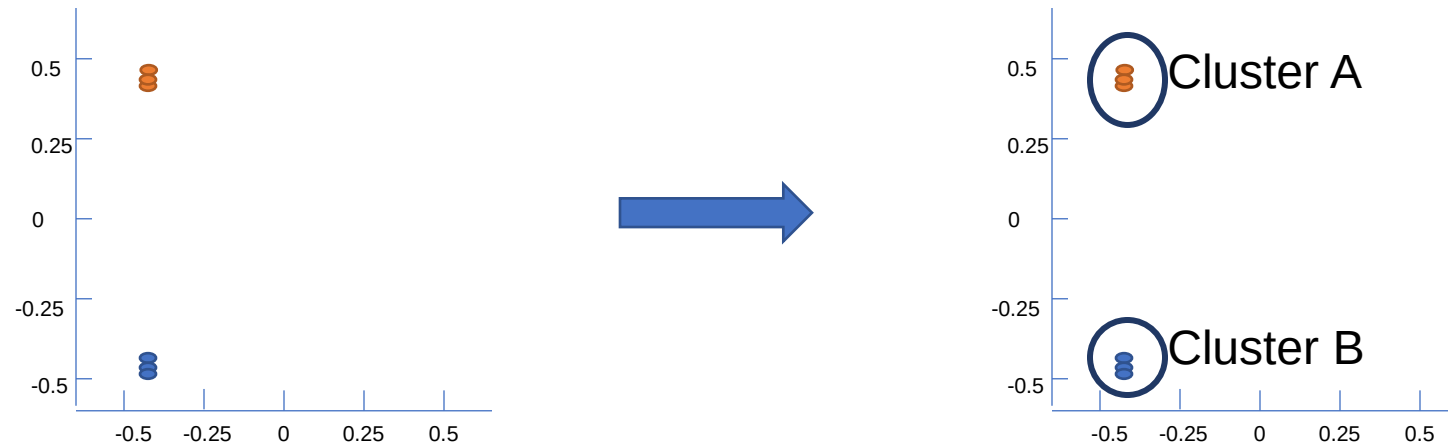
Example

- Step 5: Clustering
 - K-means clustering



Example

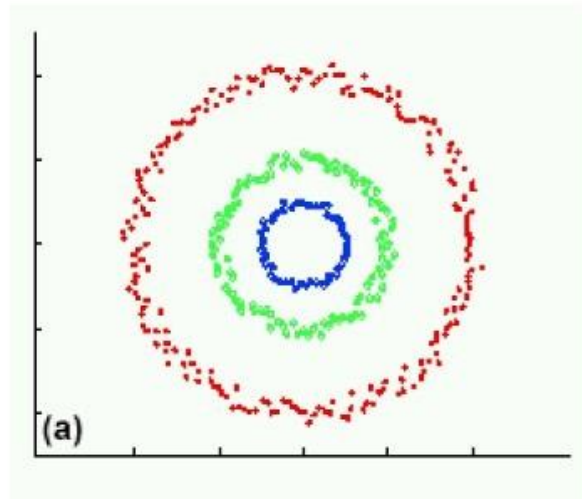
- Step 5: Clustering
 - K-means clustering



Why Spectral Clustering Works?

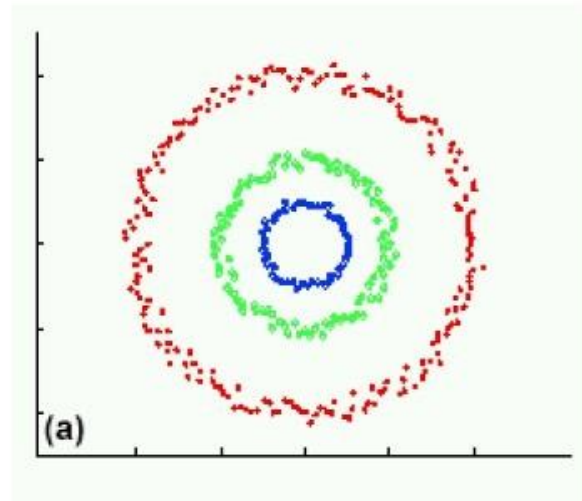
- Consider an ideal case
 - There are no similarities between any nodes in different connected components
 - This conforms to Proposition 2:

Proposition 2 (Number of connected components and the spectrum of L) Let G be an undirected graph with non-negative weights. The multiplicity k of the eigenvalue 0 of L equals the number of connected components $A_1, \dots, A_k$ in the graph. The eigenspace of eigenvalue 0 is spanned by the indicator vectors $\mathbb{1}_{A_1}, \dots, \mathbb{1}_{A_k}$ of those components.



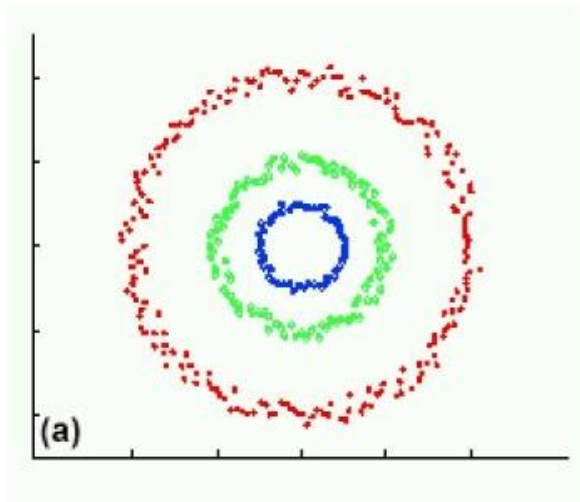
Why Spectral Clustering Works?

- Consider an ideal case
 - There are no similarities between any nodes in different connected components
 - Compute the weighted adjacency matrix W and degree matrix D .
 - $L = D - W$; compute L 's 3 eigenvectors of eigenvalue = 0.



Why Spectral Clustering Works?

- Consider an ideal case
 - There are no similarities between any nodes in different connected components
 - Compute the weighted adjacency matrix W and degree matrix D .
 - $L = D - W$; compute L 's 3 eigenvectors of eigenvalue = 0.



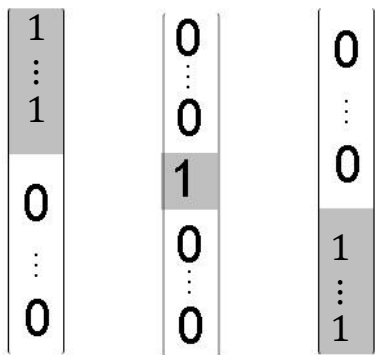
$$L = \begin{bmatrix} L_1 & & \\ & L_2 & \\ & & L_3 \end{bmatrix}$$

$$\begin{bmatrix} 1 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

First three eigenvectors

Why Spectral Clustering Works?

- Consider an ideal case
 - Let the three eigenvectors be three columns of matrix U .

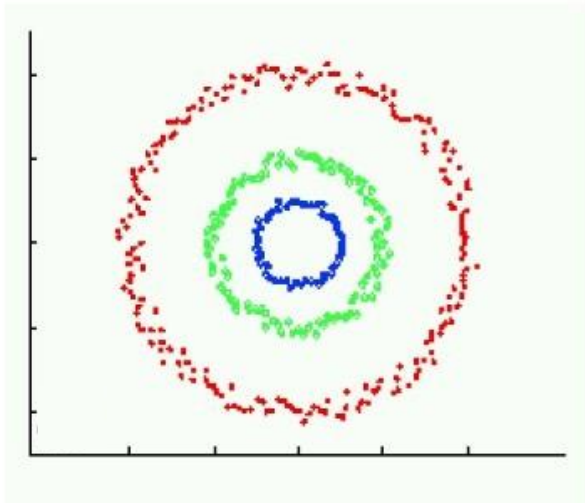
$$L = \begin{bmatrix} L_1 & & 0 \\ & L_2 & \\ 0 & & L_3 \end{bmatrix}$$


First three eigenvectors

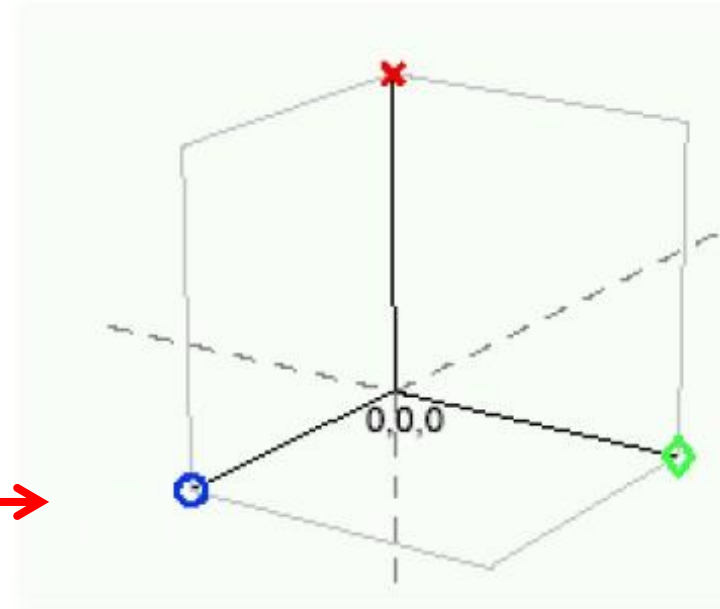
$$U = \begin{bmatrix} 1 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 1 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 1 \end{bmatrix}$$

Why Spectral Clustering Works?

- Consider an ideal case
 - Let the three eigenvectors be three columns of a matrix U .
 - Project the rows in U to a 3-dimensional space.

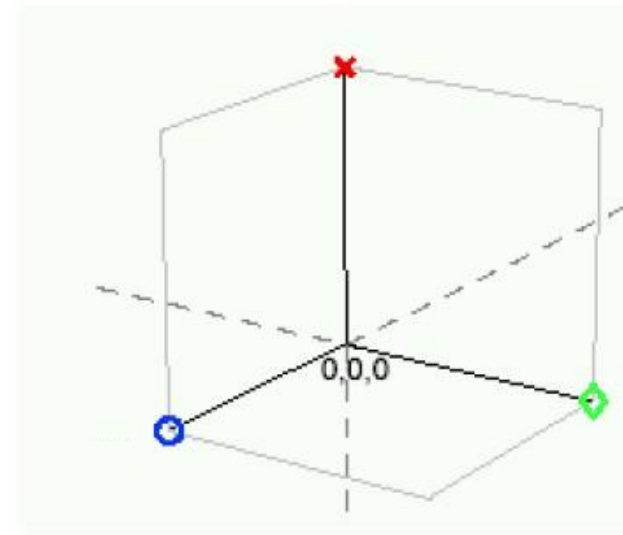
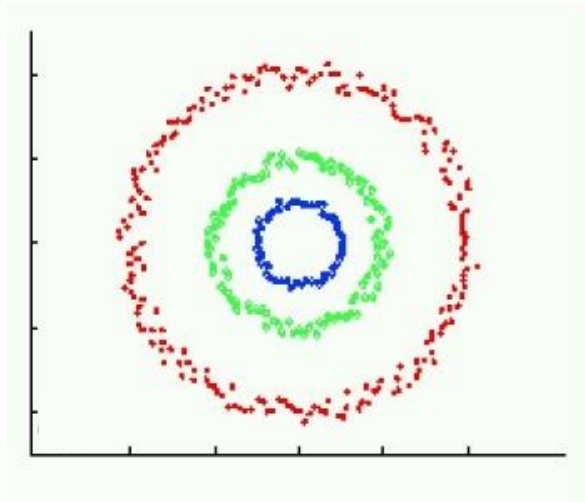


$$U = \begin{bmatrix} 1 & 0 & 0 \\ \vdots & \vdots & \vdots \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ \vdots & \vdots & \vdots \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ \vdots & \vdots & \vdots \\ 0 & 0 & 1 \end{bmatrix}$$



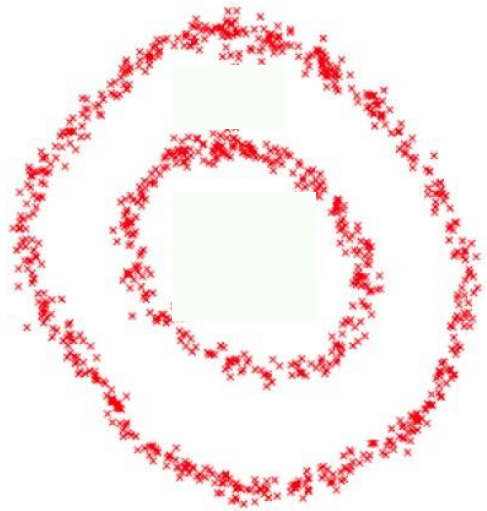
Why Spectral Clustering Works?

- Consider an ideal case
 - Now we use K-Means in this space, we can have very good results.
 - # of 0 eigenvalues = # of connected components



Why Spectral Clustering Works?

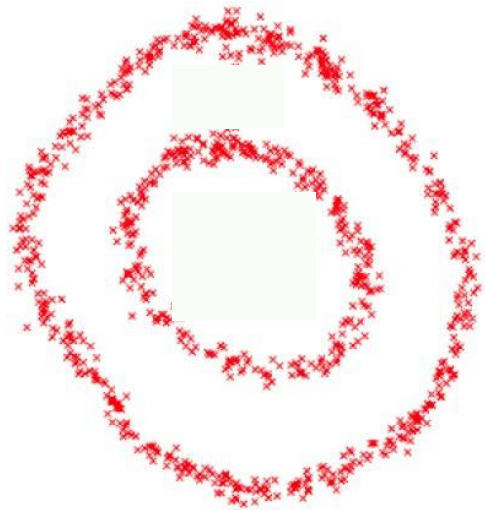
- What if **not** the ideal case?
 - We need to introduce Perturbation Theory.



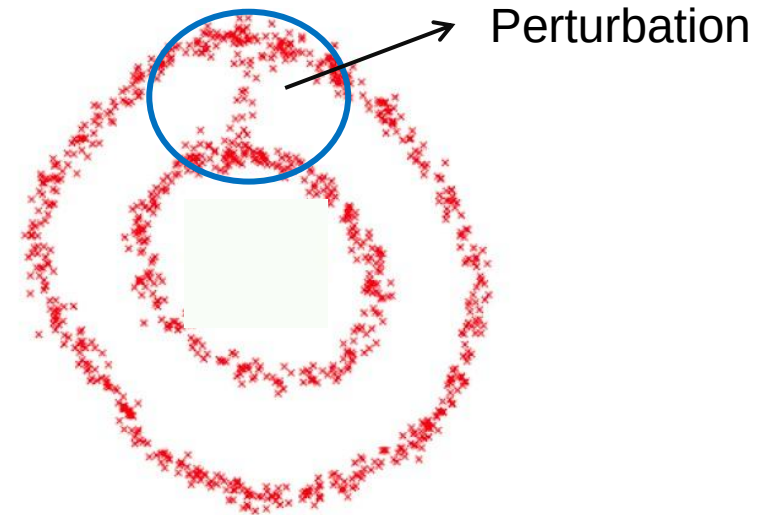
Ideal Case

Why Spectral Clustering Works?

- What if not the ideal case?
 - We need to introduce Perturbation Theory.
 - Perturbation is like noise.



Ideal Case

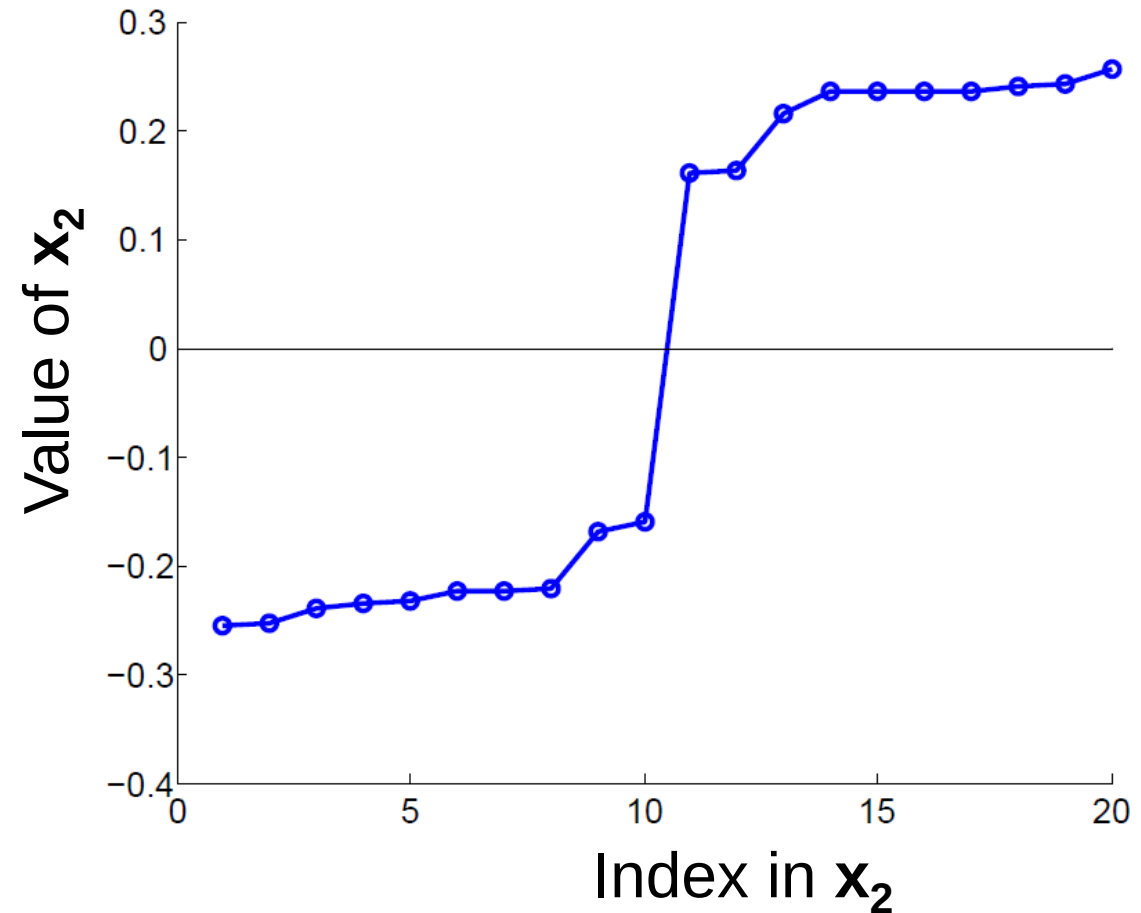
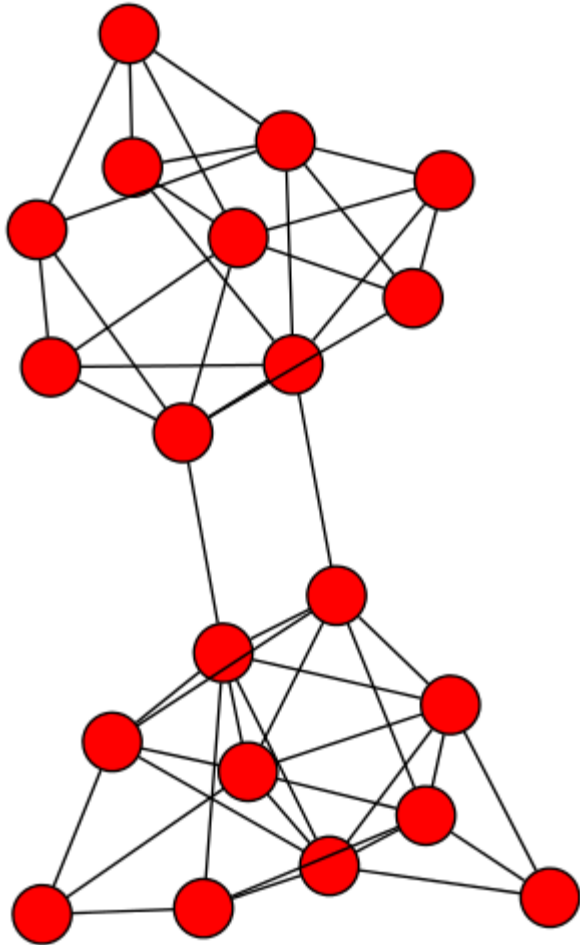


Nearly ideal Case

Why Spectral Clustering Works?

- What if not the ideal case?
- Perturbation Theory will not be formally discussed here.
 - For ideal case, the between-cluster similarity is 0.
 - The first k eigenvectors of Laplacian matrix L are indicators of clusters.
 - For real case, $L' = L + H$, where H is the perturbation.
- Perturbation theory tells us the eigenvectors generated from L' will be **very close** to the ideal vectors from L , bounded by **a small value**.

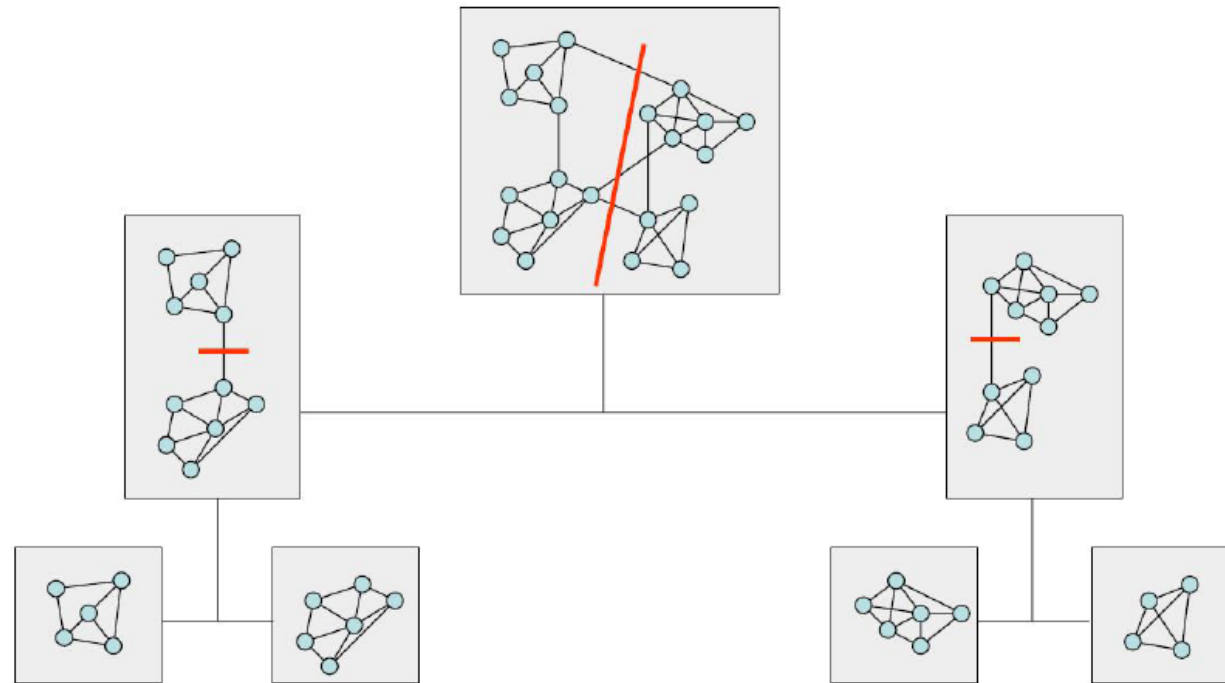
Example: Spectral Partitioning



k -Way Spectral Clustering

How do we partition a graph into k clusters?

- **Recursively apply a bi-partitioning algorithm** in a hierarchical divisive manner
 - Disadvantages: Inefficient, unstable



Spectral Clustering summary

- Use the lowest k eigenvalues of L to construct the $n \times k$ graph G' that has these eigenvectors as columns
- *The n -rows represent the graph vertices in a k -dimensional Euclidean space*
- Group these vertices in k clusters using k-means clustering or similar techniques

User's Prerogative

- Affinity matrix construction
- Choice of scaling factor
 - Realistically, search over σ^2 and pick value that gives the tightest clusters
- Choice of k , the number of clusters
- Choice of clustering method

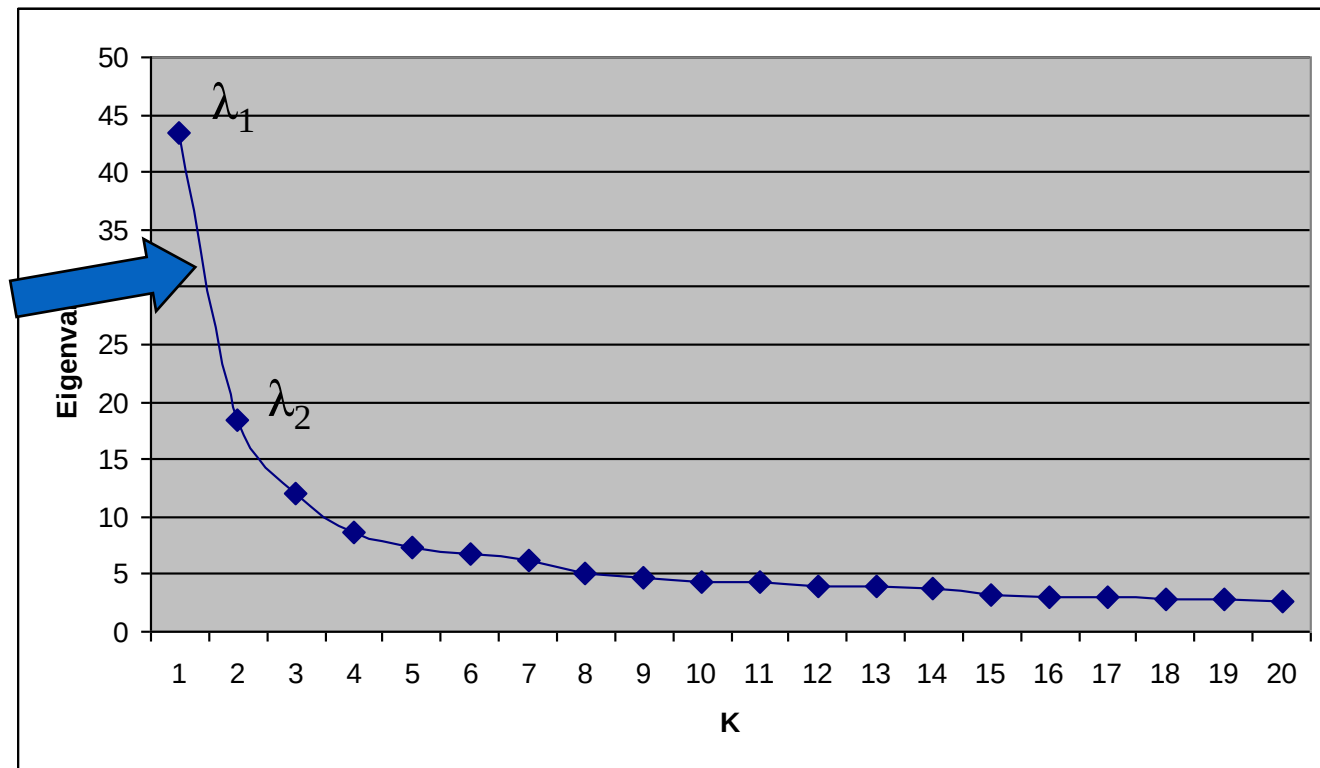
How to select k ?

- *Eigengap*: the difference between two consecutive eigenvalues.
- Most stable clustering is generally given by the value k that maximises the expression

$$\Delta_k = |\lambda_k - \lambda_{k-1}|$$

$$\max \Delta_k = |\lambda_2 - \lambda_1|$$

$\Rightarrow$ Choose $k=2$



Modularity (Newman)

Modularity

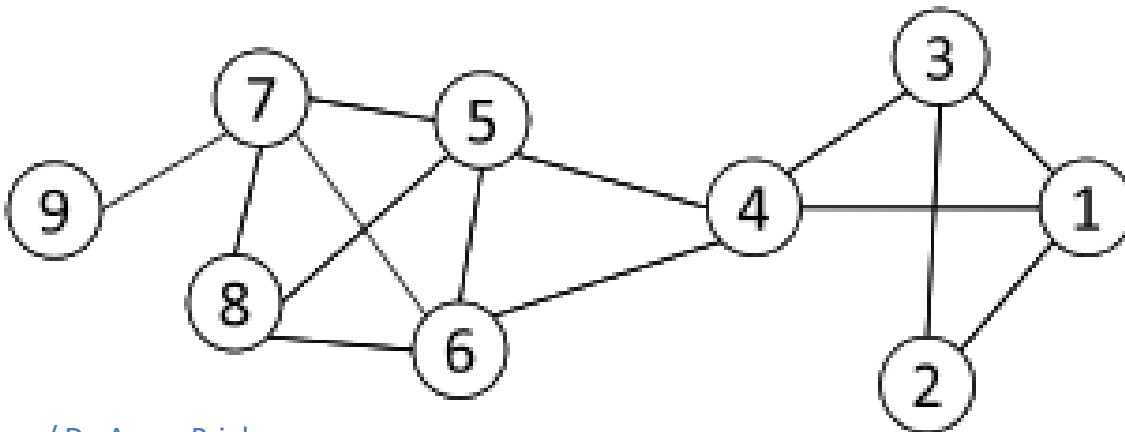
- How can we tell how “good” a partition is?
- Modularity measures the **strength of a community partition** by taking into account the **degree distribution**
- A random network is not expected to have communities, because nodes are connected to each other randomly
- Independent of link density - **random networks do not have communities**
- Consequently, a random subnetwork does not make a good community
- **We need a better way to measure the quality of a community** (partition)

Modularity

- The modularity of a partition evaluates the communities **not in absolute terms**, but with respect to a *random baseline*
- It's done by discounting the internal links that could be attributed to a randomized version of the original network.
- Or, the **modularity** is the difference between the number of links internal to all the clusters and the value of this number expected in a randomized network.

Modularity (Maximization)

- Modularity measures the strength of a community partition by taking into account the degree distribution
- Given a network with m edges, the expected number of edges between two nodes in a random graph with degrees d_i and d_j is $d_i d_j / 2m$

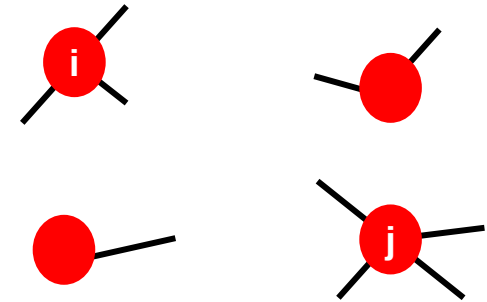


The expected number of edges between **nodes 1 and 2** is $3 \cdot 2 / (2 \cdot 14) = 3/14$

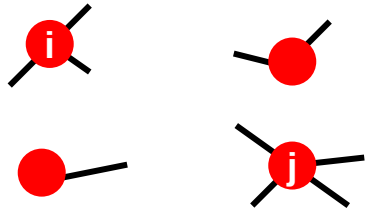
Null Model: Configuration Model

- Given real G on n nodes and m edges, construct rewired network:
 - Same degree distribution but random connections
 - The expected number of edges between nodes i and j of degrees d_i and d_j equals to:

$$d_i \cdot \frac{d_j}{2m} = \frac{d_i d_j}{2m}$$



Null Model: Configuration Model



$$d_i \cdot \frac{d_j}{2m} = \frac{d_i d_j}{2m}$$

Note:

$$\sum_{u \in N} d_u = 2m$$

For any edge going out of i randomly,

the probability of this edge getting connected to node j is $\frac{d_j}{2m}$

Because the degree for i is d_i , we have d_i number of such edges

Modularity (Maximization)

- **Strength of a community:** $\sum_{i \in C, j \in C} A_{ij} - d_i d_j / 2m$ $d_i d_j / 2m$

Given the degree distribution

- **Modularity:** $Q = \frac{1}{2m} \sum_{\ell=1}^k \sum_{i \in C_\ell, j \in C_\ell} (A_{ij} - d_i d_j / 2m)$

- A larger value indicates a good community structure

Modularity

- Modularity of partitioning S of graph G :

- $Q \propto \sum_{s \in S} [(\# \text{ edges within group } s) - (\text{expected } \# \text{ edges within group } s)]$

- $$Q(G, S) = \frac{1}{2m} \sum_{s \in S} \sum_{i \in s} \sum_{j \in s} \left(A_{ij} - \frac{d_i d_j}{2m} \right)$$

Normalizing cost.: $-1 < Q < 1$

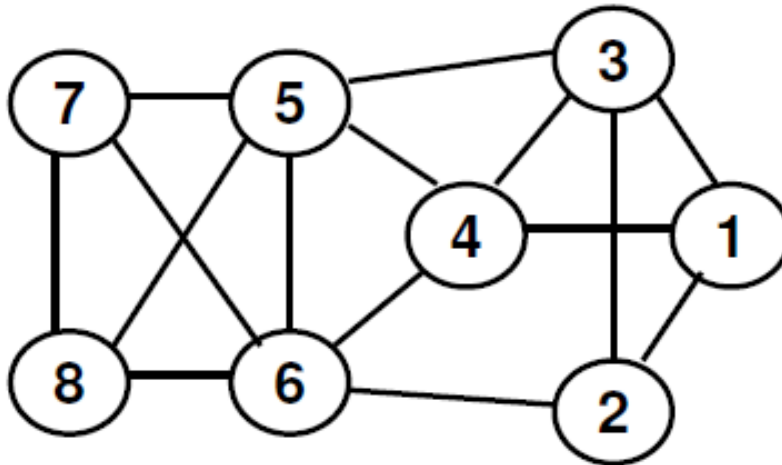
$$A_{ij} = \begin{cases} 1 & \text{if } i \rightarrow j, \\ 0 & \text{else} \end{cases}$$

- Modularity values take range $[-1, 1]$

- It is positive if the number of edges within groups exceeds the expected number
- $0.3-0.7 < Q$ means significant community structure

Modularity example

$m = 15$ edges



Community [1, 4, 5, 7]

Edges with $A_{ij} = 1$ Modularity

1 – 4	$1 - (3)(4)/(2*15) = 0.60$
4 – 5	$1 - (4)(5)/(2*15) = 0.33$
5 – 7	$1 - (3)(5)/(2*15) = 0.50$

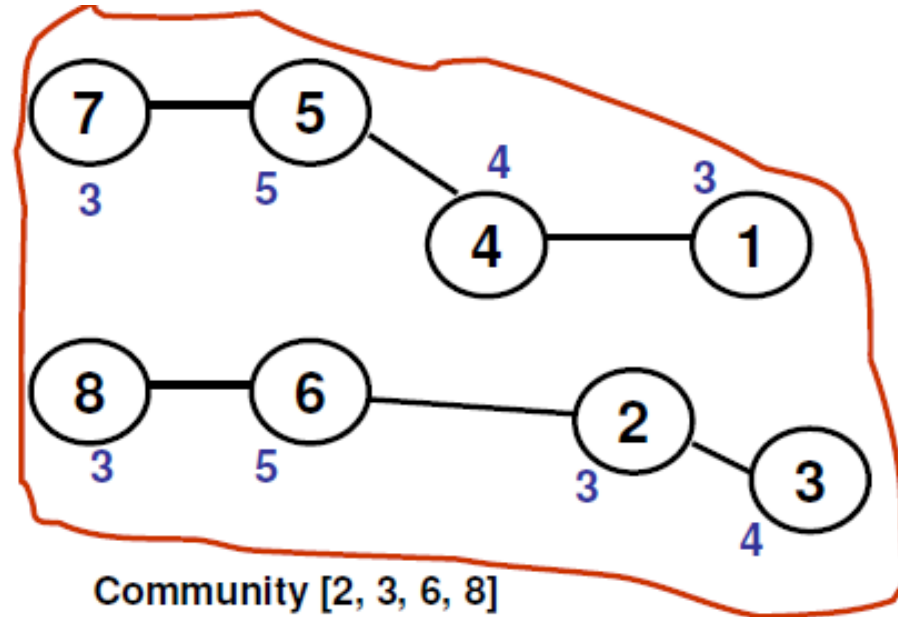
Edges with $A_{ij} = 0$

1 – 5	$0 - (3)(5)/(2*15) = -0.50$
1 – 7	$0 - (3)(3)/(2*15) = -0.30$
4 – 7	$0 - (4)(3)/(2*15) = -0.40$

Total Modularity Score for Community [1, 4, 5, 7] 0.23

Total Modularity for the two Communities: $0.23 + 0.23 = 0.46$

Should normalize Q by $2*m$



Community [2, 3, 6, 8]

Edges with $A_{ij} = 1$ Modularity

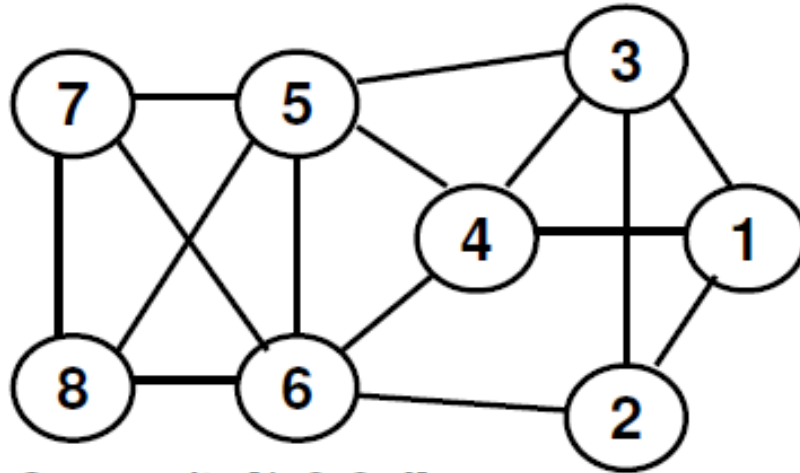
2 – 3	$1 - (3)(4)/(2*15) = 0.60$
2 – 6	$1 - (3)(5)/(2*15) = 0.50$
6 – 8	$1 - (3)(5)/(2*15) = 0.50$

Edges with $A_{ij} = 0$

2 – 8	$0 - (3)(3)/(2*15) = -0.30$
3 – 6	$0 - (4)(5)/(2*15) = -0.67$
3 – 8	$0 - (4)(3)/(2*15) = -0.40$

Total Modularity Score for Community [2, 3, 6, 8] 0.23

Modularity example



Community [1, 2, 3, 4]

Edges with $A_{ij} = 1$ Modularity

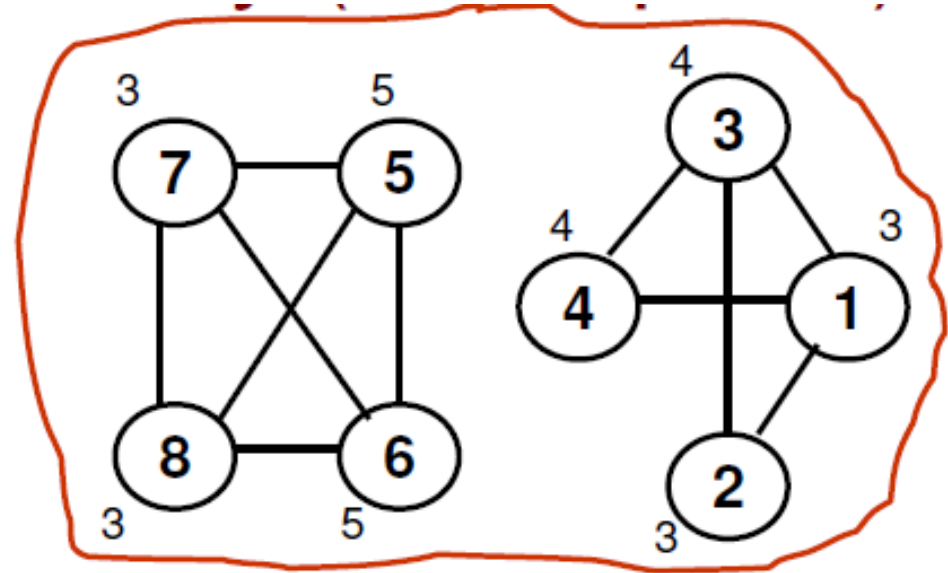
1 – 2	$1 - (3)(3)/(2*15) = 0.70$
1 – 3	$1 - (3)(4)/(2*15) = 0.60$
1 – 4	$1 - (3)(4)/(2*15) = 0.60$
2 – 3	$1 - (3)(3)/(2*15) = 0.70$
3 – 4	$1 - (4)(4)/(2*15) = 0.47$

Edges with $A_{ij} = 0$

2 – 4	$0 - (3)(4)/(2*15) = -0.40$
-------	-----------------------------

Total Modularity Score for
Community [1, 2, 3, 4] 2.67

Total Modularity for the two
Communities: $2.67 + 2.87 = 5.54$



Community [5, 6, 7, 8]

Edges with $A_{ij} = 1$ Modularity

5 – 6	$1 - (5)(5)/(2*15) = 0.17$
5 – 7	$1 - (3)(5)/(2*15) = 0.50$
5 – 8	$1 - (3)(5)/(2*15) = 0.50$
6 – 7	$1 - (3)(5)/(2*15) = 0.50$
6 – 8	$1 - (3)(5)/(2*15) = 0.50$
7 – 8	$1 - (3)(3)/(2*15) = 0.70$

Total Modularity Score for
Community [2, 3, 6, 8] 2.87

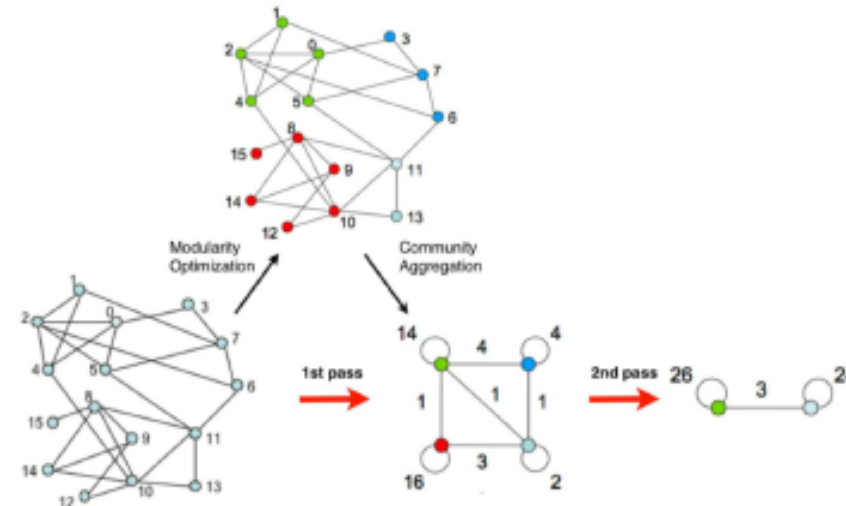
Louvain Algorithm

- **Greedy algorithm** for community detection
 - $O(n \log n)$ run time
- Supports weighted graphs
- Provides hierarchical partitions
- Widely utilized to study large networks because:
 - Fast
 - Rapid convergence properties
 - High modularity output (i.e., “better communities”)

Louvain Algorithm – High Level

- Louvain algorithm **greedily maximizes** modularity
- **Each pass is made of 2 phases:**
 - **Phase 1:** Modularity is **optimized** by allowing only local changes of communities
 - **Phase 2:** The identified communities are **aggregated** in order to build a new network of communities
- **Goto Phase 1**

The passes are repeated **iteratively** until no increase of modularity is possible!



Louvain Algorithm: 1'st phase, partitioning

- Put each node in a graph into a **distinct community** (one node per community)
- For each node i , the algorithm performs two calculations:
 - Compute the modularity gain (ΔQ) when putting node i into the community of some neighbor j
 - Move i to a community of node j that yields the largest gain ΔQ
- **The loop runs until no movement yields a gain**

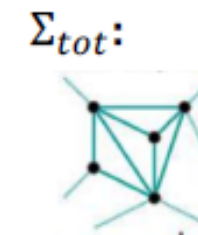
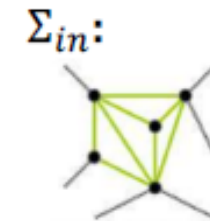
Louvain Algorithm: Modularity Gain

What is ΔQ if we move node i to community C ?

$$\Delta Q(i \rightarrow C) = \left[\frac{\Sigma_{in} + k_{i,in}}{2m} - \left(\frac{\Sigma_{tot} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

■ where:

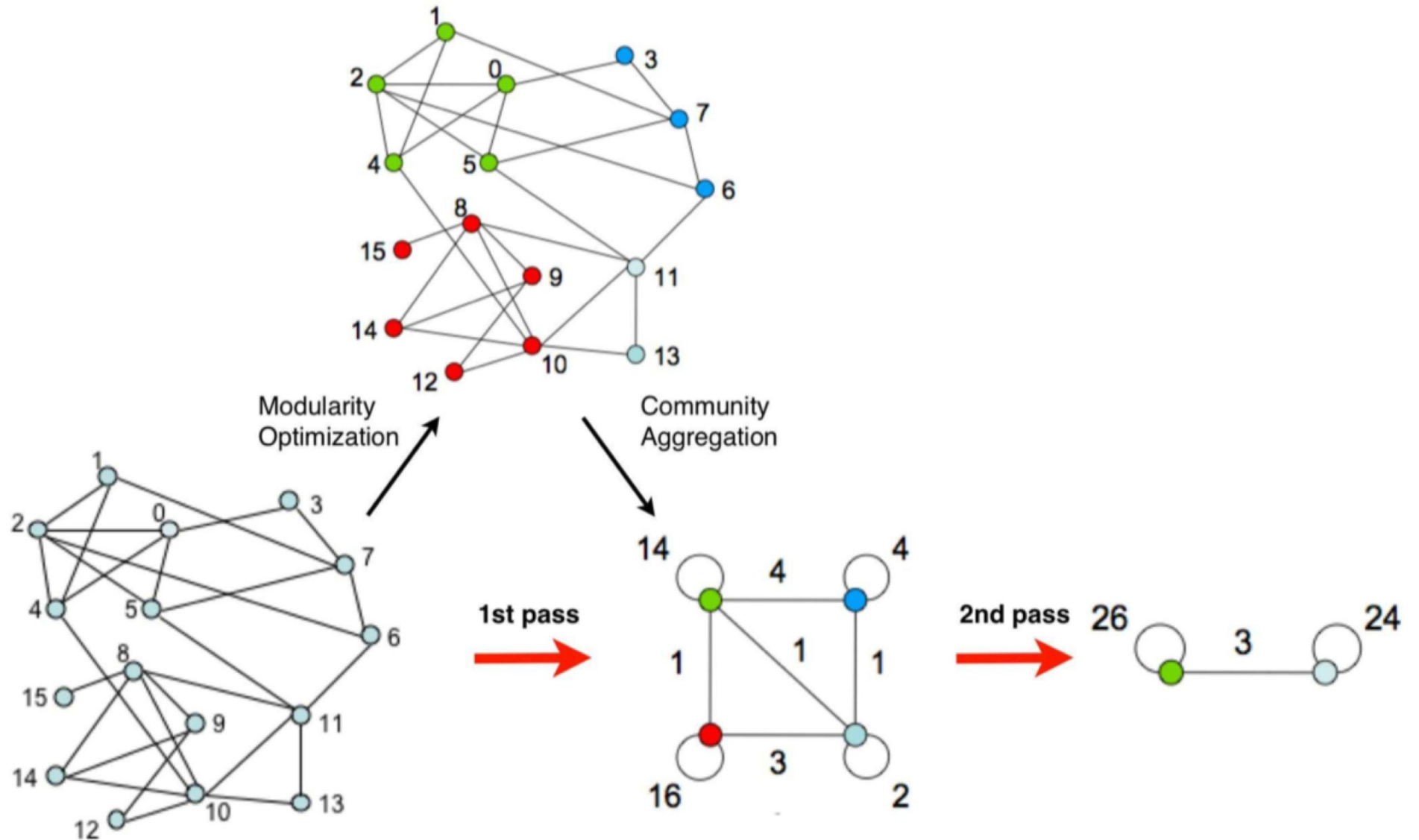
- Σ_{in} ... sum of link weights between nodes in C
- Σ_{tot} ... sum of all link weights of nodes in C
- $k_{i,in}$... sum of link weights between node i and C
- k_i ... sum of all link weights (i.e., degree) of node i
- Also need to derive $\Delta Q(D \rightarrow i)$ of taking node i out of community D .
- And then: $\Delta Q = \Delta Q(i \rightarrow C) + \Delta Q(D \rightarrow i)$



Louvain Algorithm: 2'nd phase, restructuring

- The partitions obtained in the first phase are contracted into **super-nodes**, and the network is created accordingly
 - Super-nodes are connected if there is at least one edge between nodes of the corresponding partitions
 - The weight of the edge between the two super-nodes is the sum of the weights from all edges between their corresponding partitions
- The loop runs until the community configuration does not change anymore

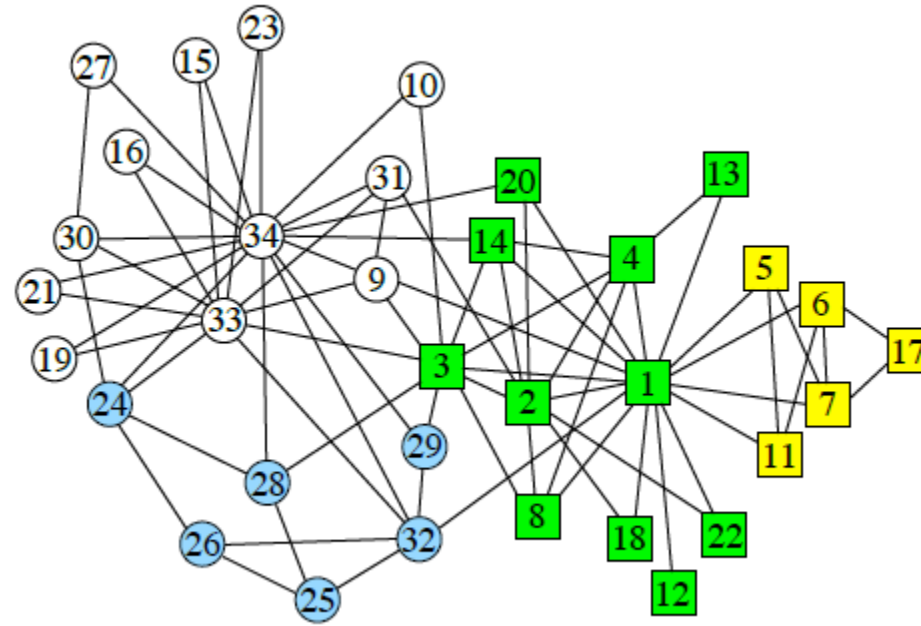
Louvain Algorithm: Example



Community Evaluation

- **With ground truth**
- **Without ground truth**

Evaluation with ground truth



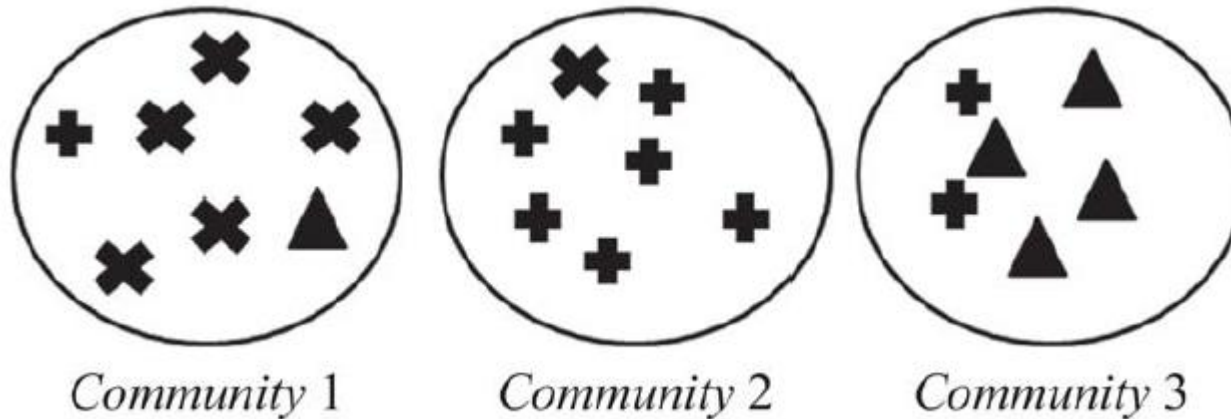
Zachary's Karate Club

Club president (34) (circles) and instructor (1) (rectangles)

Metrics: purity

the fraction of instances that have labels equal to the label of the community's majority

$$Purity = \frac{1}{N} \sum_{i=1}^k \max_j |C_i \cap L_j|$$



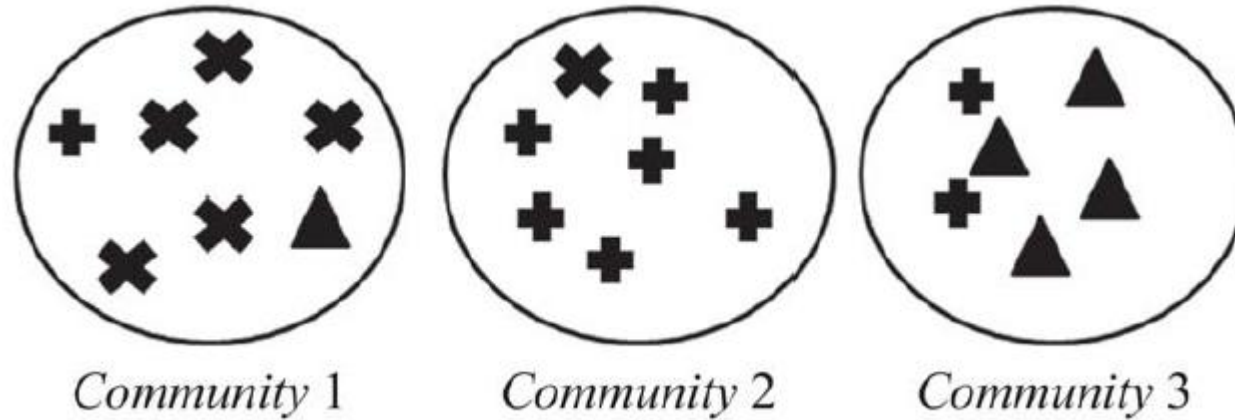
$$(5+6+4)/20 = 0.75$$

Metrics

Based on pair counting: the number of pairs of vertices which are classified in the same (different) clusters in the two partitions.

- **True Positive (TP) Assignment:** when **similar members** are assigned to the **same community**. This is a correct decision.
- **True Negative (TN) Assignment:** when **dissimilar members** are assigned to **different communities**. This is a correct decision.
- **False Negative (FN) Assignment:** when **similar members** are assigned to **different communities**. This is an incorrect decision.
- **False Positive (FP) Assignment:** when **dissimilar members** are assigned to the **same community**. This is an incorrect decision.

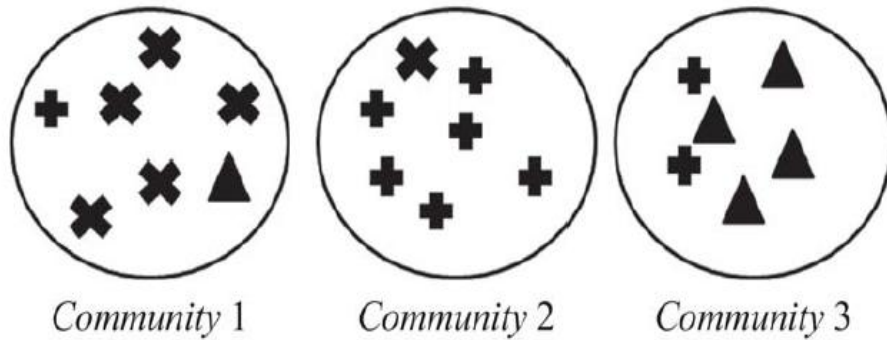
Metrics: pairs



For TP, we need to compute the number of pairs with the same label that are in the same community

$$TP = \underbrace{\binom{5}{2}}_{\text{Community 1}} + \underbrace{\binom{6}{2}}_{\text{Community 2}} + \underbrace{\left(\binom{4}{2} + \binom{2}{2}\right)}_{\text{Community 3}} = 32$$

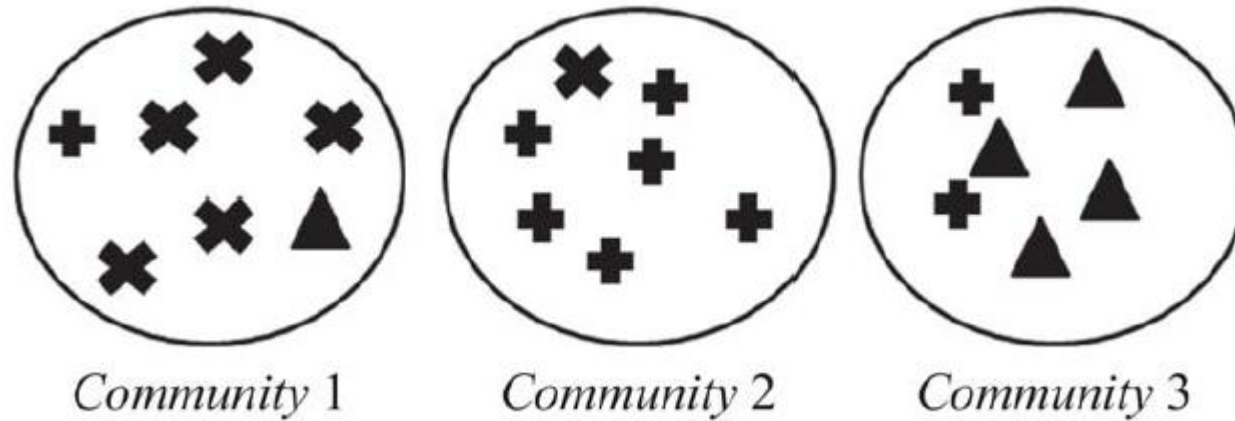
Metrics: pairs



For TN: compute the number of dissimilar pairs in dissimilar communities

$$\begin{aligned}
 TN = & \underbrace{\overbrace{(5 \times 6 + 1 \times 1)}^{x,+} + \overbrace{(1 \times 6 + 1 \times 1)}^{\Delta,+}}_{\text{Communities 1 and 2}} \\
 & + \underbrace{\overbrace{(5 \times 4 + 5 \times 2)}^{x,\Delta} + \overbrace{(1 \times 4 + 1 \times 2)}^{+,\Delta}}_{\text{Communities 1 and 3}} \\
 & + \underbrace{\overbrace{(6 \times 4 + 1 \times 2)}^{+,\Delta} + \overbrace{(1 \times 4)}^{x,\Delta}}_{\text{Communities 2 and 3}} = 104.
 \end{aligned}$$

Metrics: pairs



For FP, compute dissimilar pairs that are in the same community.

$$FP = \underbrace{(5 \times 1 + 5 \times 1 + 1 \times 1)}_{\text{Community 1}} + \underbrace{(6 \times 1)}_{\text{Community 2}} + \underbrace{(4 \times 2)}_{\text{Community 3}} = 25$$

For FN, compute similar members that are in different communities.

$$FN = \underbrace{(5 \times 1)}_{\times} + \underbrace{(6 \times 1 + 6 \times 2 + 2 \times 1)}_{+} + \underbrace{(4 \times 1)}_{\Delta} = 29$$

Metrics: pairs

Precision (P): the fraction of pairs that have been correctly assigned to the same community.

$$TP/(TP+FP)$$

Recall (R): the fraction of pairs assigned to the same community of all the pairs that should have been in the same community.

$$TP/(TP+FN)$$

F-measure

$$2PR/(P+R)$$

Evaluation without ground truth

- **Cluster Cohesion:** Measures how closely related are objects in a cluster
- **Cluster Separation:** Measure how distinct or well-separated a cluster is from other clusters
- Example: Squared Error

- Cohesion is measured by the within cluster sum of squares (SSE)

$$WSS = \sum_i \sum_{x \in C_i} (x - m_i)^2$$

- Separation is measured by the between cluster sum of squares

$$BSS = \sum_i |C_i| (m - m_i)^2$$

- Where $|C_i|$ is the size of cluster i

References

Partially taken from:

- R. Zafarani, M. A. Abbasi, and H. Liu, Social Media Mining: An Introduction
- Community Detection and Mining in Social Media. Lei Tang and Huan Liu